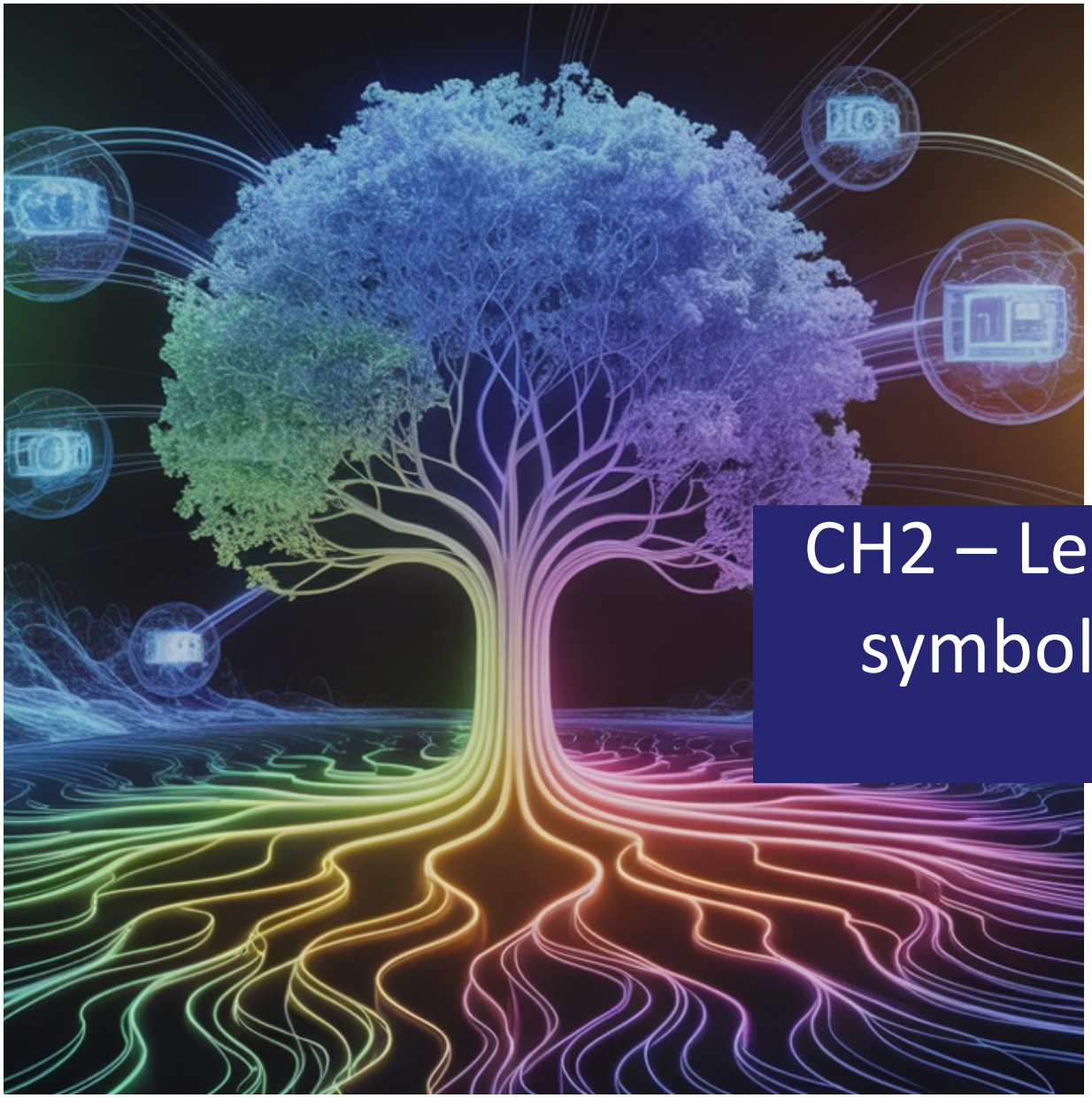


UNIVERSITE DE CORSE
Master Informatique
parcours DFS et DE
2ème année
2026-2027

CH2 – Les représentations de l'IA symbolique : des origines aux ontologies

Evelyne VITTORI
vittori_e@univ-corse fr

Partie I - Représenter le monde : des symboles aux vecteurs

CH1 – Représenter le monde en IA : un enjeu central

CH2 – Représentations en IA symbolique : des origines aux ontologies

CH3 – Représenter par des vecteurs : fondements des embeddings

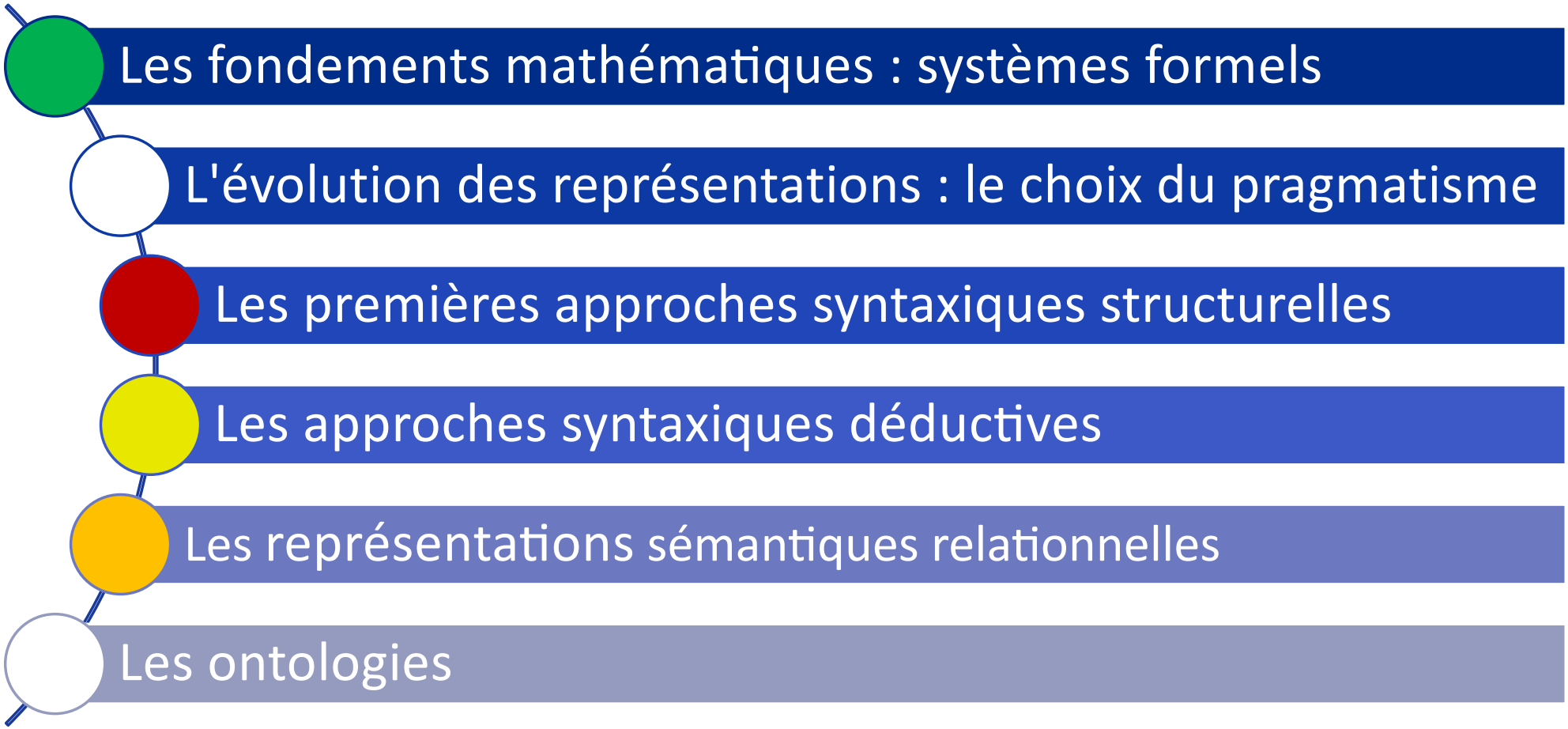
CH4 – Des embeddings aux BD Vectorielles

Objectifs du chapitre

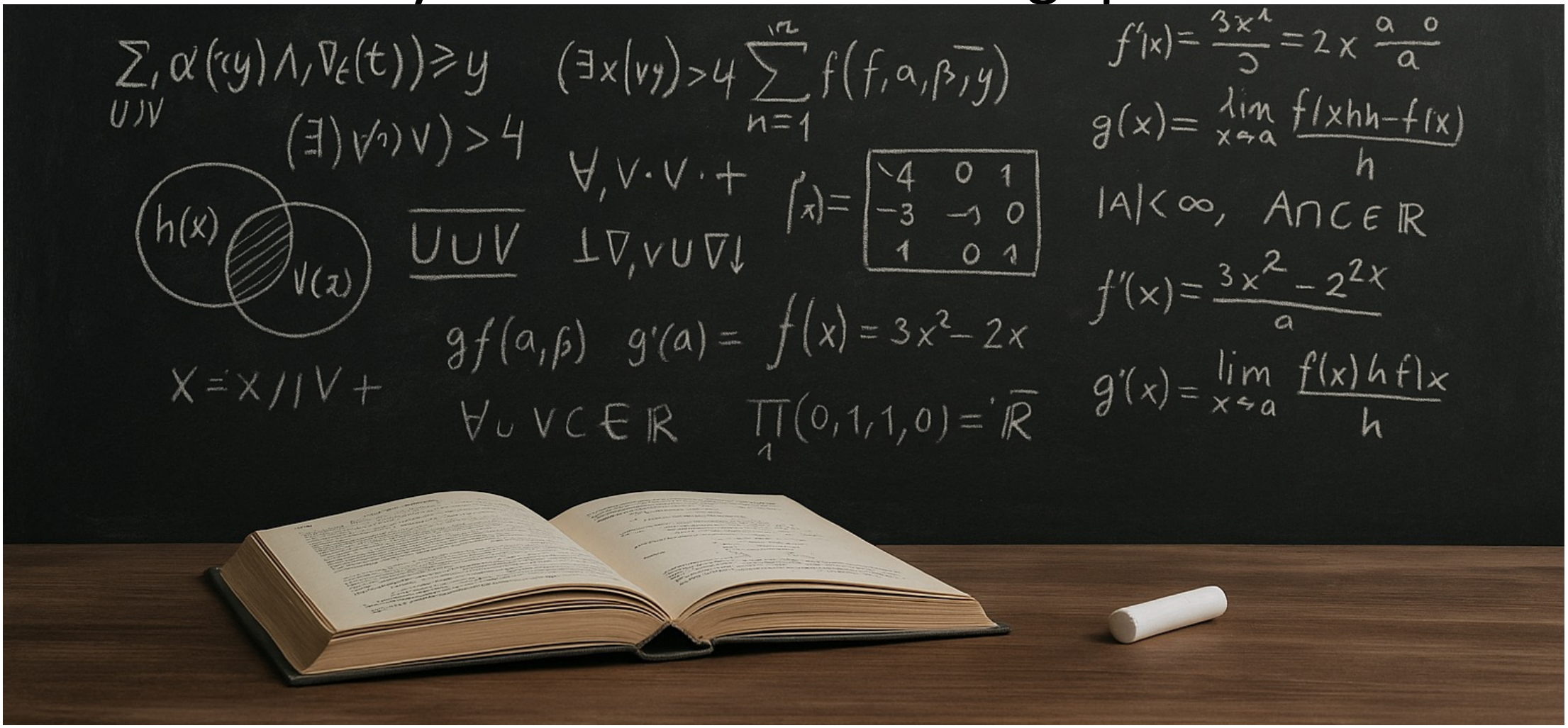
- Comprendre les bases logiques et formelles des représentations symboliques
- Identifier les principales approches syntaxiques et déductives de l'IA symbolique
- Situer les représentations relationnelles non formelles (thésaurus, graphes, frames)
- Découvrir les **ontologies** comme forme avancée de structuration du savoir

Activité guidée: Manipuler une ontologie simple avec **Protégé**

CH2 – Les représentations en IA symbolique: Des origines aux ontologies



1 - Les origines de l'IA symbolique : systèmes formels et logique

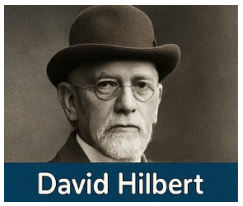


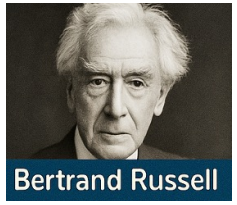
Les origines (avant 1930) : Les mathématiciens fondateurs de la logique formelle

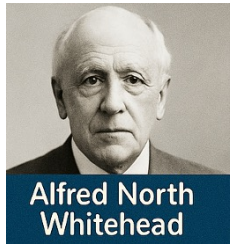
Leurs objectifs :

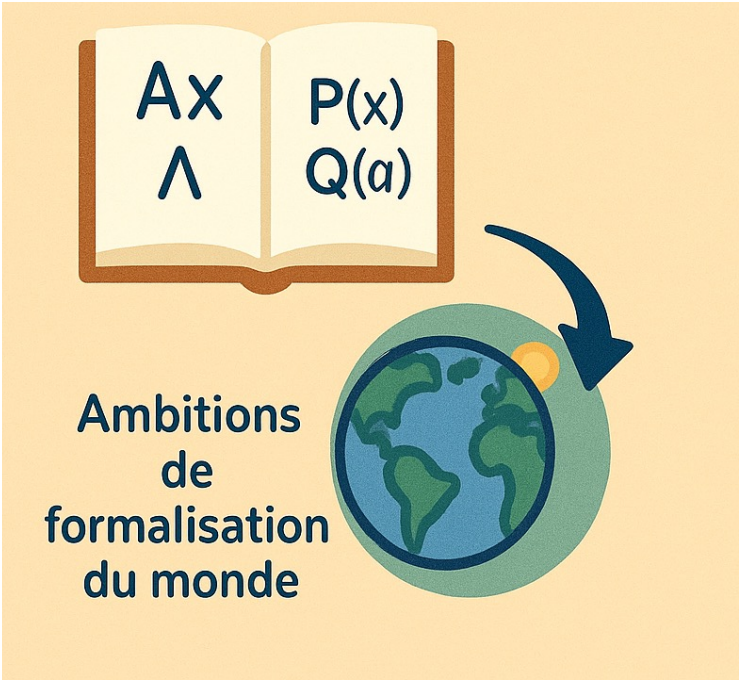
- **Éliminer l'ambiguïté** du langage naturel
  → Concevoir un langage formel
  "Begriffsschrift" (1879)
- **Formaliser le raisonnement**
  → Énoncer toutes les règles de déduction logiques
  "système axiomatique" (années 1920)
- Exprimer **tout le savoir mathématique**… voire scientifique
  → Construire des systèmes formels complets et cohérents
  "Principia Mathematica" (1910–1913)

Gottlob Frege


David Hilbert


Bertrand Russell


Alfred North Whitehead



Vers un système formel des mathématiques

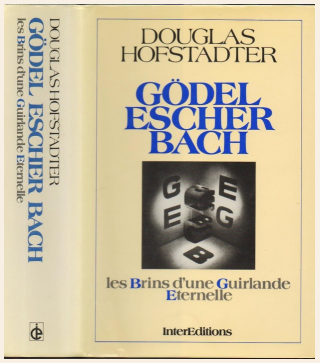
Qu'est ce qu'un système formel ?

- Alphabet — langage
  - ensemble des symboles permettant de construire les formules
- Formules Bien formées
- Axiomes
  - formules valides par définition
- Règles d'inférence
  - permettent de dériver de nouvelles formules : les théorèmes

Ex: Le système formel MIU (Hofstadter)

- Alphabet : M, I, U
- Formules Bien formées : toute suite de symboles de l'alphabet
- Axiomes : MI
- Règles d'inférence :

Soient f et g des FBF



1. fI → fIU (règle de production)
2. Mf →Mff (règle de production)
3. fIIIg → fUg (règle de réécriture)
4. fUUg → fg (règle de réécriture)

Déduction dans un système formel

Ex: Le système formel MIU (Hofstadter)

Alphabet : M, I, U

Formules Bien formées : toute suite de symboles de l'alphabet

Axiomes : MI

Règles d'inférence :

Soient f et g des FBF

1. fI → fIU (règle de production)
2. Mf →Mff (règle de production)
3. fIIIg → fUg (règle de réécriture)
4. fUUg → fg (règle de réécriture)

- Exercice

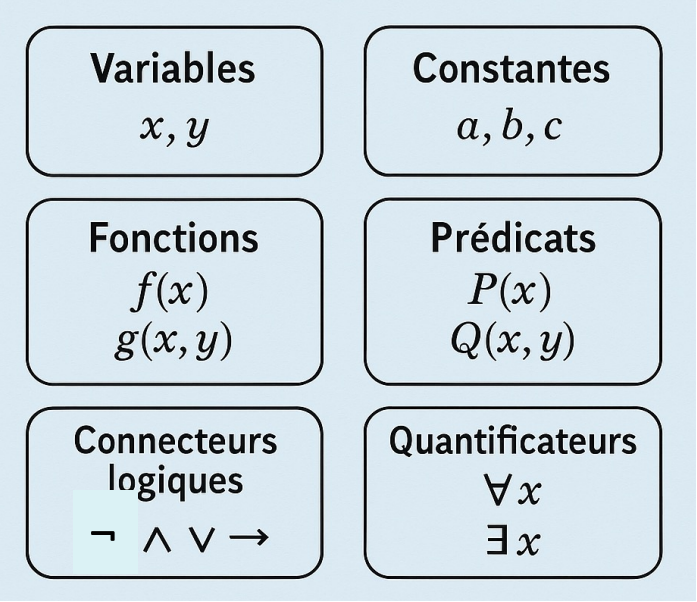
  Montrer que MUII est un théorème du système MIU

  - MI par l'axiome
  - MII par la règle 2
  - MIIII par la règle 2
  - MIIIIIIII par la règle 2
  - MIIIIIIIIU par la règle 1
  - MIIIIIUU par la règle 3
  - MIIIII par la règle 4
  - MUII par la règle 3

Le système formel de référence : La logique des prédicats

FOL First Order Logic

- **Alphabet**



- **Formules bien formées**

Exemple :
P(a), ¬Q(x), ∀x (Aime(x, marie) → Humain(x))

- **Axiomes logiques** : formules vraies par définition
  - (ex : ϕ → (ψ → ϕ)) Conservation d'une vérité

  Si ϕ est vrai,

  alors ψ implique ϕ

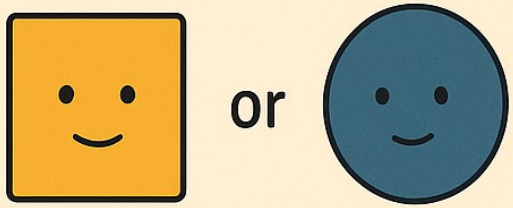
- **Règles d'inférence** : permettent de dériver de nouvelles formules
  - Exemple : **Modus ponens**

    Si ϕ et ϕ → ψ,

    alors on peut inférer ψ

Ce que FOL peut exprimer

$\forall x\,(P(x) \vee Q(x))$

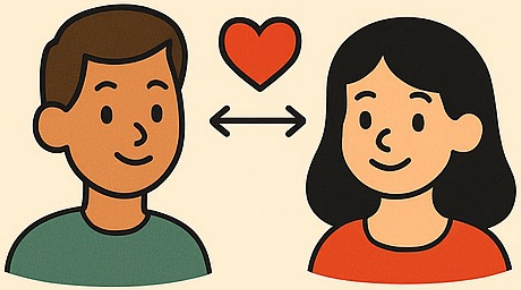


Tout a la propriété
P ou la propriété Q

$R(\text{Alice},x)$

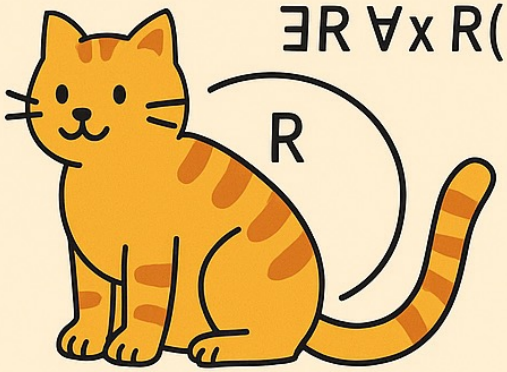
Alice est liée à quelqu'un

$\forall x\,(M(x) \rightarrow \exists y L(y,x) \wedge L(x,y))$



Être marié implique de
s'aimer l'un l'autre

$\exists R\,\forall x\,R(x,x)$



Il existe une relation
que tout a avec soi - même

Une déduction simple en FOL

On dispose de deux formules

$$\forall x\ (Chat(x) \rightarrow Mammifère(x))$$

Tous les chats sont des mammifères.

$$Chat(Félix)$$

Félix est un chat.

Félix

1 Instanciation de la règle

On remplace la variable x par l'individu Félix dans la première formule.

$$Chat(Félix) \rightarrow Mammifère(Félix)$$

2 Application du modus ponens

À partir de $Chat(Félix)$ et $Chat(Félix) \rightarrow Mammifère(Félix)$, on peut déduire $Mammifère(Félix)$.

$$\frac{Chat(Félix) \qquad \text{(fait)} \qquad Chat(Félix) \rightarrow Mammifère(Félix) \qquad \text{(règle instanciée)}}{Mammifère(Félix) \qquad \text{(conclusion)}}$$

On peut donc déduire la formule suivante :

$$Mammifère(Félix)$$

Le langage ne traduit pas toujours la réalité…

La Trahison des images (René Magritte 1928–1929)

La syntaxe ne suffit pas

Donner un sens aux symboles : Sémantique d'un système formel

Définir une interprétation = définir un monde possible

1. Définir un domaine d'objets
2. Définir une affectation des symboles (constantes, fonctions, prédicats)

Interprétation 1 : Yoga

Symbole	Interprétation
M	Méditation
I	Inspire
U	Posture Utthita (extension)

✅ *Vrai*

M → I → U

✅ *Vrai*

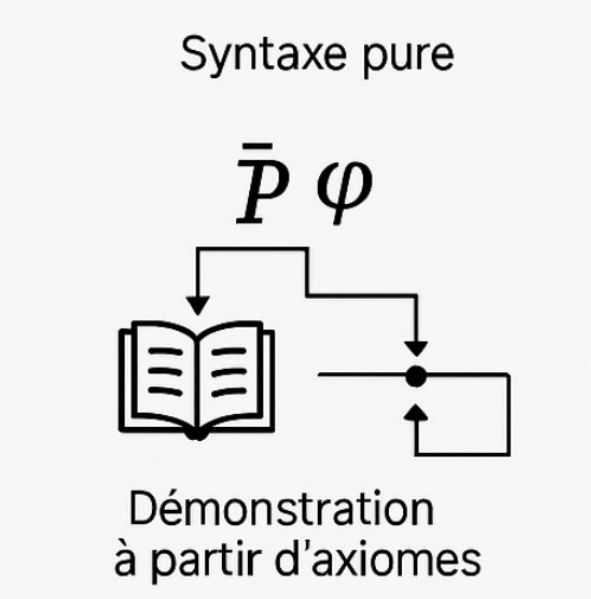
Interprétation 2 : Pluie

Symbole	Interprétation
M	Il pleut
I	Je prends mon parapluie
U	Je reste au sec

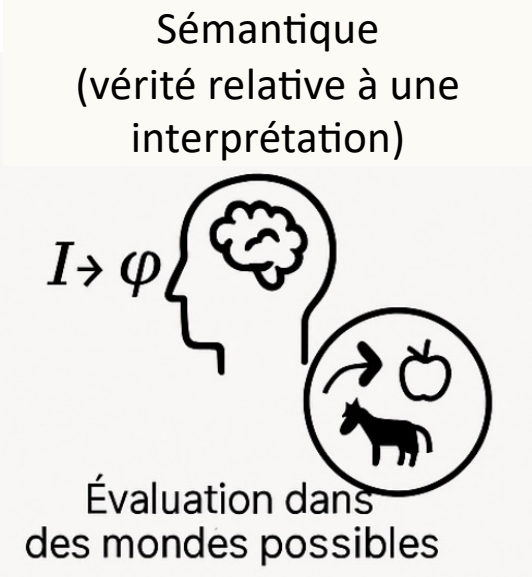
Prouvabilité (syntaxique) VS Vérité/Validité (sémantique)

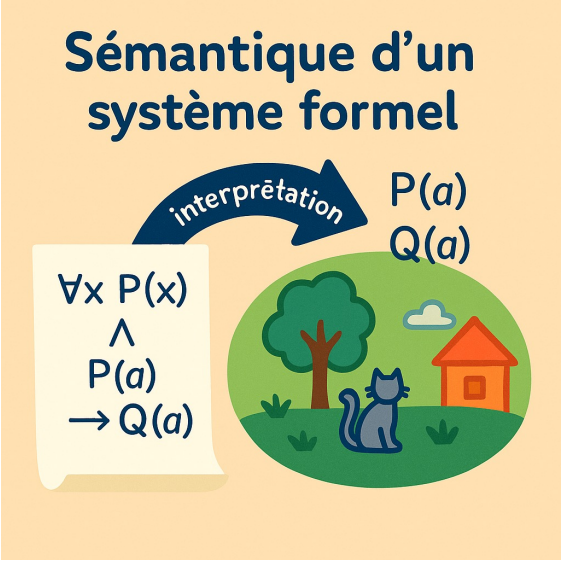
- Une formule est **prouvable** si elle est démontrable syntaxiquement

Théorème



- Une formule est "**vraie**" si elle est vérifiée dans une interprétation
- Une formule est "**valide**" si elle est vraie dans toutes les interprétations





Propriétés des systèmes formels

CORRECTION & COMPLÉTUDE
(adéquation syntaxe / sémantique)

Correction (soundness)

$$\vdash \varphi \quad \Rightarrow \quad \models \varphi$$

Toute formule démontrable est sémantiquement valide.

Complétude

$$\models \varphi \quad \Rightarrow \quad \vdash \varphi$$

Toute formule valide est démontrable.

Idée : la syntaxe et la sémantique coïncident : on démontre exactement toutes les vérités sémantiques.

COHÉRENCE

On ne peut pas démontrer à la fois une formule et sa négation.

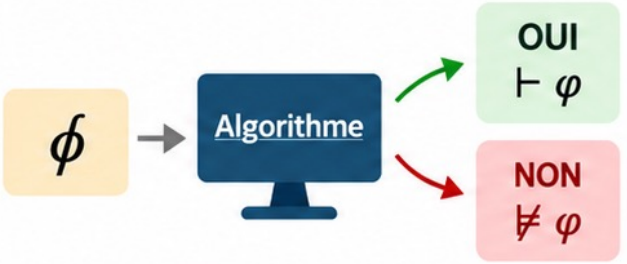
$$\neg \exists \, (\vdash \varphi \text{ et } \vdash \neg\varphi)$$

Le système ne permet pas de prouver des contradictions.

Idée : le système ne déduit pas tout : il n'y a pas de formules contradictoires démontrables.

DÉCIDABILITÉ

Il existe un algorithme qui, pour toute formule φ, détermine si elle est démontrable.



L'algorithme termine toujours et donne la bonne réponse.

Idée : la propriété « être démontrable » est entièrement automatisable.

Le théorème de complétude de FOL (Gödel, 1929)

Dans le système formel de la logique des prédicats (FOL)
Toute formule valide est prouvable (et inversement) : $\models \phi \Leftrightarrow \vdash \phi$

✅ Adéquation syntaxe / sémantique :

- Il existe une correspondance parfaite entre vérité dans tous les modèles (sémantique) et preuve à partir d'axiomes (syntaxe)

🧩 Base des systèmes automatisés de preuve :

- Permet de créer des moteurs d'inférence complets pour la logique du premier ordre
- 🤖 Fondement des systèmes experts, Prolog, reasoners OWL

Après la logique FOL : des théories pour formaliser toute la mathématique

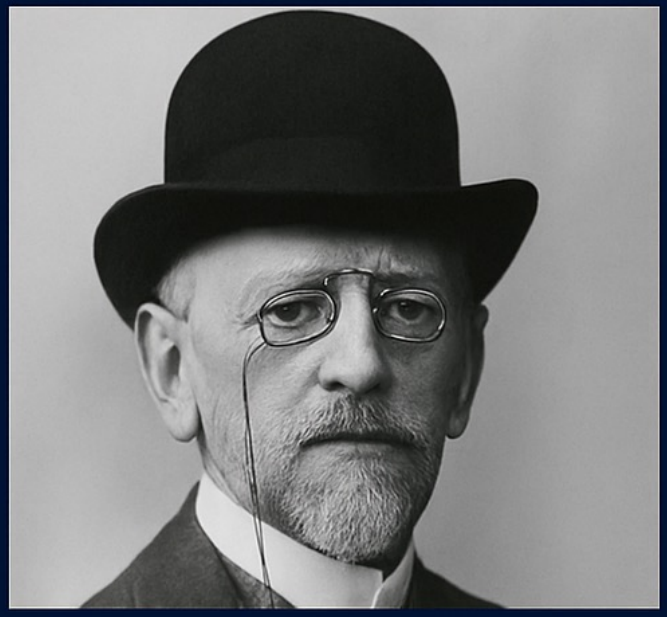
Théorie définie à partir de FOL:

- Langage formel spécifique défini à partir du langage de FOL et adapté au domaine : (variables, symboles, quantificateurs…)
- Axiomes spécifiques

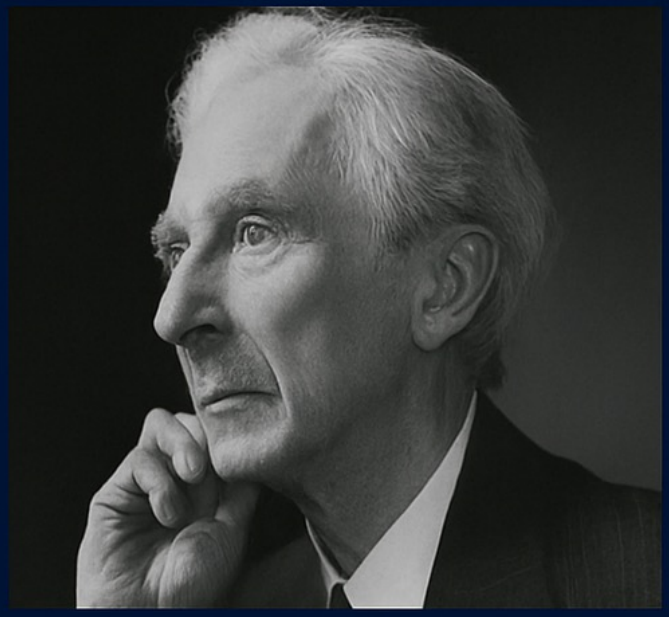
Mais ces théories ont des limites

L’espoir d’une IA symbolique

Nous croyions pouvoir tout modéliser à l’aide d’un système formel (axiomes + règles)



David Hilbert



Bertrand Russell

Le rêve du raisonnement mécanique… brisé

Formaliser toutes les mathématiques et mécaniser le raisonnement
(Hilbert, Russell et le projet formaliste)

David Hilbert
1862–1943

Bertrand Russell
1872–1970

INCOMPLÉTUDE
(Gödel, 1931)

Des vérités indémontrables

Dans un système formel suffisamment puissant, il existe des vérités qui ne peuvent pas être démontrées.

INDÉCIDABILITÉ
(Church & Turing, 1936)

Pas d'algorithme général

Il n'existe pas d'algorithme qui décide tous les problèmes formalisés.

(ex. validité en FOL)

COMPLEXITÉ
(années 1960–1970)

Un coût parfois prohibitif

Même lorsqu'un problème est décidable, le coût de son traitement peut devenir très élevé.

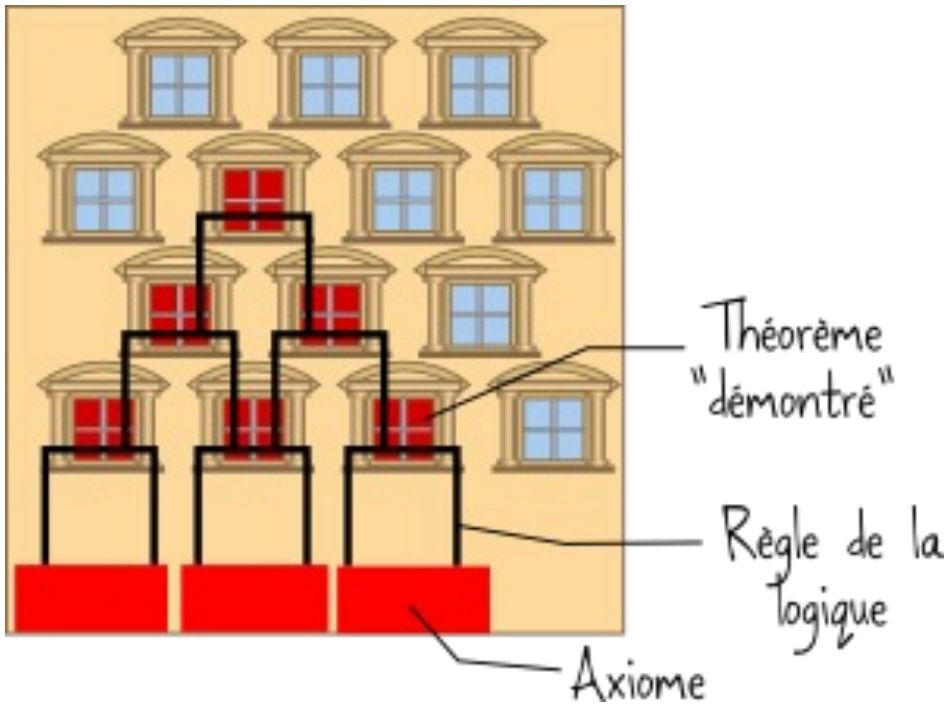
Formaliser ne **suffit** pas :
il faut aussi pouvoir **automatiser** le raisonnement de manière **calculable** et **exploitable**.

Théorème d'incomplétude (Gödel, 1931)

Dans toute théorie formelle suffisamment expressive pour formaliser l'arithmétique :

Il existe des énoncés vrais mais qu'il est impossible de démontrer

🔍 **INCOMPLÉTUDE**



Il est impossible de construire un système de preuve automatique complet pour les mathématiques

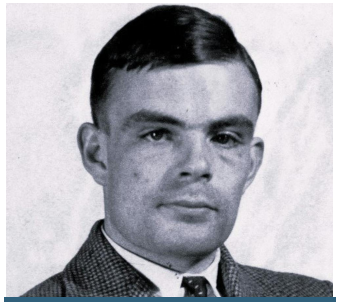
https://scienceetonnante com/blog/2013/01/14/le-theoreme-de-godel/

L'indécidabilité de FOL - Church et Turing (1936)

Peut-on automatiser complètement le raisonnement ?

NON
Il n'existe pas d'algorithme capable de déterminer, **pour toute formule de FOL**, si elle est valide.
INDECIDABILITE

Alonzo CHURCH


Alan TURING

FOL est complète…
mais indécidable.

Théorie de la complexité (1960 - 1970)

Être calculable ne signifie pas être résoluble en pratique

Combien de temps faut-il pour résoudre un problème ?

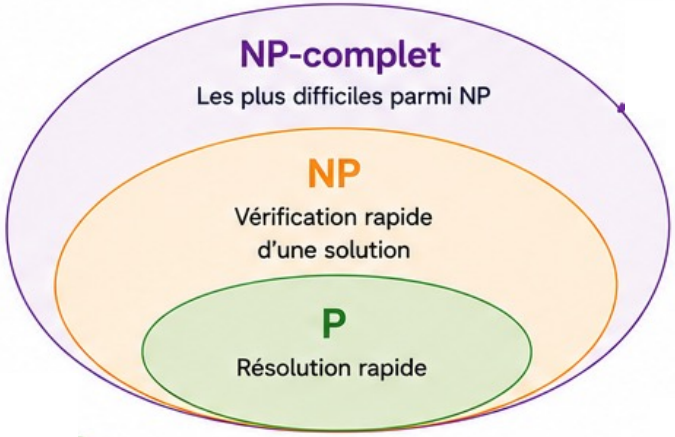
FACILES

Recherche dans une liste triée

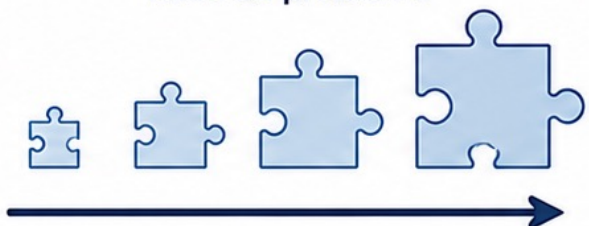
Plus court chemin (GPS)

Résolution efficace

Les grandes classes de complexité



Taille du problème



TRÈS DIFFICILES

Meilleure tournée d'un livreur

Planification complexe

Même les supercalculateurs deviennent insuffisants

Il reste encore des énigmes : où est la vérité ?

L'autoréférence peut produire des paradoxes

Le Paradoxe du menteur

« Cette phrase est fausse. »

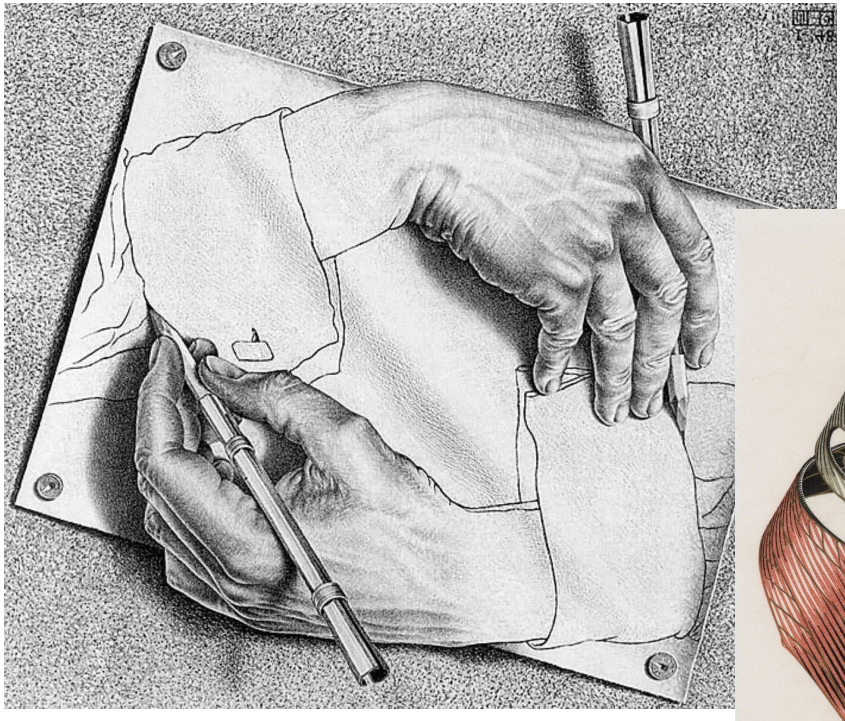
- Si elle est vraie → elle est fausse
- Si elle est fausse → elle est vraie

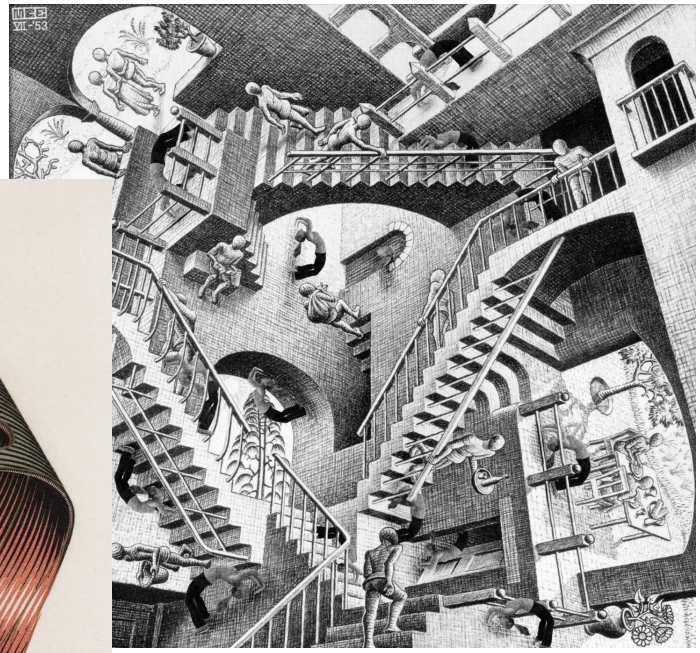
🔁 **Boucle logique** : l'énoncé ne peut être ni vrai ni faux sans contradiction

D'Épiménide au paradoxe du menteur

sources images: https://sympa-sympa com/articles/10-paradoxes-qui-embrouilleront-ton-esprit-pendant-un-moment-917610/

Des boucles logiques aux « boucles étranges »





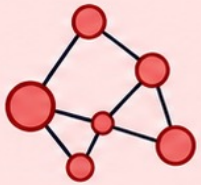
Conséquences pour l'IA symbolique : le choix du pragmatisme

Ne pas chercher à tout exprimer, mais
choisir des représentations permettant de raisonner efficacement

Pourquoi restreindre la logique du premier ordre ?

Logique du premier ordre (FOL)

Très expressive : on peut modéliser de nombreux raisonnements.
Mais le raisonnement automatique est en général indécidable
(et peut être très coûteux).



Restreindre le langage

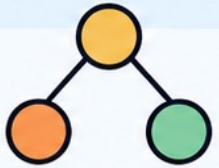
Clauses de Horn
(logique de programmation logique)

$$A_1 \wedge \ldots \wedge A_n \rightarrow B$$

Forme de logique restreinte,
adaptée à l'écriture de règles.

Applications : Prolog, systèmes experts,
raisonnement à base de règles.

Logiques de description (DL)



Forme de logique restreinte,
centrée sur les concepts et les relations.

Applications : ontologies, bases de connaissances,
raisonneurs (ex. OWL).

Moins d'expressivité → raisonnement plus facilement automatisable

Pour clarifier: une vidéo très pédagogique

- **Le théorème de Gödel | Voyages au pays des maths | ARTE**

- https://www youtube com/watch?v=Brj1LC42vLM

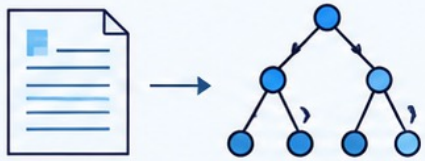
Évolution des représentations du savoir en IA symbolique

Du traitement de la structure des symboles à des représentations sémantiques formelles permettant le raisonnement.

1 Représentations syntaxiques structurelles

Représenter la structure des symboles (texte, langage)

- Codage linéaire du texte
- Grammaires formelles
- Arbres syntaxiques



Objectif : analyser et représenter la structure.

2a Représentations logiques et déductives

Représenter pour raisonner à l'aide de règles formelles.

- Logique des prédicats (FOL)
- Clauses de Horn
- Prolog
- Systèmes experts

$A \wedge B \rightarrow C$

2b Représentations sémantiques structurées

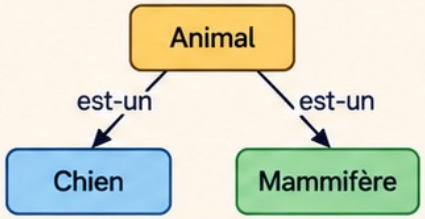
Représenter des connaissances sous forme de concepts et de relations.

- Réseaux sémantiques
- Frames (cadres)
- Thesaurus

3 Ontologies formelles

Une représentation sémantique formelle et standardisée, permettant le raisonnement automatique.

- Logiques de description (DL)
- OWL / RDF
- Ontologies
- Raisonneurs



Objectif : représenter formellement le sens et raisonner sur les connaissances.

1950	Analyse de la structure du langage	Développement des approches logiques et sémantiques	Formalisation des connaissances (ontologies)	Aujourd'hui

Ces trois grandes approches forment le fil conducteur de la suite du cours.

2 - Les représentations syntaxiques structurées

Une approche syntaxique minimale (1950…)

Codage historique du texte

- Chaînes de caractères (strings)
- **Chaque** caractère est représenté par un code numérique (ex : ASCII, Unicode)
- Le texte est une suite ordonnée de symboles, sans analyse sémantique

- ***Exemple***
  - Chaîne : "Chat"
  - Représentation ASCII :

[67, 104, 97, 116]

  - En binaire : 01000011 01101000 01100001 01110100

Représentation linéaire des mots
uniquement syntaxique, sans accès à la signification

Modélisations syntaxiques structurées (1956 – 1960)

- **Introduction des grammaires formelles**
  - Noam Chomsky, 1956 : grammaires génératives
  - Modèles utilisés pour reconnaître et générer des phrases grammaticales
- **Arbres syntaxiques**
  - Représentation visuelle de la structure grammaticale interne
  - Mise en évidence de la hiérarchie entre les éléments (sujet, verbe, complément…)

📌 *On retrouve ici des concepts familiers de la compilation : grammaires hors-contexte, analyse syntaxique, arbres de dérivation*

Représentation de phrases grammaticalement correctes, mais toujours sans accès à leur sens

Grammaires context-free (CFG)

- Règles syntaxiques décrivant la structure des phrases
- Utilisables pour la reconnaissance et la génération de phrases correctes

- Phrases conformes à cette grammaire :
  - *"Le chat dort"*
  - *"Un enfant regarde la souris"*
  - *"Un chien dort sur le tapis"*

Exemple de Règles

```
S → NP VP
NP → Det N | N
VP → V NP | V PP | V
PP → P NP
Det → "le" | "la" | "un" | "une"
N → "chat" | "chien" | "tapis" | "souris" | "enfant"
V → "dort" | "mange" | "regarde" | "aime"
P → "sur" | "sous" | "avec"
```

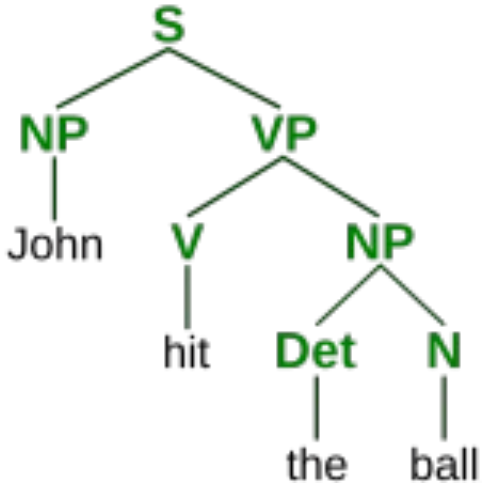
N (Noun) → Nom
V (Verb) → Verbe
P (Preposition) → Préposition
Det (Determiner) → Déterminant

SS (Sentence) → Phrase
NP (Noun Phrase) →
Groupe nominal (GN)
VP (Verb Phrase) →
Groupe verbal(GV)
PP (Prepositional Phrase) →
Groupe prépositionnel (GP)

Arbres syntaxiques

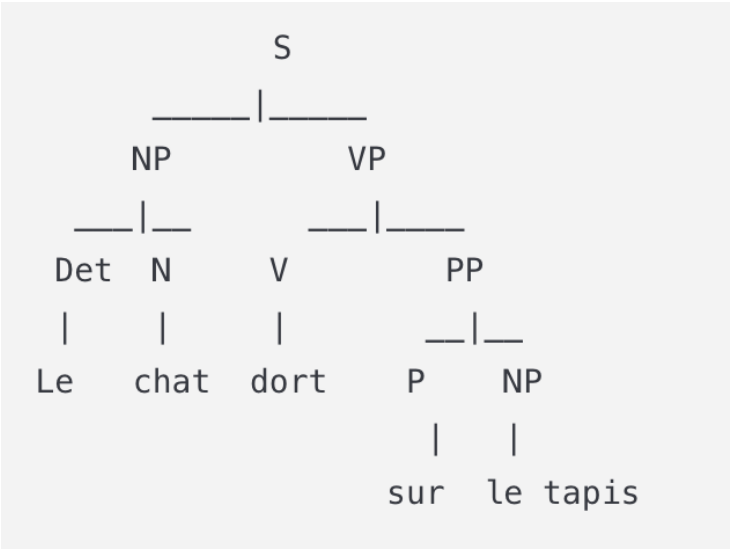
- L'arbre décrit la structure grammaticale
- La compréhension passe par l'analyse hiérarchique des constituants

Phrase analysée : "John hit the ball"



source: https://en wikipedia org/wiki/Parse_tree

Phrase analysée : "Le chat dort sur le tapis"

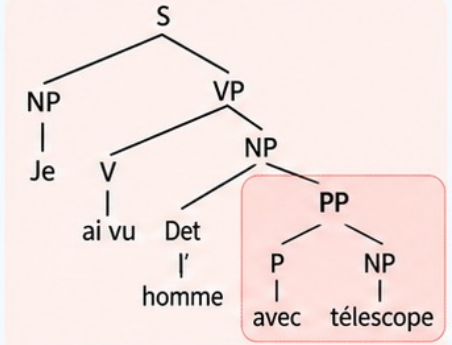


Les limites des représentations syntaxiques

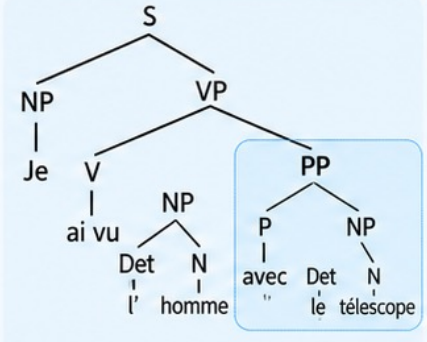
1 Ambiguïté syntaxique

Une même phrase peut admettre plusieurs analyses syntaxiques.

« J'ai vu l'homme avec le télescope. »



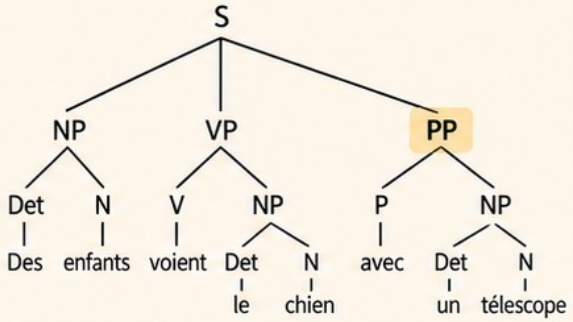
Interprétation 1 :
le télescope appartient à l'homme.
« J'ai vu l'homme qui avait un télescope. »



Interprétation 2 :
le télescope est l'instrument pour voir.
« J'ai utilisé un télescope pour voir l'homme. »

2 Explosion combinatoire

Dès que plusieurs analyses sont possibles, leur nombre peut croître très rapidement avec la longueur et la complexité des phrases.

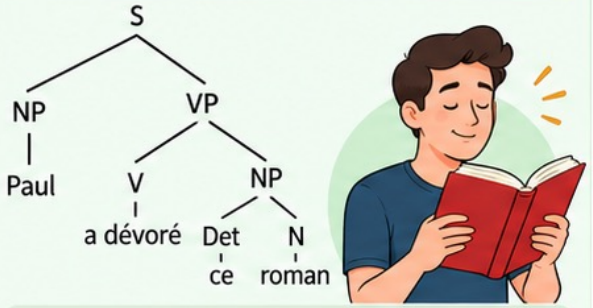


Le nombre d'arbres syntaxiques candidats **augmente** très vite (ambiguïtés, structures imbriquées), rendant l'analyse coûteuse.

3 La syntaxe ne suffit pas pour représenter le sens

Une analyse syntaxiquement correcte ne dit pas nécessairement ce que l'énoncé signifie ni quelles connaissances mobiliser.

« Paul a dévoré ce roman. »

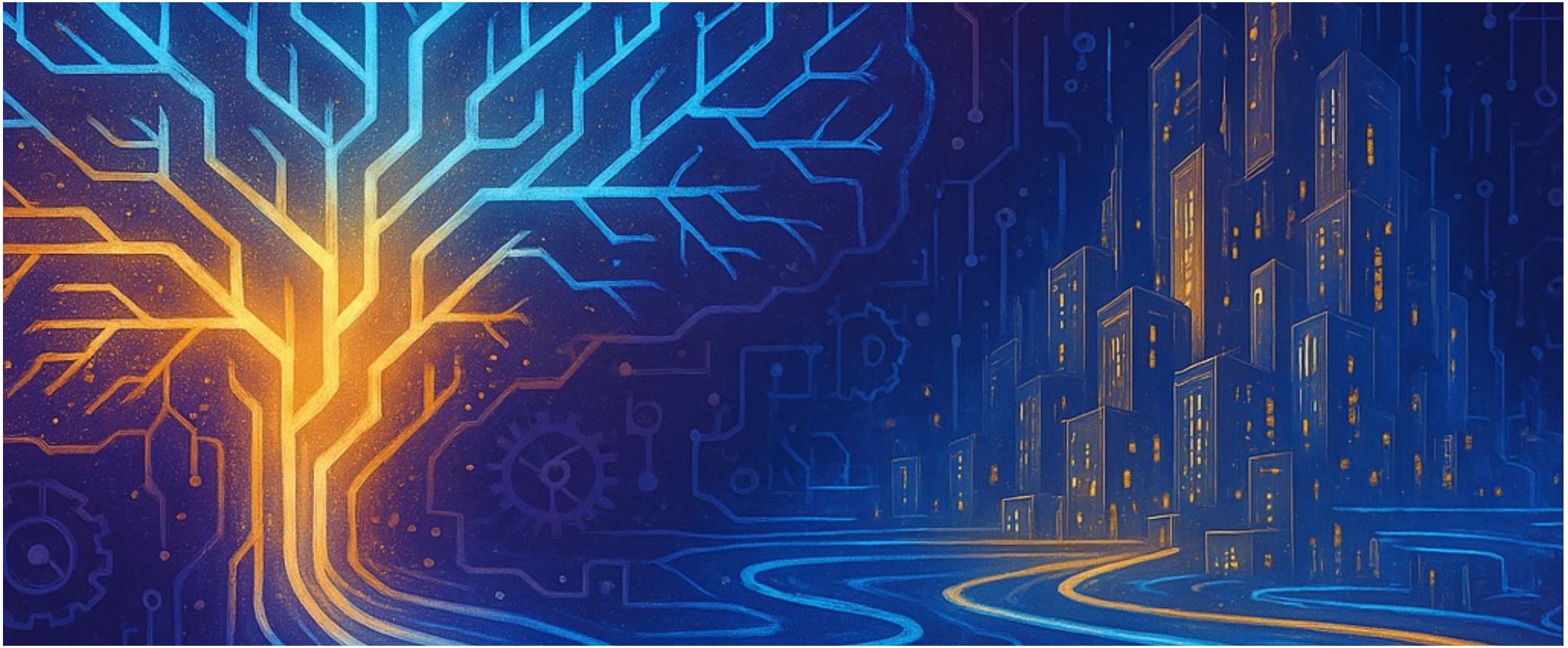


La structure est correcte, mais il faut des connaissances sur le monde pour comprendre qu'il s'agit d'une **métaphore** (lire avec beaucoup d'intérêt).

Il faut aller au-delà de la structure syntaxique : représenter explicitement les connaissances et permettre le raisonnement.

→ **Approches logiques et déductives**

3 - Les approches syntaxiques déductives



Approches syntaxiques déductives

- Restriction de la Logique des prédicats (FOL) aux clauses de Horn
  - Sous-ensemble complet
  - Semi-décidable
- Automatisation du raisonnement
- Langage PROLOG

Une clause de Horn contient **au plus un littéral positif**
Formule de de la forme
$(A \wedge B) \Longrightarrow C$ soit $\neg A \vee \neg B \vee C$
En prolog: règle C :- A, B

Systèmes experts

Comprendre et reproduire la structure du langage humain (syntaxe) par une modélisation explicite et manipulable

Qu'est-ce qu'un système expert ?

Ensemble de faits et de règles de la forme :

```
Si [une certaine condition est réalisée], alors
tirer [une certaine conclusion, avec un certain
degré de probabilité]
```

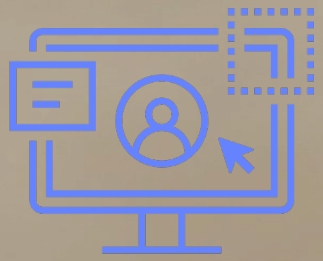
Les règles sont dictées par des experts du domaine c'est la phase **d'élicitation** ou **d'acquisition des connaissances**

Un **moteur d'inférence,** permet à partir des faits et des règles de **produire de nouveaux faits**

Ingénierie de la connaissance

Systèmes experts

Utilisateur
non expert



Moteur
d'inférence

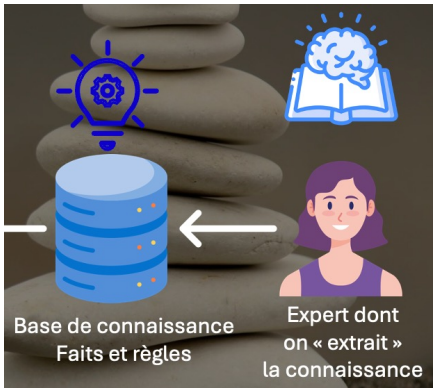
Base de connaissance
Faits et règles

Expert dont
on « extrait »
la connaissance

Aide à la décision

©Creartive FlatIcon

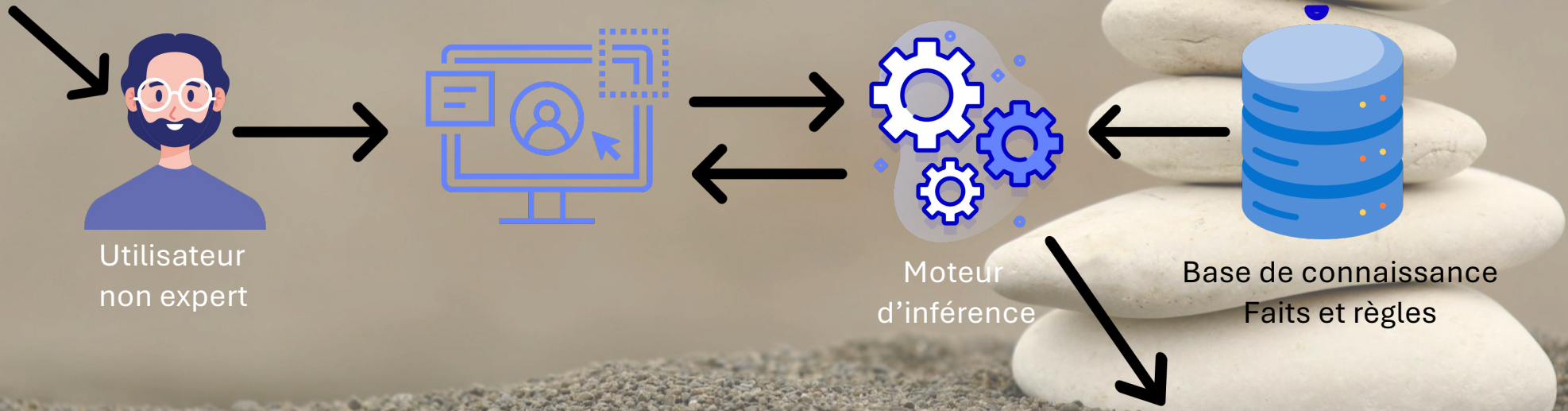
Systèmes experts, extraction des connaissances



- **Règle 1 : Si** la température du patient est supérieure à 38°C **et s'il** a des frissons, **alors** le patient pourrait avoir une infection
- **Règle 2 : Si** un patient est âgé de plus de 60 ans **et** présente une toux persistante, **alors** il pourrait avoir une pneumonie
- **Règle 3 : Si** le patient a des douleurs abdominales **et** des nausées, **alors** il pourrait souffrir d'une gastrite
- **Règle 4 : Si** le patient a des douleurs abdominales aigües, **et** localisées dans la partie inférieure droite, **alors** il pourrait souffrir d'une appendicite

Systèmes experts, inférence de nouveaux faits

Fait 1 : Le patient est âgé de 45 ans
Fait 2 : Le patient présente une température de 39°C
Fait 3 : Le patient souffre de nausées
Fait 4 : Le patient souffre de douleurs abdominales à droite



Utilisateur non expert

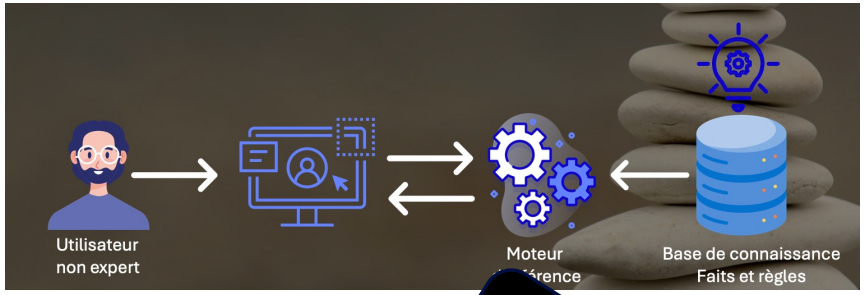
Moteur d'inférence

Base de connaissance
Faits et règles

Cela pourrait être une appendicite...
Allez consulter un médecin pour des examens complémentaires

©Creartive FlatIcon

Systèmes experts,
Utilisation du moteur d'inférence



- **Fait 1 :** Le patient est âgé de 45 ans
- **Fait 2 :** Le patient présente une température de 39°C
- **Fait 3 :** Le patient souffre de nausées
- **Fait 4 :** Le patient souffre de douleurs abdominales à droite

Good Old Fashion AI (GOFAI)

- **Règle 1 : Si la température du patient est supérieure à 38°C et s'il a des frissons, alors le patient pourrait avoir une infection**
- **Règle 2 : Si** un patient est âgé de plus de 60 ans **et** présente une toux persistante, **alors** il pourrait avoir une pneumonie
- **Règle 3 : Si** le patient a des douleurs abdominales **et** des nausées, **alors** il pourrait souffrir d'une gastrite
- **Règle 4 : Si le patient a des douleurs abdominales** aigües, **et localisées dans la partie** inférieure **droite, alors il pourrait souffrir d'une appendicite**

Règles non connues de l'utilisateur non expert, mais utilisables a posteriori pour **comprendre** les conclusions du système

Explicabilité

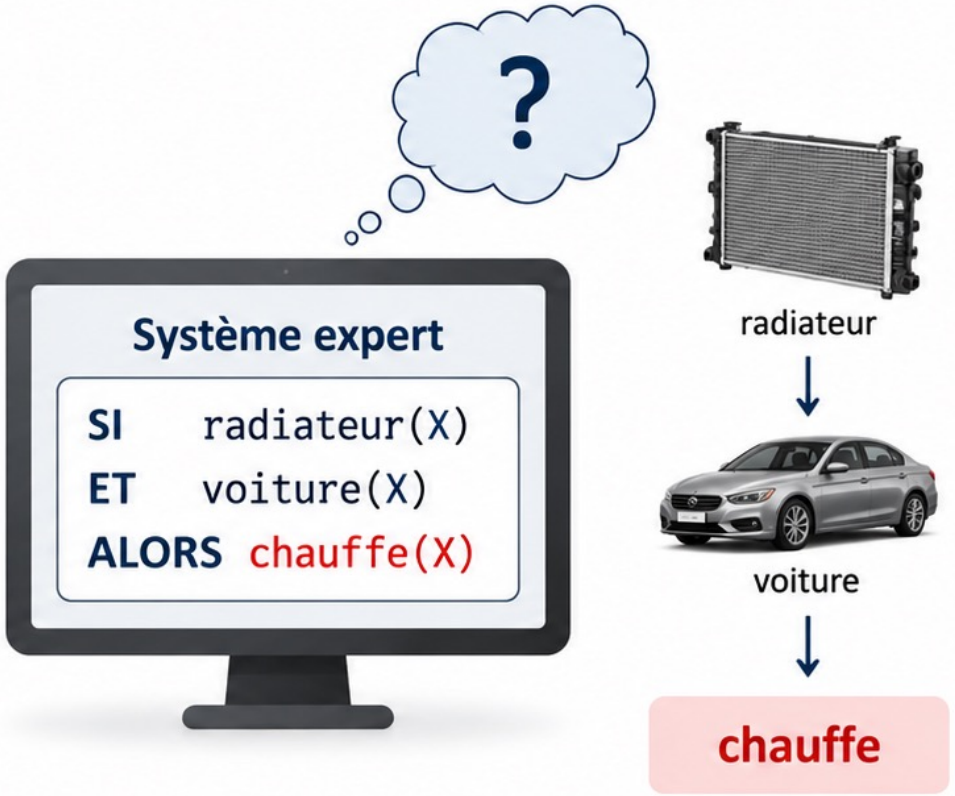
Mais le système ne « comprend » rien !

- Il manipule des **symboles** selon des **règles**
- Il raisonne sur **leur forme**, pas sur **leur sens**
- **La signification vient de l'expert**
  qui a construit la base de connaissances.

Exemple :

`radiateur(voiture)` → `chauffe`

Pour le système, « radiateur », « voiture » et « chauffe » sont avant tout des **symboles** reliés par une règle.



→ **Une intelligence explicite et explicable…**
mais sans compréhension du monde.

Pourquoi cette approche a échoué dans les années 1990 : 2ème hiver de l'IA

- *Expressivité limitée et manque de naturel*

  → Difficile de représenter des connaissances floues, incomplètes, contextuelles ou évolutives

- *Rigidité des règles*

  → Tout savoir devait être formalisé sous forme de faits ou de règles logiques, ce qui demande un effort considérable

- *Problèmes d'échelle*

  → Explosion combinatoire : la taille des bases de règles devient vite ingérable

- *Dépendance à l'expert humain*

  → Les règles sont écrites à la main : risque de subjectivité, coût élevé, couverture partielle

Nécessité de descriptions plus naturelles et plus souples

Les critiques philosophiques

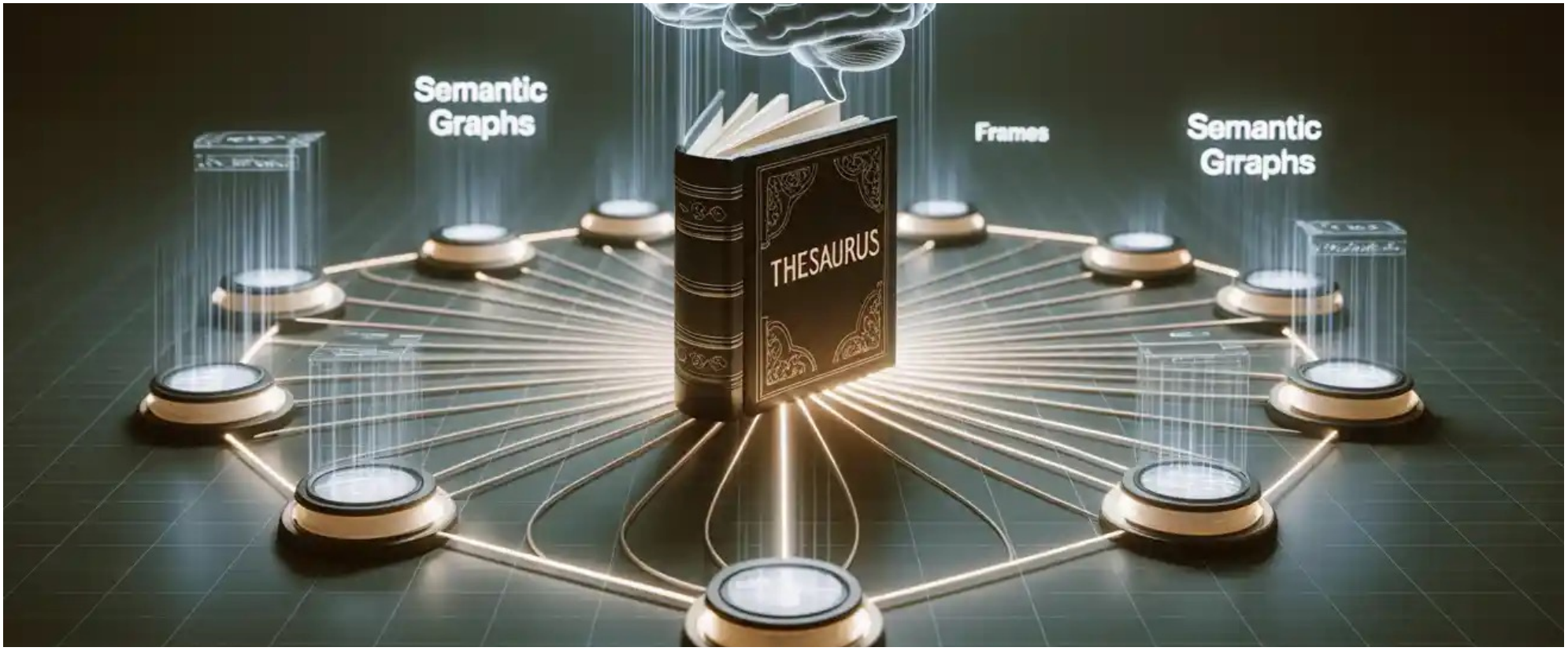
Exemples : Critiques publiées en 1964 et 1972 par le philosophe Hubert Dreyfus

- « *Les premiers pas de l'IA dans les années 50/60 sont comparables au fait de grimper dans un arbre pour atteindre la lune* »

  What computers can't do (1972, traduit en français sous le titre "Intelligence Artificielle, mythes et limites")

L'intelligence ne peut être réduite à une manipulation de symboles !

4 - Les représentations sémantiques relationnelles



Des règles à une représentation structurée des connaissances

Systèmes experts

Connaissances = faits + règles

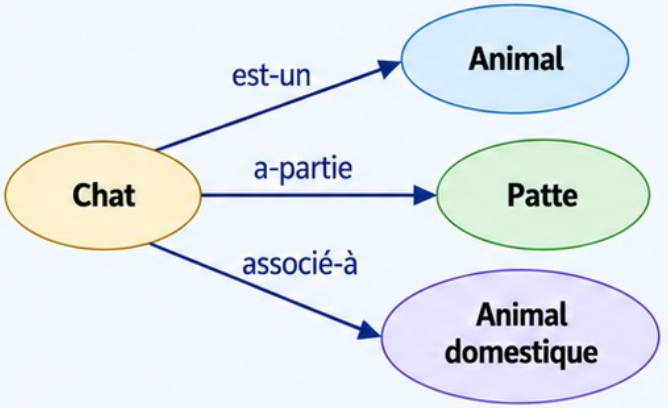
SI symptômes = fièvre ∧ toux
ALORS diagnostic = grippe

- Connaissances exprimées sous forme de règles.
- Raisonnement par application de règles.
- Efficaces dans des domaines restreints et bien formalisés.

Vers une représentation plus directe de l'organisation des connaissances

Représentations sémantiques relationnelles

Connaissances = concepts + propriétés + relations



- Les connaissances sont organisées en concepts et en relations.
- Représentation plus naturelle et plus souple que des règles.
- Permet de structurer un domaine de connaissances (vocabulaire, attributs, relations).
- Base pour une modélisation plus riche et plus formalisée.

**On ne représente plus seulement des règles permettant de raisonner :
on décrit explicitement les concepts d'un domaine et les relations qui les organisent.**

Thesaurus : Indexer le sens avec des listes de mots et de relations

- Notion de thésaurus : mots clés + relations sémantiques
- Utilisés dans les systèmes de recherche documentaire
- Définition explicite par l'expert des relations entre concepts :
  - synonymie,
  - généralisation/spécialisation
  - association thématique (ex : "chat" relié à "animal domestique")

Exemple de thesaurus simplifié

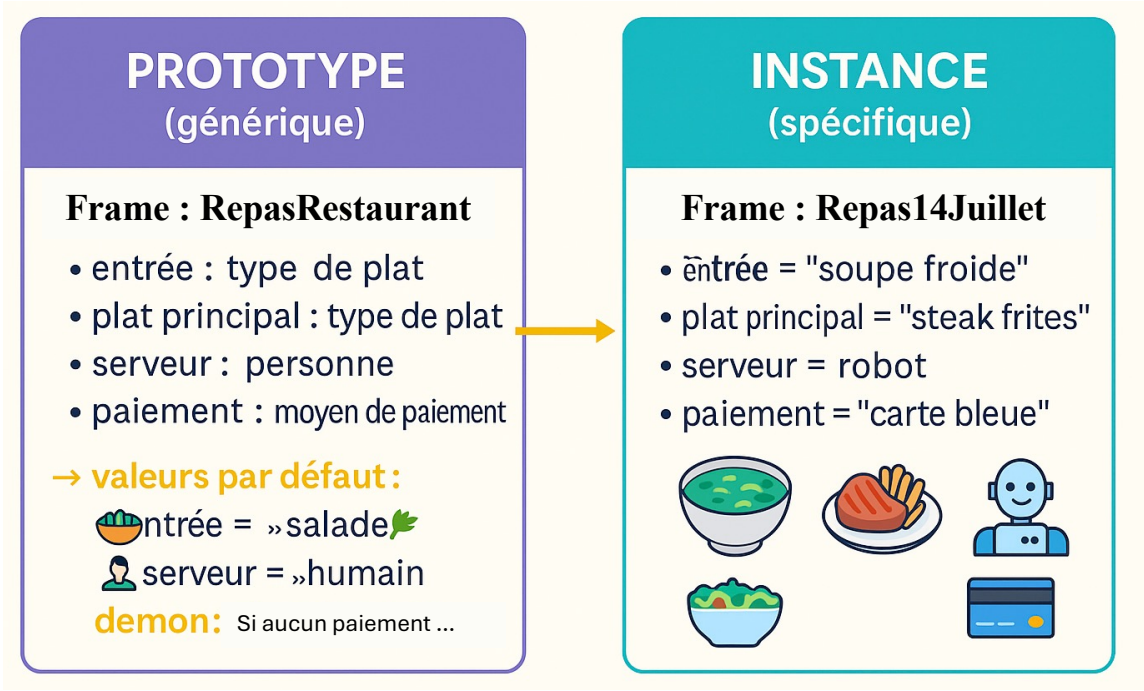
```
Chat
├── Terme générique : Mammifère
├── Synonymes : Félin domestique
├── Termes spécifiques : Chat siamois, Chat persan
├── Associé à : Souris, Litière, Ronronnement
```

Nombre de relations limité, pas d'axiomes formels permettant le raisonnement automatique

Frames (Minsky 1974)

- Structures de **représentation des connaissances** avec procédures
  - Nom du frame
  - Slots : attributs
  - Aspects de description :
    - types, Valeurs par défaut, restrictions, contraintes …
  - Procédures attachées (démons) : actions automatiques associés aux slots
  - Relations : is-a, part-of, instance-of
- Types de frames
  - Prototype : modèle générique
  - Instance : occurrence concrète

- Exemple

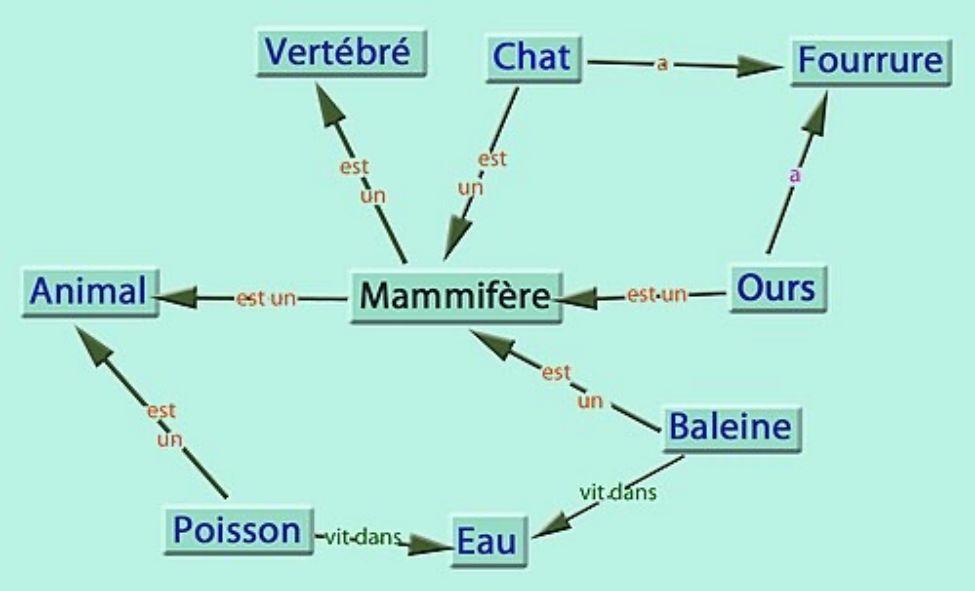


Faible formalisation: inférences difficiles

Article de référence [Minsky 1974] : https://courses media mit edu/2004spring/mas966/Minsky%201974%20Framework%20for%20knowledge pdf

Les réseaux sémantiques : premiers modèles graphiques des connaissances

- Structure de graphes avec
  - **nœuds = concepts** et
  - **arcs = relations sémantiques** (is-a, part-of, etc)
- Représentation visuelle intuitive des relations entre concepts
- Limites : **formalisation limitée**

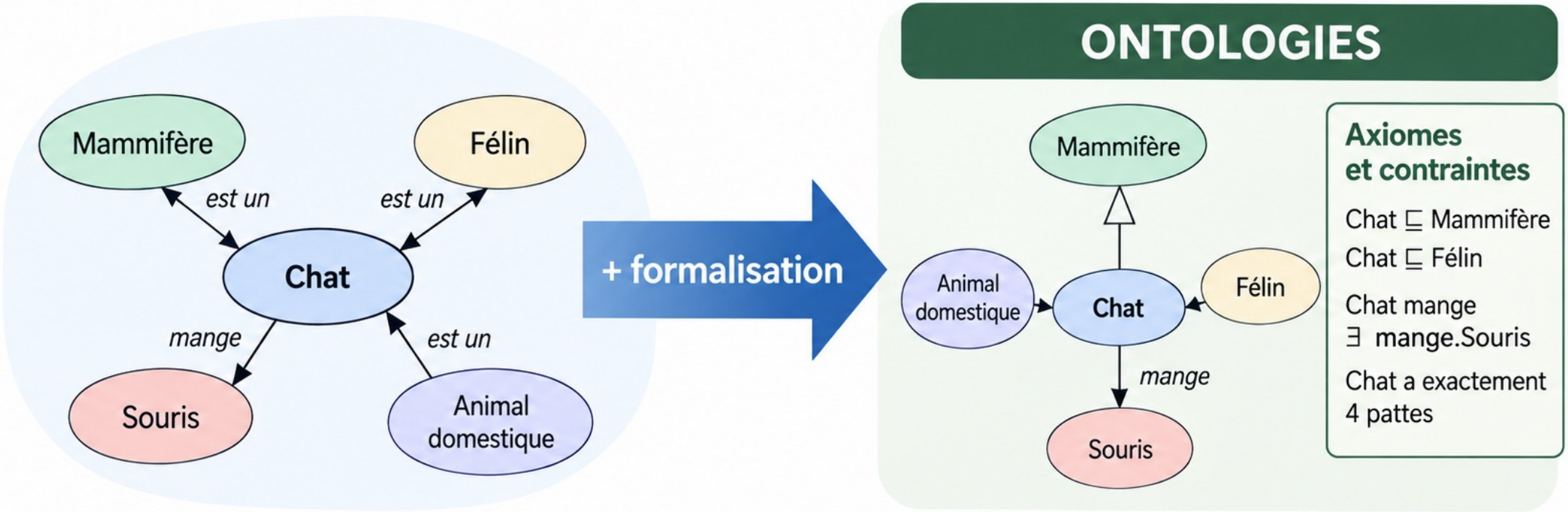


source: https://fr wikipedia org/wiki/R%C3%A9seau_s%C3%A9mantique

Etape importante pour passer d'une simple liste de termes (thesaurus) à une modélisation plus riche (ontologie)

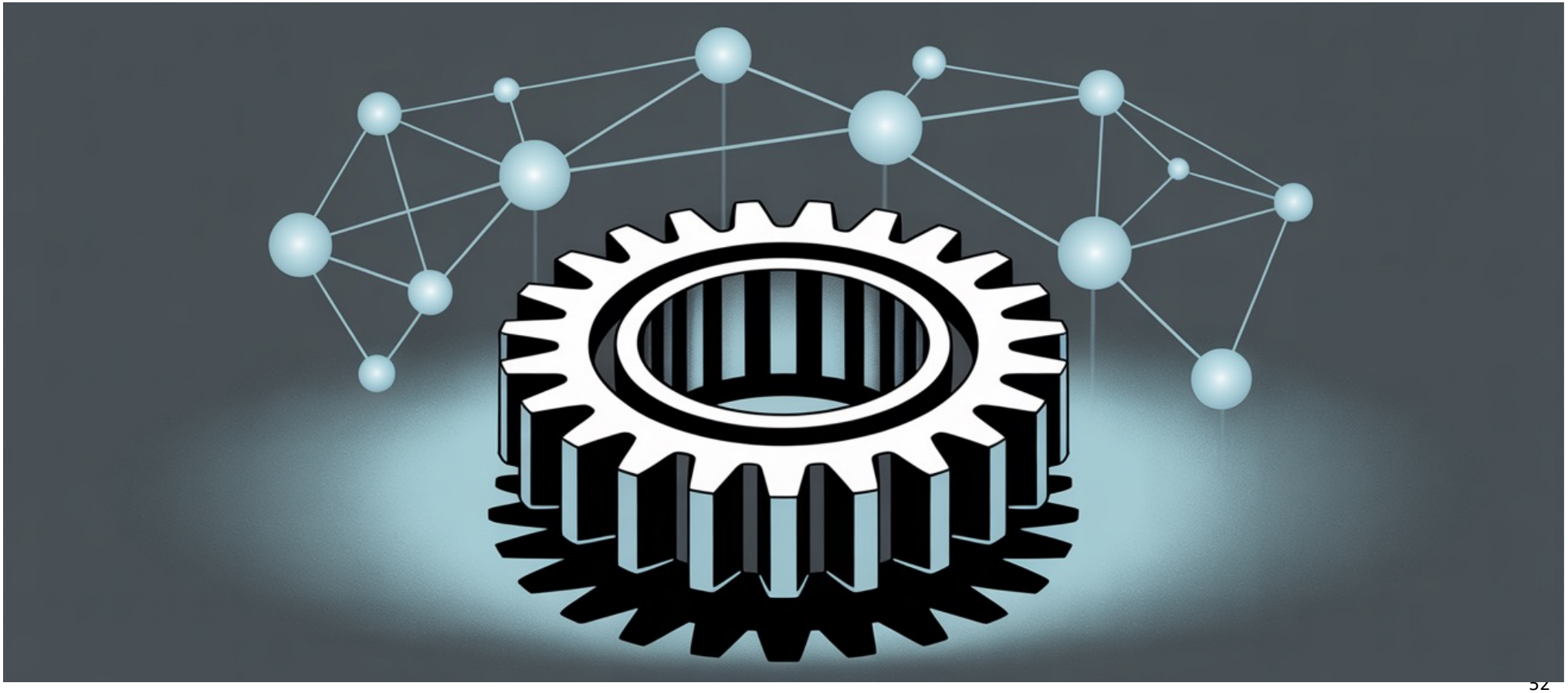
Ref: J F Sowa, "Chapter 5 Conceptual Graphs," in *Foundations of Artificial Intelligence*, vol 3, Elsevier, 2008, pp 213–237 doi: 10 1016/S1574-6526(07)03005-2

Des relations entre concepts…
à une représentation formelle du sens



Peut-on formaliser le sens pour permettre à la machine de raisonner ?

5 - Les Ontologies formelles : Aller plus loin dans la modélisation des liens entre concepts



Découvrir les Ontologies formelles

Comprendre les ontologies formelles

1

- Origines et Définition
- Composants d'une ontologie
- Ontologies et Web Sémantique
- Applications et Types d'ontologies
- Objectifs

Formaliser et raisonner avec les ontologies

2

- Formalismes RDF, RDFS
- **Logiques** de description
- Langage OWL
- Langage de requête SPARQL

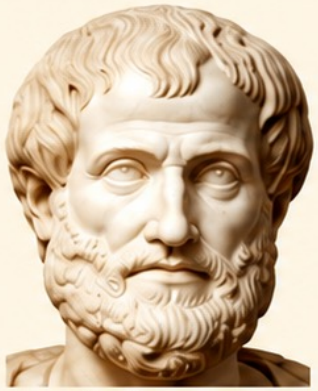
Construire et raisonner avec une ontologie : Protégé

3

Exploiter une ontologie dans une application (exemple)

4

Origines des Ontologies



Aristote
Métaphysique
(IVe siècle av. J.-C.)

*« Il y a une science qui étudie **l'être en tant qu'être**, et les attributs qui lui appartiennent essentiellement. »*
(Métaphysique, livre Γ)

- Qu'est-ce qui existe ?
- Quelles sont les catégories de l'être ?
- Quelles relations entre ces catégories ?

IVe siècle av. J.-C.
Réflexions d'Aristote sur l'être et ses catégories

Début XVIIe siècle
Apparition du terme « ontologie » en philosophie

XVIIIe siècle
Diffusion du terme (notamment avec Christian Wolff)

En philosophie, l'ontologie désigne l'étude de ce qui existe, de ses catégories et de leurs relations.

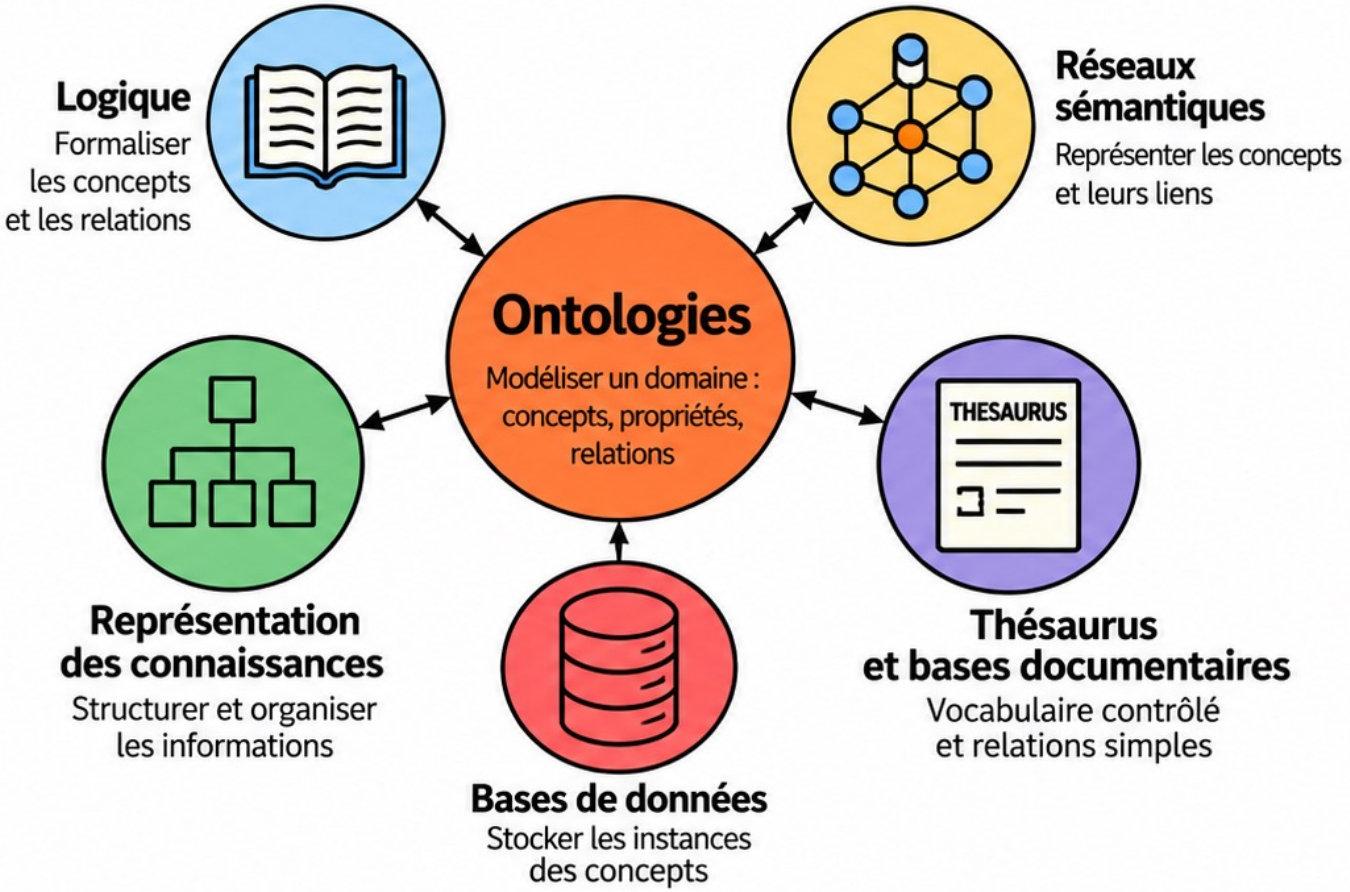
De l’ontologie philosophique aux ontologies informatiques

Idée de départ (philosophie)

- Qu’est-ce qui existe ?
- Quelles sont les catégories de l’être ?
- Quelles relations entre ces catégories ?

Transposition en informatique

- Représenter explicitement les concepts d’un domaine
- Définir leurs propriétés
- Modéliser leurs relations
- Les formaliser pour pouvoir les exploiter par la machine



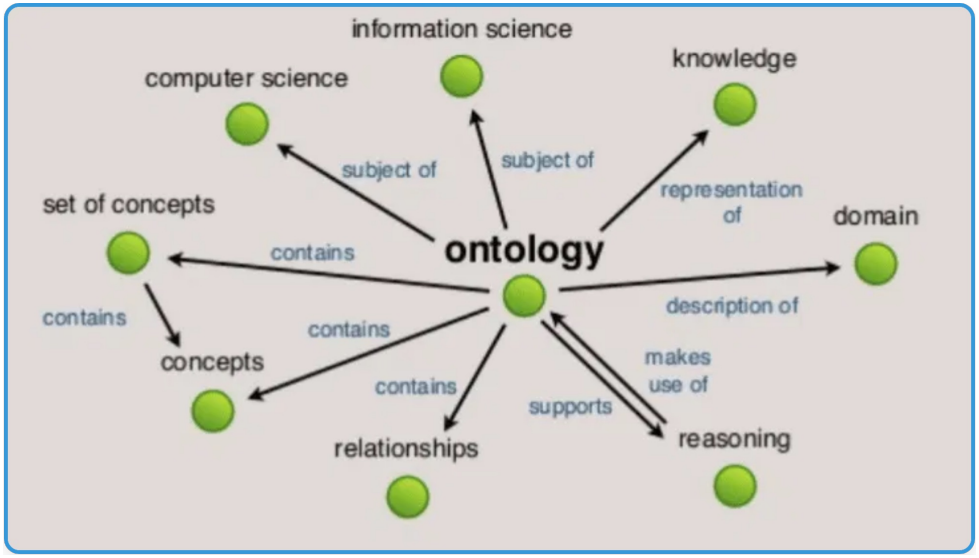
Une même idée de départ : décrire les entités et leurs relations, mais dans un modèle explicite et formalisé exploitable par la machine.

Qu'est-ce qu'une ontologie ?

- **Définition (Gruber, 1993)**

"Une ontologie est une spécification explicite d'une conceptualisation"

Tom Gruber
Co-createur de Siri



- **Modèle formel** décrivant explicitement des concepts, leurs relations, propriétés, axiomes et contraintes logiques

Composants Clés d'une Ontologie

TBox (Terminological Box)

Classes (concepts)

- Ensembles, collections ou types d'objets qui partagent des propriétés communes
- *Exemple : "Personne", "Bâtiment", "Œuvre d'art"*

Propriétés (relations)

- Liens entre individus ou classes, ou attributs caractérisant les individus
- Organisation hiérarchique des classes
- *Exemple : "estAuteurDe", "estSituéÀ", "aUneHauteurDe"*

Axiomes (Règles)

- Assertions, contraintes et règles logiques qui définissent les relations entre les concepts et permettent le raisonnement

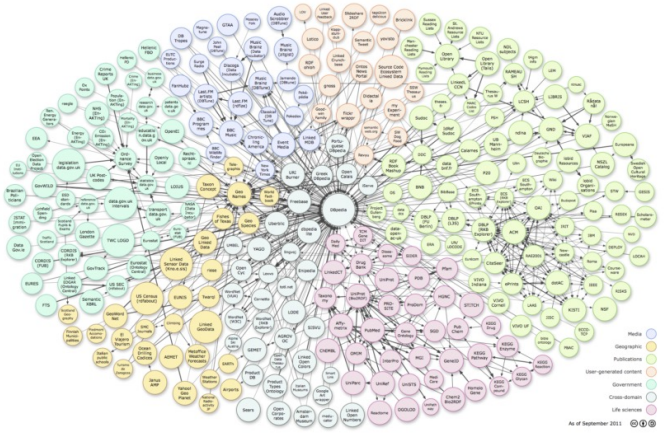
ABox (Assertional Box)

Individus (instances)

- Objets concrets ou abstraits qui représentent les entités spécifiques du domaine
- *Exemple : "Socrate", "Tour Eiffel", "Mona Lisa"*

Ontologies et Web Sémantique

- Web classique
  - Recherche par mot clés simples
- Web Sémantique (2000…)
  - Web + ontologies



Ontologies pour

- Décrire les données publiées sur le web
- Permettre le raisonnement sur ces données
  -> recherches enrichies et pertinentes

Différents types d'ontologies

Plus général
(très abstrait)

Plus spécifique
(plus concret)

Ontologies de haut niveau

Décrivent des concepts très généraux et abstraits, indépendants d'un domaine spécifique.

Temps • Espace • Objet • Événement • Propriété • Relation

Exemples : DOLCE, BFO, SUMO

Ontologies de domaine

Modélisent un domaine de connaissance particulier.

Médecine • Biologie • Finance • Écologie • Droit ...

Exemples : SNOMED CT (médecine), Gene Ontology (biologie)

Ontologies d'application

Les plus spécifiques, développées pour une application particulière. Combinent des concepts de plusieurs ontologies.

Exemple : ontologie pour un système de recommandation

Réutilisable dans différents domaines

Réutilisable dans différents domaines

Spécialisation pour une application

Ontologies de tâche

Décrivent les concepts et relations liés à une tâche spécifique, indépendamment du domaine.

Diagnostic • Planification
Classification • Raisonnement ...

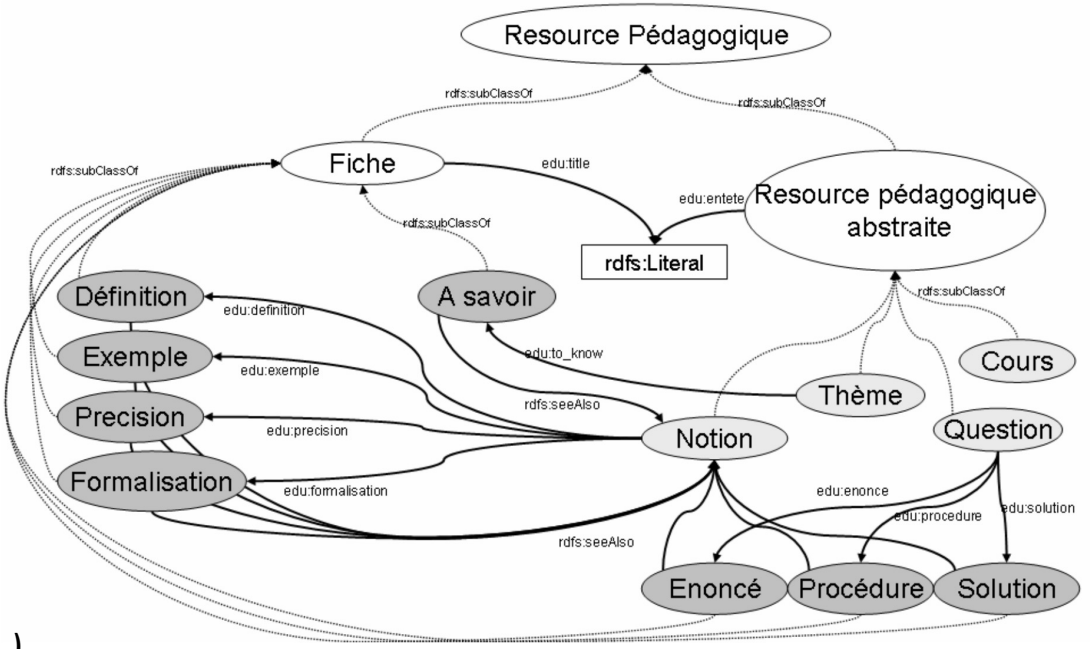
Exemples : ontologies de diagnostic, de planification

Une même idée de base : décrire explicitement des concepts et leurs relations, avec un niveau de généralité adapté aux besoins.

Un exemple : QBLS (2005)
Un système d'apprentissage basé sur les ontologies

QBLS(Semantic Web Based Learning System)

- Raisonnement automatique pour adapter le parcours aux prérequis de l'étudiant en utilisant une ontologie
- Exemple:



Domaine : Programmation en Java
L'ontologie définit :

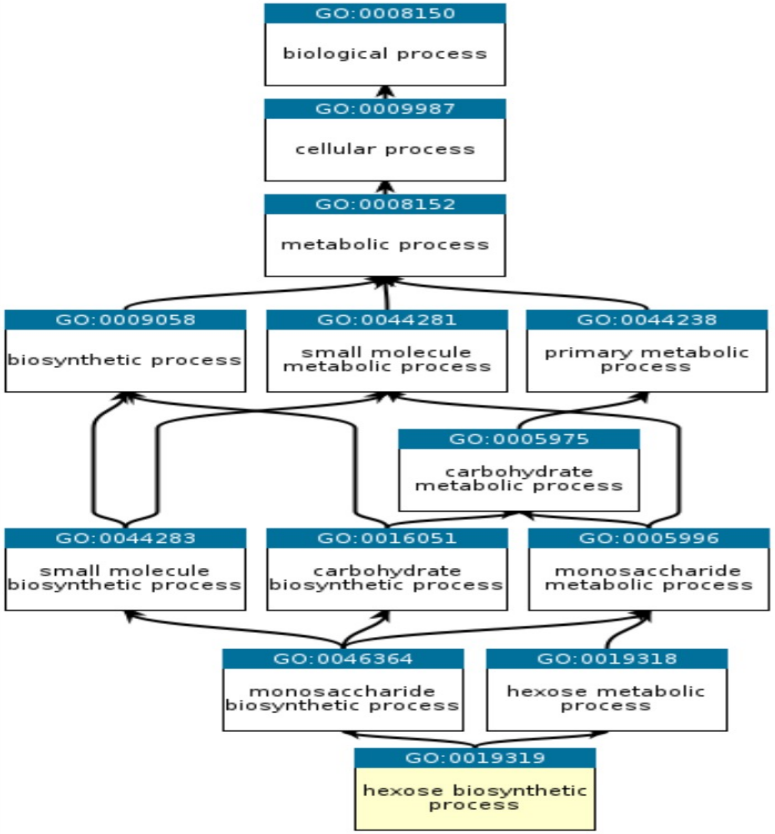
- *les concepts (boucles, classes, héritage…)*
- *les relations de prérequis (ex comprendre les boucles avant les collections)*

L'étudiant qui échoue sur "collections" est redirigé automatiquement vers des ressources sur "boucles"

Pour en savoir plus :
(Dieng & al, 2005) R Dieng-Kuntz, M Grandbastien, and D Hérin, "Web sémantique pour le e-Learning", Actes des journées AFIA / Nice, du 30 mai au 3 juin 2005, http://www-sop inria fr/acacia/afia2005/actes/ActesWSeL_Final pdf

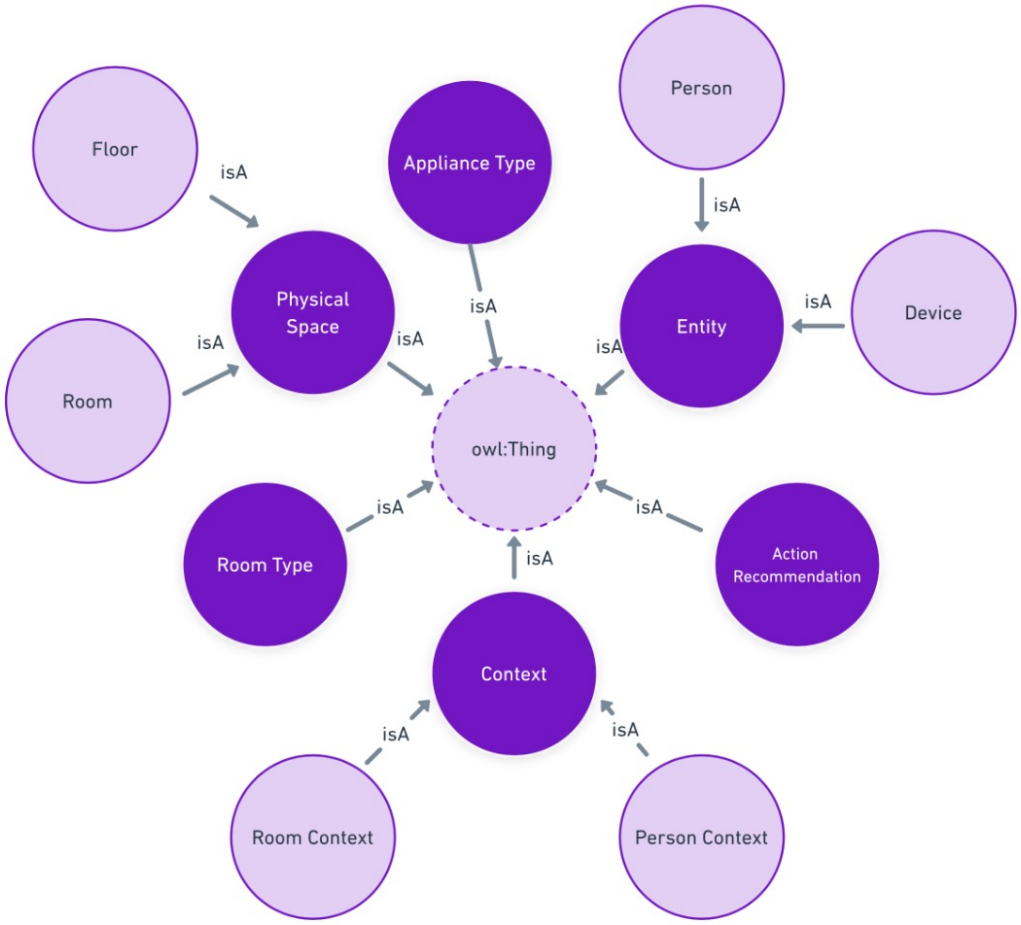
Un exemple en biologie : Gene Ontonlogy

- Représentation structurée et normalisée des études biologiques
- Téléchargeable au format owl (aussi json)



https://www geneontology org/docs/ontology-documentation/

Exemple d'Ontologie pour Smart building



source : Thèse de doctorat Loris Plasson, Université de corse, Décembre 2025

Objectifs des ontologies

Partage de connaissances

- Décrire un domaine particulier
- Fournir un vocabulaire commun et une sémantique formelle
- Réduire l'ambiguïté des termes

Interopérabilité

- Faciliter l'intégration des données provenant de sources hétérogènes
- Permettre l'échange et la réutilisation des connaissances

Recherche intelligente

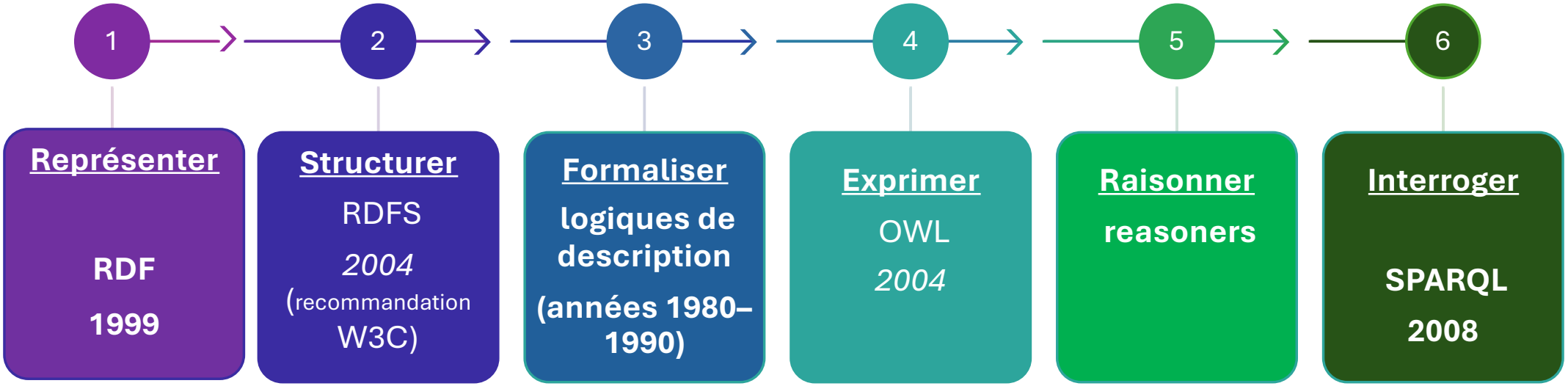
- Requêtes basées sur le sens plutôt que sur les mots-clés
- Améliorer la pertinence des résultats
- Explorer des relations implicites

Raisonnement automatisé

- Inférer de nouvelles connaissances à partir des connaissances existantes
- Vérifier la cohérence des connaissances
- Soutenir la prise de décision

Les ontologies permettent de structurer, partager, intégrer et exploiter les connaissances d'un domaine.

2 - Formaliser et raisonner avec les ontologies



Du Web de données au Web sémantique

Plus de sémantique

Données brutes

Raisonnement
Reasoners, règles (ex. SWRL)

Déduire de nouvelles connaissances à partir des connaissances existantes (*vérification de cohérence, inférences*).

OWL
Langage d'ontologies

Exprimer des connaissances riches : classes, propriétés, contraintes.

RDFS
Schéma et hiérarchies

Structurer les connaissances : classes, propriétés, hiérarchies (généralisation/spécialisation).

RDF
Modèle de données

Représenter les connaissances sous forme de triplets (sujet – prédicat – objet).

URI / IRI
Identification des ressources

Identifier de manière unique les ressources du Web.

SPARQL
Interroger les graphes de connaissances

- Requêtes sur les données RDF et les ontologies (RDFS, OWL).
- Recherche basée sur le sens.
- Exploration des relations entre ressources.

Chaque niveau enrichit le précédent en ajoutant davantage de sémantique exploitable par la machine.

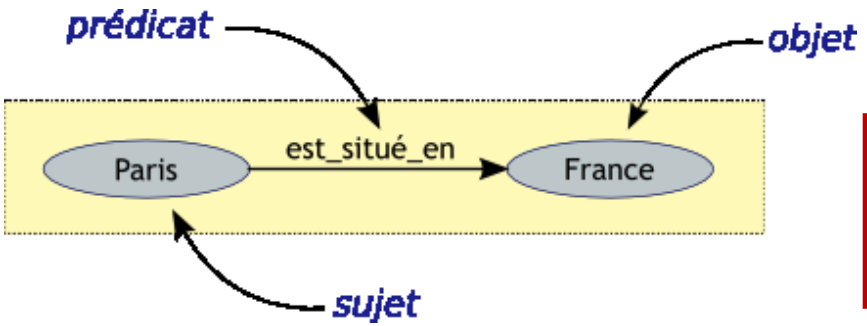
Inspiré du Semantic Web Layer Cake (Tim Berners-Lee)

RDF : un langage syntaxique pour représenter les données en graphes

RDF (Resource Description Framework) (1999)

- Langage de base du Web sémantique : **modélisation syntaxique**, utile pour l'échange de données
- Permet de représenter des données sous forme de **graphes de triplets**

  (sujet – prédicat – objet)
- Stocké au format RDF/XML



source : https://www researchgate net/figure/Exemple-de-triplet-RDF_fig21_278639053

```
<Pays rdf:ID="France"> <nomPays>France</nomPays>
</Pays>
<!-- création de "Paris", qui se trouve en France -->
<Ville rdf:ID="Paris"> <nomVille>Paris</nomVille>
<estSitueEn rdf:resource="#France" /> </Ville>
```

Simple langage de description

- Peu lisible
- Ne repose pas sur une logique formelle

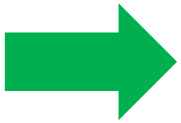
Turtle (Terse RDF Triple Language)

Une syntaxe compacte alternative à RDF : plus lisible

- Utilise des préfixes (@prefix) pour simplifier l'écriture
- Représente directement les triplets (sujet – prédicat – objet)
- Moins verbeux que RDF/XML

RDF/XML

```
<rdf:Description
rdf:about=":alice">
  <rdf:type
rdf:resource="foaf:Person"/>

<foaf:firstName>Alice</foaf:fi
rstName>
</rdf:Description>
```



Turtle

```
@prefix foaf: <http://xmlns
com/foaf/0 1/>

:alice a foaf:Person ;
       foaf:firstName "Alice"
```

foaf = "friend of a friend"

standardisé W3C en 2014

RDFS : un premier ajout de sémantique à RDF

RDFS (RDF Schema) (1999)

- Extension de RDF pour décrire des vocabulaires (ontologies simples)
- Permet d'introduire de la hiérarchie et des contraintes minimales
  - Classes et sous-classes (rdfs:Class, rdfs:subClassOf)
  - Propriétés et sous-propriétés (rdfs:Property, rdfs:subPropertyOf)
  - Domaine et portée (rdfs:domain, rdfs:range)

Exemple en Turtle

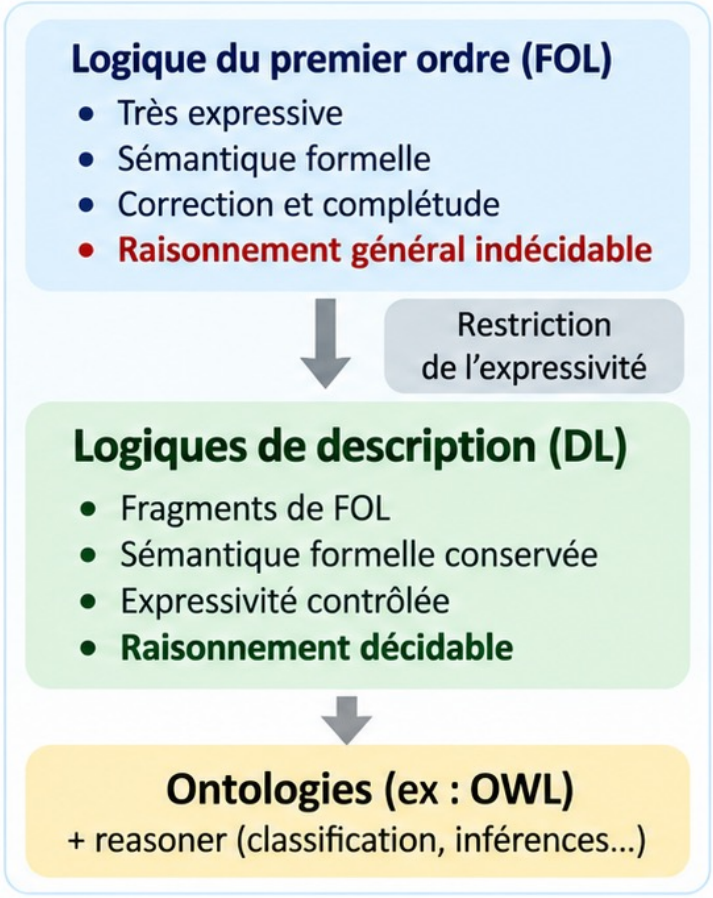
```
:Etudiant a rdfs:Class ;
          rdfs:subClassOf :Personne .

:enseigneA a rdf:Property ;
           rdfs:domain :Enseignant ;
           rdfs:range :Etudiant .
```

Limites
Expressivité faible (pas de cardinalités, pas de contraintes complexes)

Les Logiques de description

DL: Description Logics

- Famille de logiques formelles : **fragments de FOL**
  → **expressivité volontairement restreinte** pour garantir la décidabilité du raisonnement.
- Une **sémantique formelle** héritée de FOL
  → permet de donner un **sens** précis aux concepts, relations (rôles) et individus.
- Permet de :
  - définir des classes,
  - formuler des contraintes,
  - faire de l'inférence automatique (ex : classification, détection d'incohérences)
- DL est indépendant de tout format de fichier ou syntaxe.



Moins expressive que FOL → mais raisonnement automatisable et décidable

Principales logiques de description (DL)

DL	Date apparition	Composants clés	Objectif principal	Usage / Base OWL
ALC	1991	∃, ∀, ⊓, ⊔, ¬	Base logique expressive	Fondation théorique d'OWL
SHIQ	1999	ALC + hiérarchies, inverses, cardinalités	Représentation riche	OWL 1 DL
SROIQ	2006	SHIQ + nominals, propriétés complexes	Très expressive, encore décidable	**OWL 2 DL**
EL	2005	∃, ⊓, ⊤ (sans négation, ni disjonction)	Raisonnement rapide (polynomial)	**OWL 2 EL**
DL-Lite	2006	Simple, optimisé pour requêtes	Mapping vers bases de données (RDB)	**OWL 2 QL**

utilisée dans Protégé 5 6 6

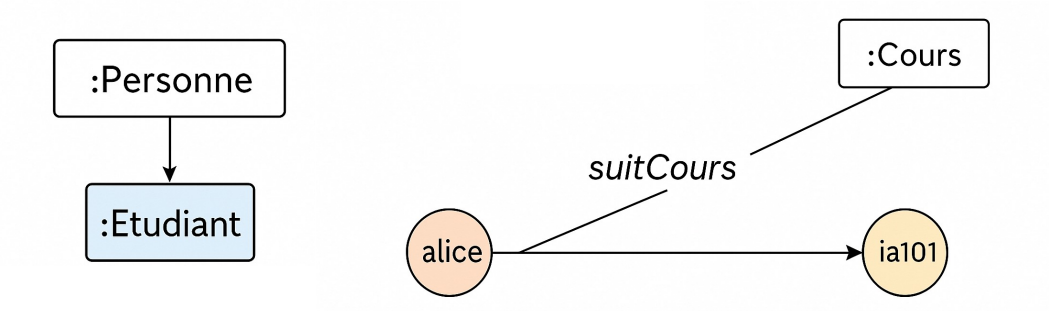
OWL : un langage fondé sur DL pour modéliser des ontologies

OWL (Web Ontology Language)

- OWL1 (2004), OWL2 (2012)
- Langage standard du Web sémantique, qui s'appuie explicitement sur certaines DL
- Implémentation concrète d'une DL
- Plusieurs syntaxes : RDF/XML (OWL) ou Turtle/OWL
- Permet d'inférer de nouvelles données avec des reasoners

📖 **Référence officielle W3C**: https://www w3 org/TR/owl2-overview/

Exemple d'ontologie OWL en Turtle et RDF/XML




```
@prefix : <http://ex org/onto#>
@prefix rdf: <http://www w3 org/1999/02/22-rdf-syntax-ns#>
@prefix rdfs: <http://www w3 org/2000/01/rdf-schema#>
@prefix owl: <http://www w3 org/2002/07/owl#>

:Personne a owl:Class
:Etudiant a owl:Class ; rdfs:subClassOf :Personne
:Cours a owl:Class

:suitCours a owl:ObjectProperty ;
   rdfs:domain :Etudiant ;
   rdfs:range :Cours

:alice a :Etudiant
:ia101 a :Cours
:alice :suitCours :ia101
```

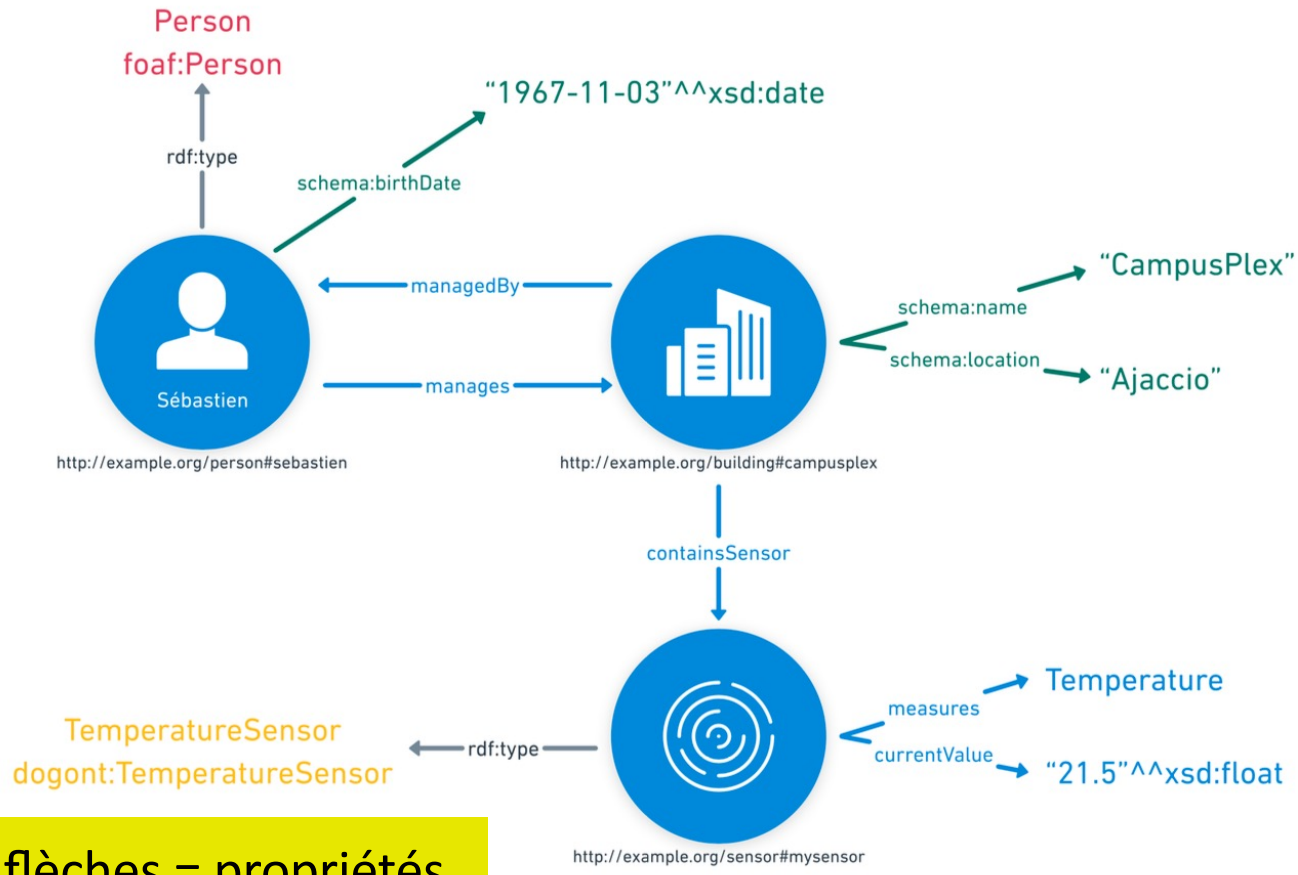
Turtle / OWL

RDF/XML (OWL)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
         xmlns:owl="http://www.w3.org/2002/07/owl#"
         xmlns="http://ex.org/onto#">
  <!-- Classes -->
  <owl:Class rdf:about="http://ex.org/onto#Personne"/>
  <owl:Class rdf:about="http://ex.org/onto#Etudiant">
    <rdfs:subClassOf rdf:resource="http://ex.org/onto#Personne"/>
  </owl:Class>
  <owl:Class rdf:about="http://ex.org/onto#Cours"/>
  <!-- Propriété -->
  <owl:ObjectProperty rdf:about="http://ex.org/onto#suitCours">
    <rdfs:domain rdf:resource="http://ex.org/onto#Etudiant"/>
    <rdfs:range rdf:resource="http://ex.org/onto#Cours"/>
  </owl:ObjectProperty>
  <!-- Individus -->
  <owl:NamedIndividual rdf:about="http://ex.org/onto#alice">
    <rdf:type rdf:resource="http://ex.org/onto#Etudiant"/>
    <suitCours rdf:resource="http://ex.org/onto#ia101"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="http://ex.org/onto#ia101">
    <rdf:type rdf:resource="http://ex.org/onto#Cours"/>
  </owl:NamedIndividual>
</rdf:RDF>
```

OWL ajoute un vocabulaire spécialisé à RDFS et Turtle (URI prédéfinis)

Exemple d'Ontologie pour Smart building



flèches = propriétés

```
<building#campusplex>
    a dogont:Building ;
    schema:name "CampusPlex" ;
    schema:location "Ajaccio" ;
    ex:managedBy <person#sebastien> ;
    ex:containsSensor <sensor#mysensor> .

<person#sebastien>
    a foaf:Person ;
    foaf:firstName "Sebastien" ;
    schema:birthDate "1967-11-03"^^xsd:date
    ex:manages <building#campusplex> .

<sensor#mysensor>
    a dogont:TemperatureSensor ;
    ex:measures "Temperature" ;
    ex:currentValue "21.5"^^xsd:float .
```

Syntaxe Turtle (fichier ttl)

source : Thèse de doctorat Loris Plasson, Université de corse, Decembre 2025

Comparaison RDF, RDFS et OWL

Besoin	Exemple	RDF	RDFS	OWL
Structuration des données	`Max rdf:type Chien` (Max est un chien)	✔	✔	✔
Hiérarchie des classes	`Chien rdfs:subClassOf Mammifere` (Un chien est un mammifère)	✖	✔	✔
Hiérarchie des propriétés	`aUneMere rdfs:subPropertyOf aUnParent` (Une mère est un parent)	✖	✔	✔
Contraintes de cardinalité	Un mammifère a exactement 2 parents	✖	✖	✔
Définition de classes équivalentes	`Voiture ≡ Automobile` (Voiture et Automobile sont identiques)	✖	✖	✔
Définition de classes disjointes	`Homme ⊥ Femme` (Un individu ne peut pas être les deux à la fois)	✖	✖	✔
Raisonnement automatique avancé	Si `Max` est un `Chien` et que `Chien ⊆ Mammifere` alors on peut dire que `Max` est aussi un `Mammifere`	✖	✖	✔

source : Thèse de doctorat Loris Plasson, Université de corse, Decembre 2025

Langage SWRL (Semantic Web Rule Language)

- Extension d'OWL (2004–2005)
- Permet d'exprimer des règles conditionnelles de type si… alors… qu'OWL ne peut pas capturer (ex chaînes de propriétés complexes, contraintes sur des valeurs numériques, etc)
- Syntaxe proche des règles logiques (clauses de Horn) :

```
Person(?x) ^ hasChild(?x, ?y) ^ Doctor(?y) → ParentOfDoctor(?x)
```

Limite
SWRL n'est pas toujours décidable

Les **reasoners** : moteurs d'inférence pour OWL

- Exploitent la logique formelle (DL) derrière OWL
- Dérivent automatiquement des faits implicites à partir des axiomes
- Détectent les incohérences, classent les concepts, infèrent l'appartenance des individus aux classes

Quelques reasoners	
HermiT	Raisonneur OWL 2, supporte SROIQ, performant
Pellet	Raisonneur complet, supporte SPARQL + règles
Fact++	Très rapide, bon pour les grands ensembles
ELK	Ultra-rapide, mais limité à OWL EL (pas complet)

Langage SPARQL : Principe

- Langage standard W3C pour interroger des données RDF/OWL
- Permet d'interroger des **triplets RDF** : sujet – prédicat – objet
  - phrases sujet-verbe-complément
- Chaque requête SPARQL consiste à **chercher des motifs** qui correspondent à ces triplets
- Syntaxe déclarative : SELECT, WHERE, FILTER, PREFIX
- Permet extraction, filtrage, construction

Fonctionnement en Monde Clos

- Très utile pour **extraire des données**, y compris les **littéraux** (ex: noms, dates…)
- Mais SPARQL **ne raisonne pas** : il ne trouve que ce qui est **explicitement** dans le graphe RDF

SPARQL Update : modifier des graphes RDF

- Depuis 2013, SparQL permet aussi de modifier des graphes RDF
  - INSERT DATA → ajouter des triplets
  - DELETE DATA → supprimer des triplets
  - DELETE/INSERT … WHERE → mettre à jour conditionnellement
  - LOAD, CLEAR, CREATE GRAPH, DROP GRAPH → gérer les graphes

```
INSERT DATA {
  :alice a :Etudiant ;
         :nom "Alice" .
}
```

```
DELETE { :alice :nom "Alice" }
INSERT { :alice :nom "Alice Dupont" }
WHERE  { :alice :nom "Alice" }
```

3 - Construire et raisonner avec une ontologie : Protégé

TP guidé ontocampus

Outils de Création et d'Édition d'ontologies

- Notre choix : **Protégé**
  - Environnement de développement open-source
  - Une version en ligne : WebProtégé
- **TopBraid Composer**
  - IDE pour les applications de Web Sémantique
- **NeOn Toolkit**
  - Environnement opensource

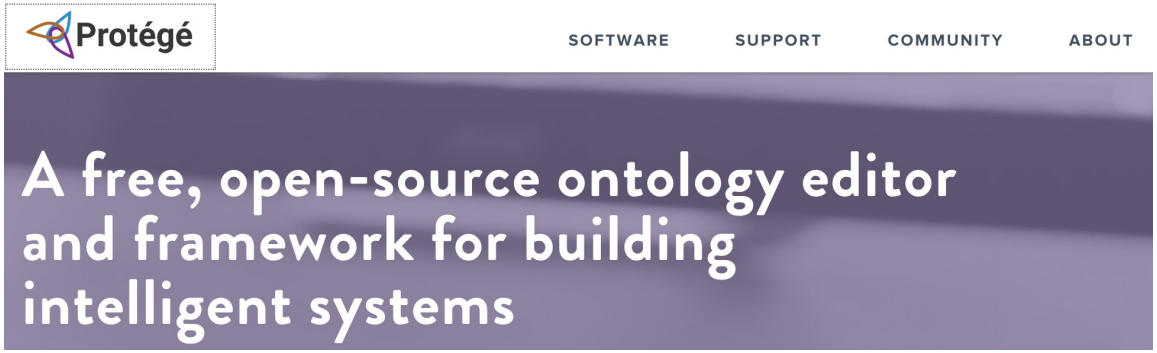
  de développement collaboratif d'ontologies

Stanford University

Plateforme Protégé

protégé

🛠️ **Développé par Stanford** Center for Biomedical Informatics Research



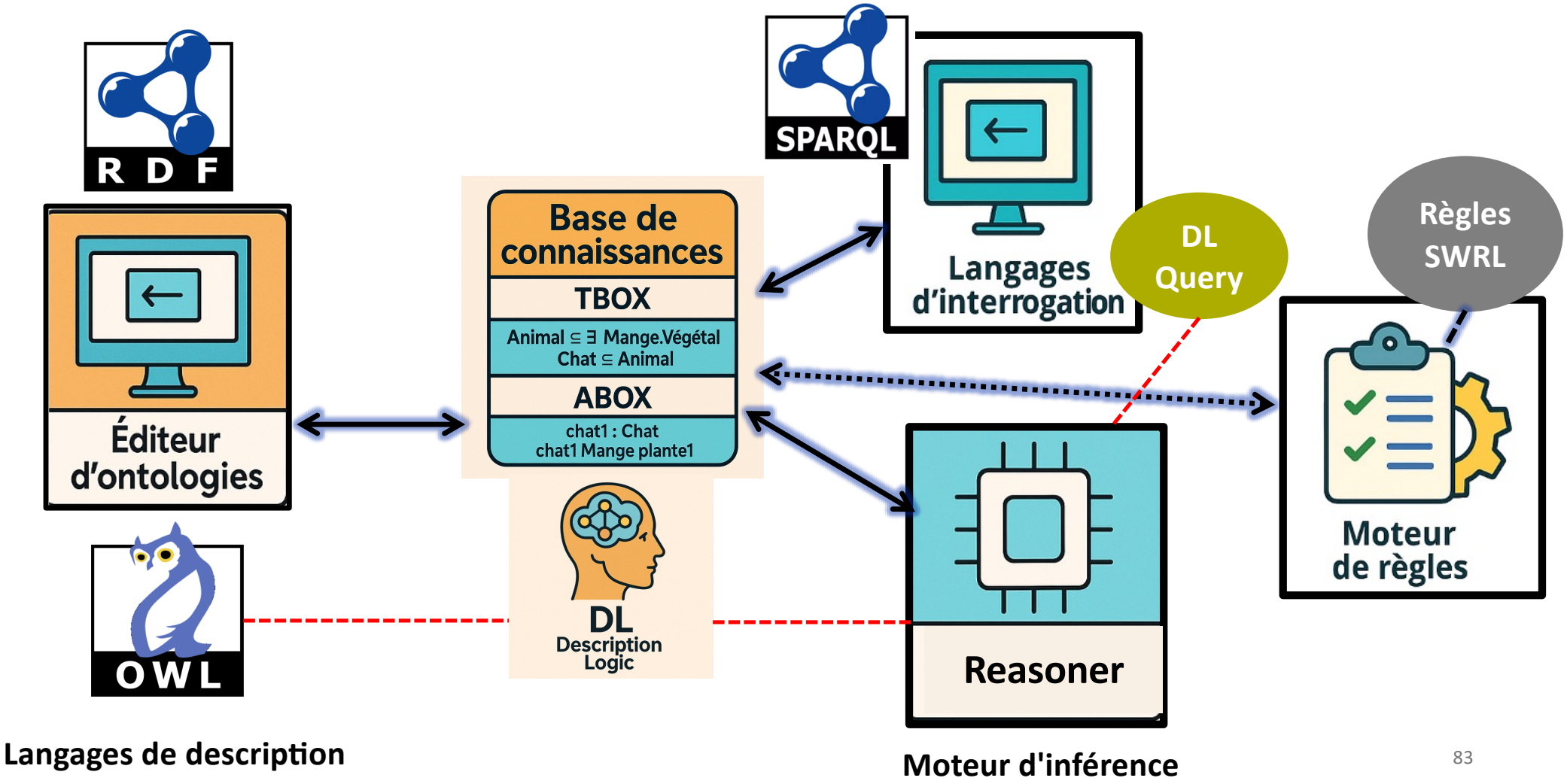
🌐 **Format et interopérabilité**

- Supporte les standards du **Web sémantique** : RDF, RDFS, OWL
- Export possible en RDF/XML, OWL/XML, Turtle…

👨‍💻 **Avantages**

- Interface graphique simple pour découvrir les ontologies
- Très utilisé dans l'enseignement, la recherche et la bio-informatique

Composants de la plateforme Protégé



Langages de description

Moteur d'inférence

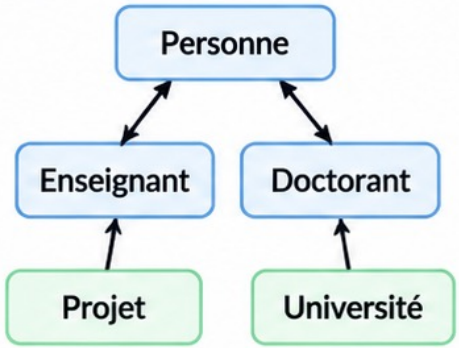
Etapes de définition/manipulation d'une ontologie

1 – Construire l'ontologie : définir le vocabulaire (TBox)

protégé

1 Classes
concepts du domaine

Quels sont les concepts que l'on veut représenter ?



Exemples :
Personne, Enseignant, Doctorant, Projet, Université…

2 Propriétés
attributs et relations

Comment décrire les objets et leurs relations ?

Data properties
valeurs littérales

- nom
- statutThese
- adresse
- …

Object properties
liens entre concepts

- encadre
- dirige
- estMembreDe
- …

3 Contraintes
sur les propriétés

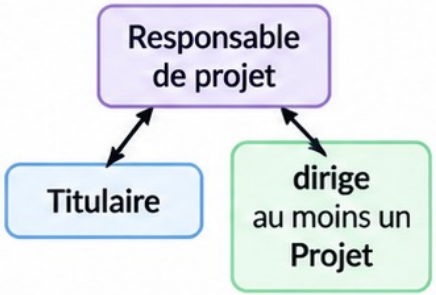
Quelles sont les règles à respecter ?

Domain	classe source de la propriété
Range	classe cible
Cardinalité	min, max, exacte
Inverse	propriété inverse
Transitivité	propriété transitive
Caractéristique	fonctionnelle, symétrique, …

4 Classes définies
par équivalence

Quels concepts peut-on définir à partir de conditions nécessaires et suffisantes ?

Exemple :
ResponsableDeProjet ≡
Titulaire **and**
(**dirige some** Projet)



La TBox décrit le modèle du domaine
Classes, propriétés, contraintes et classes définies.

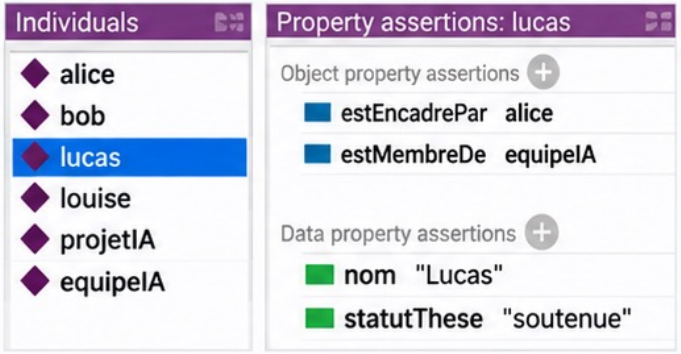
2 – Exploiter l'ontologie : peupler, raisonner, interroger

protégé

5 Peupler l'ontologie (ABox)

Ajouter des individus et leurs assertions

Créer des instances des classes et leur associer des propriétés (valeurs et liens).



Exemples :

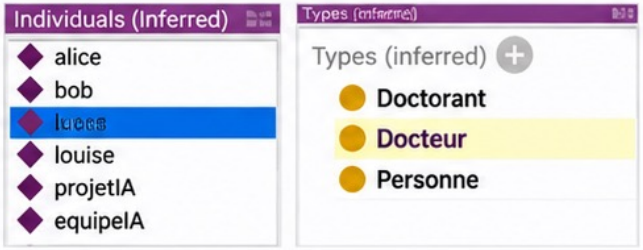
lucas : **Doctorant**
lucas **estEncadrePar** alice
lucas **statutThese** "soutenue"

6 Lancer le raisonnement

Déduire des connaissances implicites et vérifier la cohérence

Le raisonneur (ex : HermiT) calcule automatiquement :

- les appartenances à des classes
- les relations implicites
- et détecte les éventuelles incohérences



Exemple :

Si lucas **statutThese** "soutenue" et
lucas : **Doctorant**
alors lucas : **Docteur** *(inféré)*

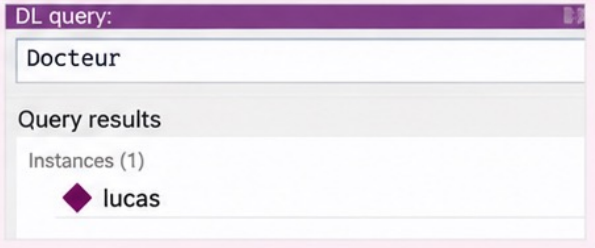
Le raisonneur vérifie aussi la cohérence de l'ontologie.

7 Interroger l'ontologie

DL Query / SPARQL

Rechercher des individus ou des relations dans les connaissances explicites et inférées.

DL Query (dans Protégé)

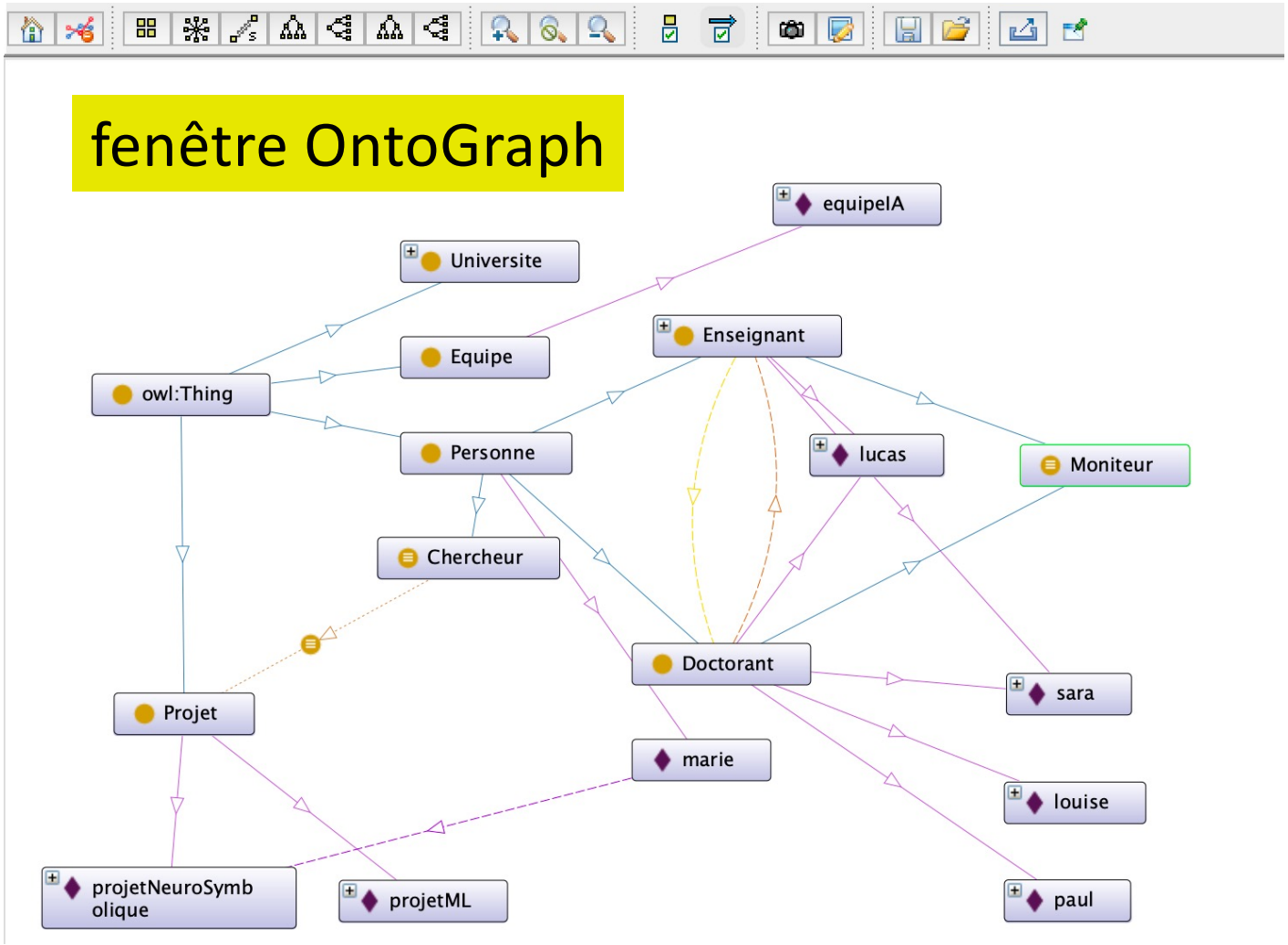


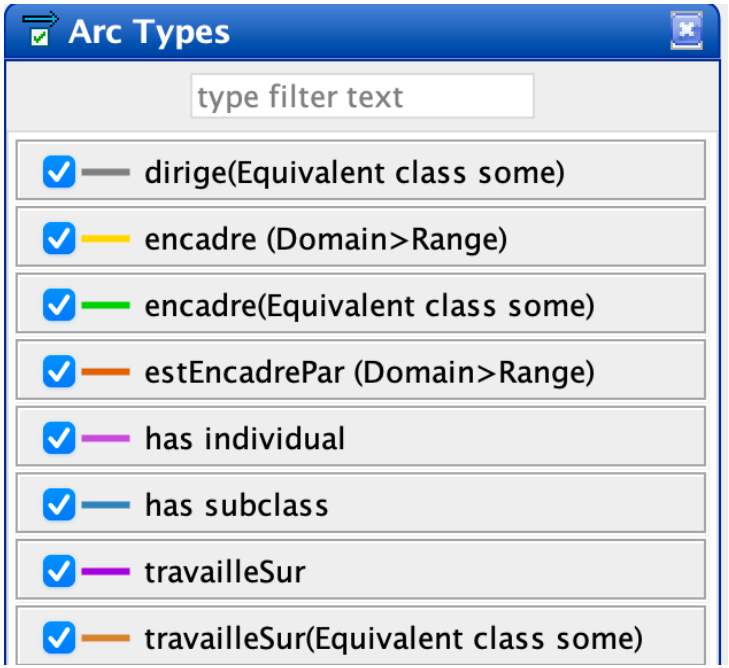
SPARQL (requêtes sur l'ontologie)

```
PREFIX : <http://www.univ-corse.fr/ontologies/ontocampus#>
SELECT ?x ?nom
WHERE {
      ?x a :Docteur .
      ?x :nom ?nom .
}
```

Construire l'ontologie (TBox) → Peupler (ABox) → Raisonner → Interroger (DL Query / SPARQL)

OntoCampus : Exemple d'ontologie





Modélisation des acteurs, des projets de recherche et des relations d'encadrement et de collaboration au sein d'une université.

Installation de Protégé

- Téléchargez la version descTop de Protégé
  - https://protege stanford edu/software php#desktop-protege
- Choisissez si possible (non offert pour Macos) une version incluant le JDK ("Protégé Desktop (with embedded JDK)"
- Téléchargez l'archive zip (tar gz sous Linux)

⚠ Protégé nécessite Java 17+ (LTS)

- **Sous Windows :**
  - Décompressez-la dans un dossier de votre choix (ex C:\Program Files\Protege)
  - Allez dans le dossier et double-cliquez sur run bat

- **Sous macOS :**
  - Décompressez, puis faites glisser **le dossier complet** dans le dossier Applications
  - Dans le dossier, ouvrez run command ou Protégé app
- **Sous Linux :**
  - Allez dans le dossier, ouvrez un terminal
  - Lancez avec : /run sh

Si macOS vous empêche d'ouvrir l'app :
Clic droit > Ouvrir (au lieu de double-clic)
Ou allez dans *Préférences Système > Sécurité > Autoriser Protégé*

Création d'une ontologie dans Protégé

Pour créer une nouvelle Ontologie dans protégé :

- File -> new
- IRI = **Uniform Resource Identifier**
- C'est un **identifiant unique et global** pour chaque entité de l'ontologie : classe, propriété, individu…
- Syntaxe : <namespace>#<nom_local>

Ex: http://www univ-corse fr/ontologies/ontocampus#Personne

  - http://www univ-corse fr/ontologie → le **namespace** (préfixe)
  - Personne → le **nom local** de l'entité
  - Le caractère # sert à **séparer** les deux

- Choisir le format de stockage

  standard **RDF/XML syntax**

Contraintes / Bonnes pratiques

✅ Le namespace doit être une **URL valide** (même si la ressource n'existe pas en ligne)

✅ Il est courant d'utiliser # pour séparer les entités (mais / est aussi autorisé techniquement)

🚫 Le nom local **ne doit pas contenir d'espace** ni de caractère spécial

✅ Le nom local doit commencer par une **lettre** et peut contenir lettres, chiffres, tirets ou underscores

2 TP guidé : Compléter et manipuler une ontologie ontocampus

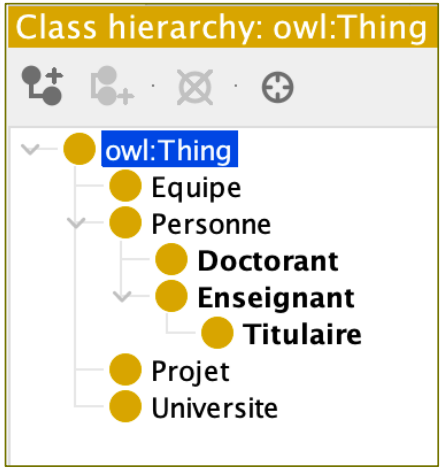
- Vous disposez d'une première version de l'ontologie. Certaines classes, propriétés et instances sont déjà définies.
- Récupérez sur l'ENT le fichier **ontoCampusDepart.owl**
- Lancez Protégé et choisissez d'ouvrir une nouvelle ontologie (File-> Open)
  - Vous devriez voir l'IRI **IRI = http://www univ-corse fr/ontologies/ontocampus#**

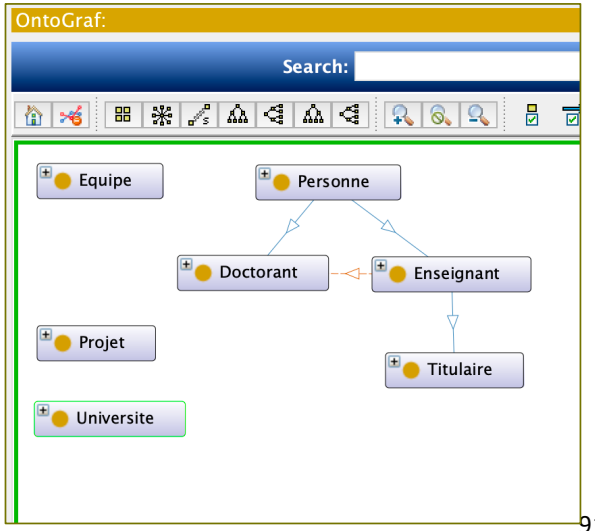
Objectif : comprendre comment enrichir une ontologie et comment le raisonneur permet de déduire de nouvelles connaissances.

3 TP guidé ontocampus

Explorez la structure de l'ontologie

- Dans l'onglet **Entities → Classes** :
- Observez la hiérarchie des classes déjà définies.
- Observez leur organisation hiérarchique.
- Visualisez cette hiérarchie avec **OntoGraph :**
  - Window->Views-> Class views->OntoGraph
  - Cliquez sur les classes pour les faire apparaitre dans la fenetre





Définition des propriétés

1 - Définition des **Data properties** (attributs)

2 - Définition des **Object properties** explicites

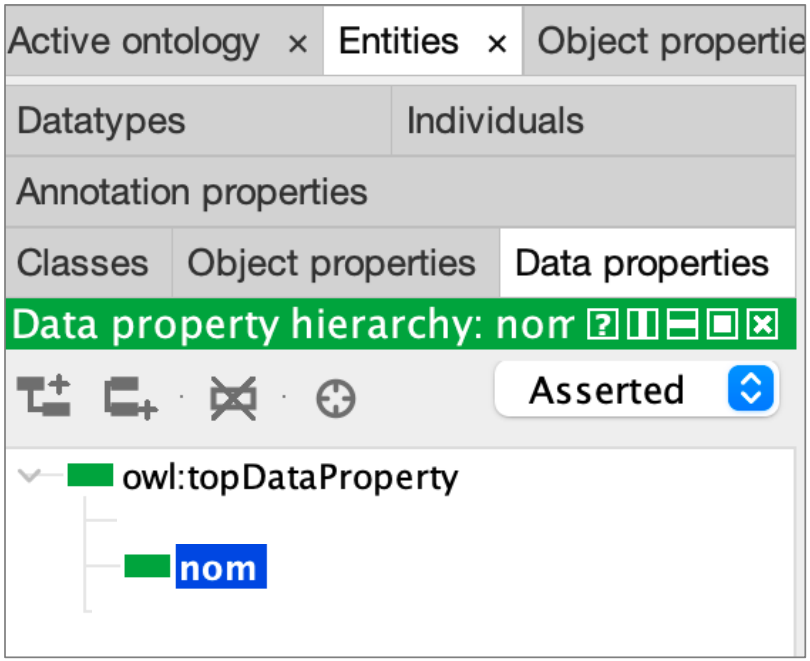
3 - Définition des **propriétés** à inférer

4 - Définition des **Chain of object properties**

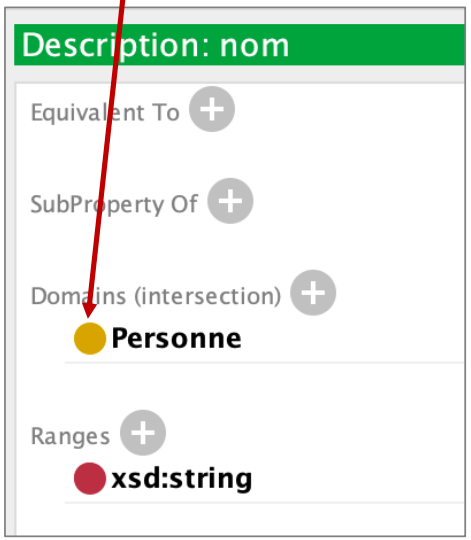
1 - Définition des Data properties explicites

Exemple: *nom dans la classe Personne*

notion classique d'attribut



Classe que décrit la propriété



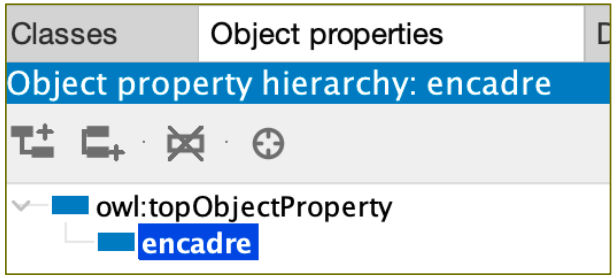
2 - Définition des Object properties explicites

Définissent des liens entre classes
(cf association en UML)

Exemple:

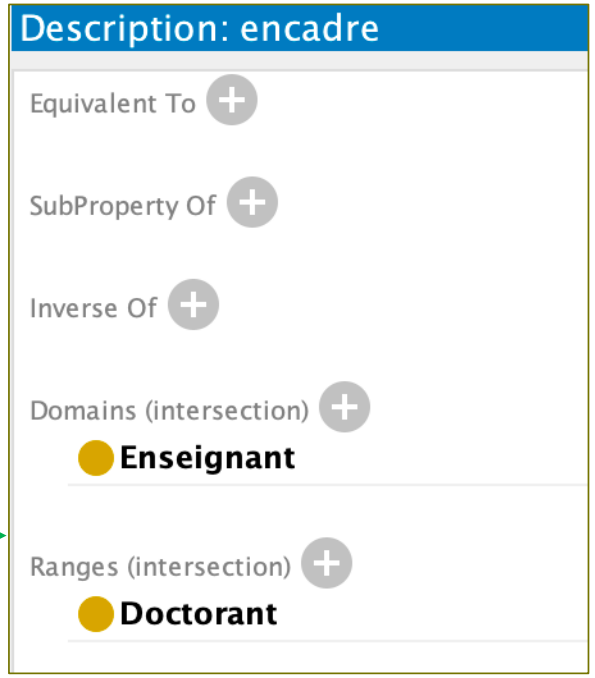
Un Enseignant encadre

un Doctorant



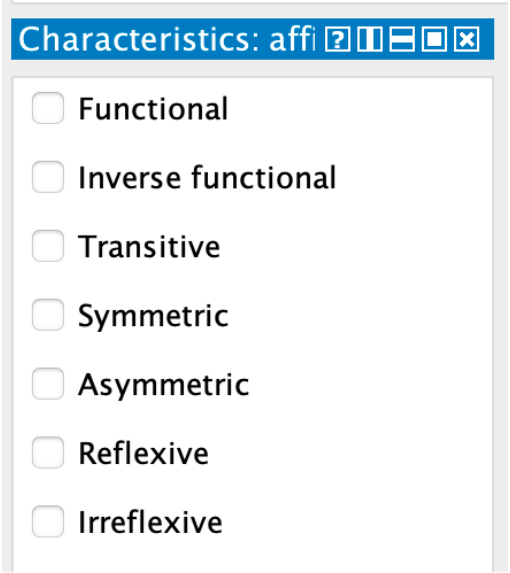
Entite sur laquelle
porte la propriété

Entite cible



Caractéristiques des Object properties

Caractéristique	Signification	Effet sur l'inférence
Functional (fonctionnelle)	Une entité a **au plus un** objet lié par cette propriété	Si x :p y et x :p z, alors y = z
Inverse Functional	Une entité est liée par **au plus un sujet**	Si x :p y et z :p y, alors x = z
Symmetric	Si x :p y, alors automatiquement y :p x	Ex : amiDe est symétrique
Asymmetric	Si x :p y, alors **jamais** y :p x	Ex : parentDe (on ne peut pas être parent l'un de l'autre)
Reflexive	Chaque individu est lié à lui-même	x :p x est toujours vrai
Irreflexive	Aucun individu n'est lié à lui-même	x :p x est toujours faux
Transitive	Si x :p y et y :p z, alors x :p z	Ex : estAncienDe
Inverse Of	Permet de définir une propriété comme l'**inverse logique** d'une autre	Ex : estEncadréPar est l'inverse de encadre



TP guidé ontocampus

Propriétés explicites

La propriété **encadre** est déjà définie dans l'ontologie.

1. Observez sa définition
2. À partir de cet exemple, créez les Object properties suivantes :

Propriété	Direction	Description
travailleSur	Personne → Projet	Une personne travaille sur un projet de recherche
dirige	Titulaire → Projet	Un titulaire est responsable scientifique d'un projet
estMembreDe	Personne → Équipe	Une personne est membre d'une équipe de recherche
affilieA	Personne → Université	Une personne est affiliée à une université

3. Ajoutez une Data Property de votre choix pour caractériser Personne.

3 - Définition des Object properties à inférer

Certaines relations n'ont pas besoin d'être saisies explicitement : elles peuvent être déduites à partir d'autres relations.

- Exemple :
  - Alice —> encadre —> Louise
  - Si l'on définit :

    estEncadrePar **inverseOf** encadre
  - le raisonneur peut inférer :

    Louise —> estEncadrePar —> Alice
  - La relation estEncadrePar n'a pas été saisie : elle est inférée.

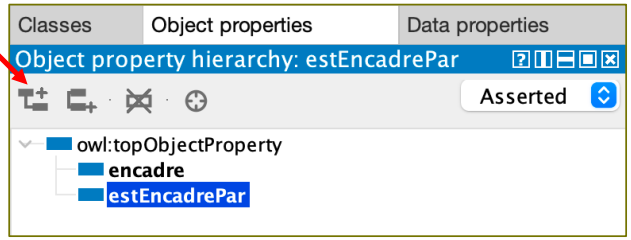
TP guidé ontocampus

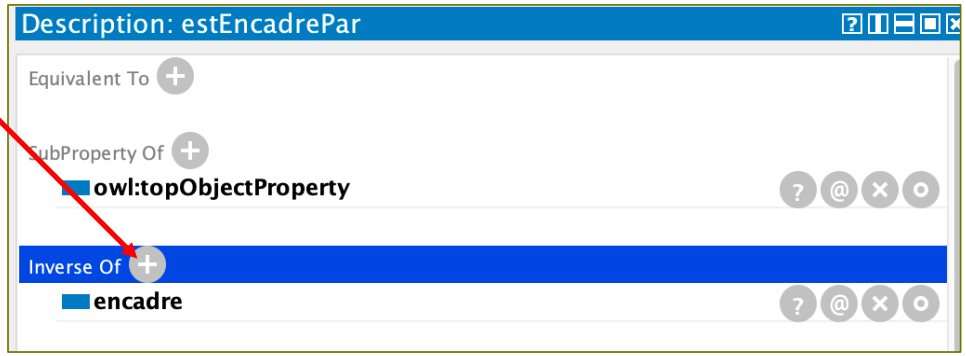
Propriétés à inférer

Définissez l'object property à inférer suivante :

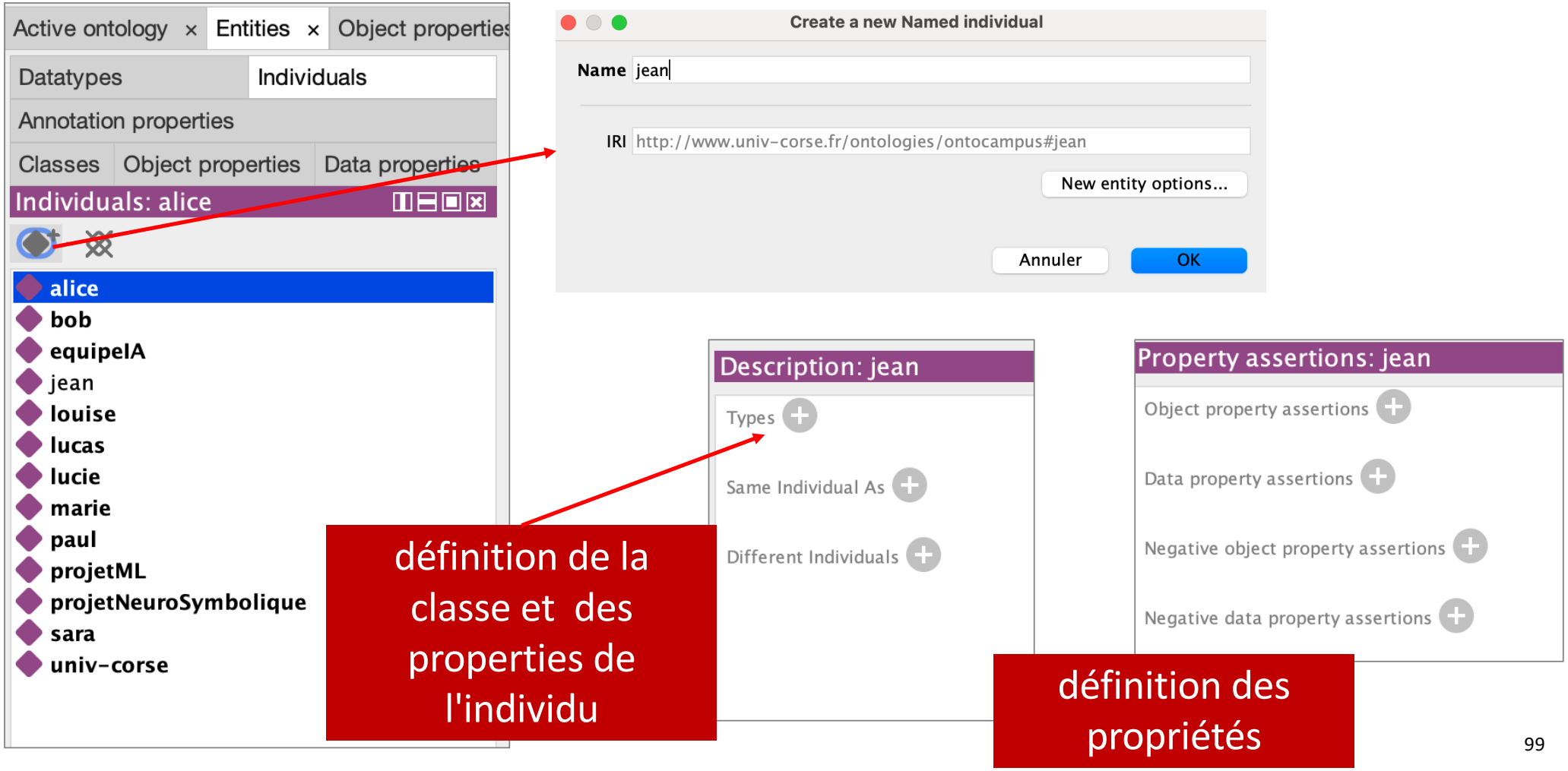
- **estEncadrePar** inverse de la propriété encadre
- Ne renseignez pas son domaine ni sa portée

estEncadrePar	Doctorant → Enseignant	Un doctorant est encadré par un enseignant (inverse de encadre)

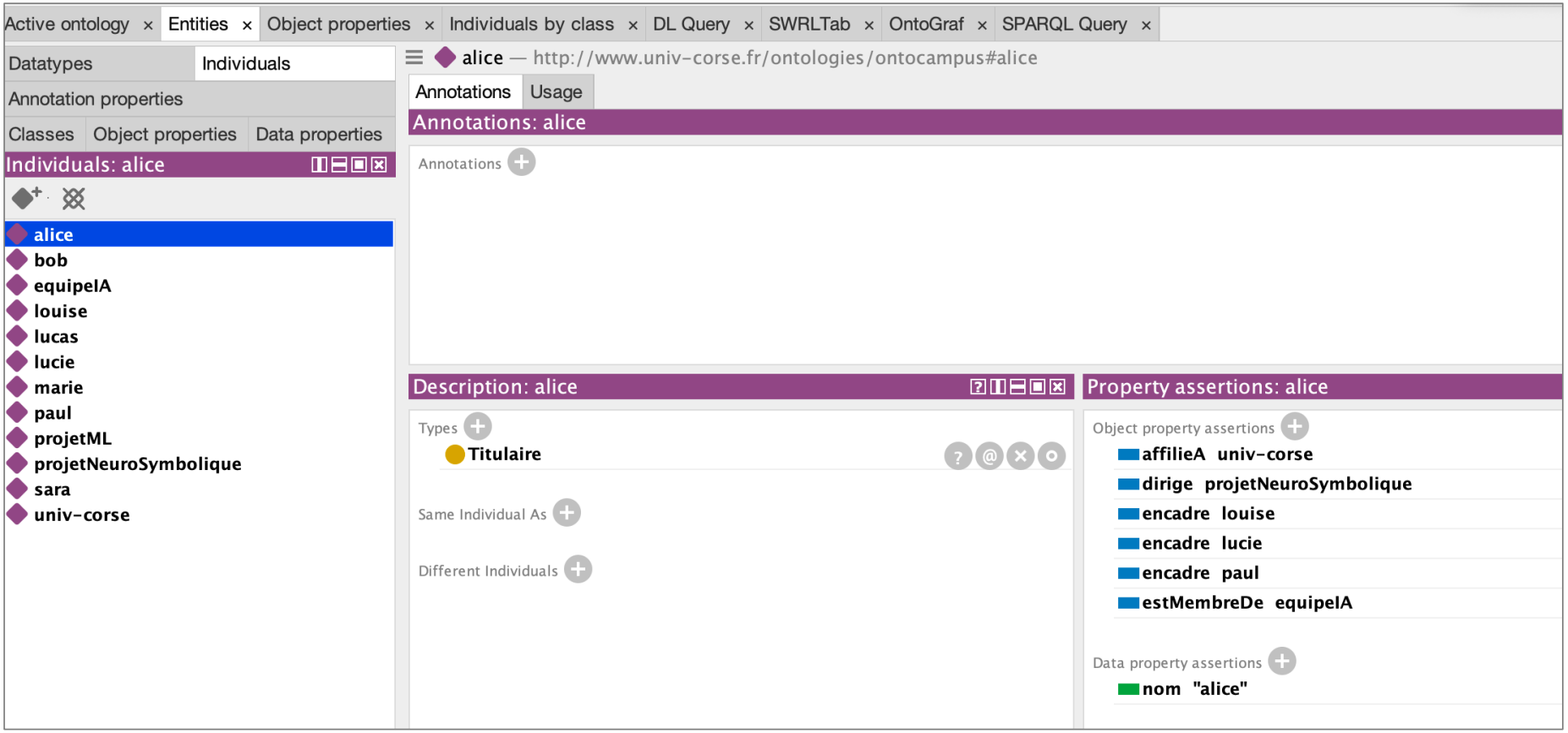




3 – Définition des individus



Définition des individus



TP guidé ontocampus

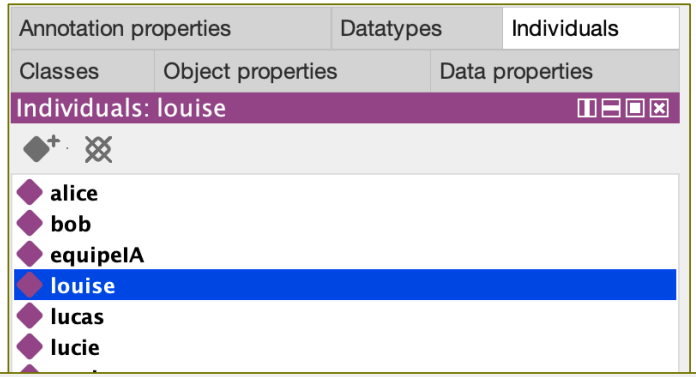
Définition des individus

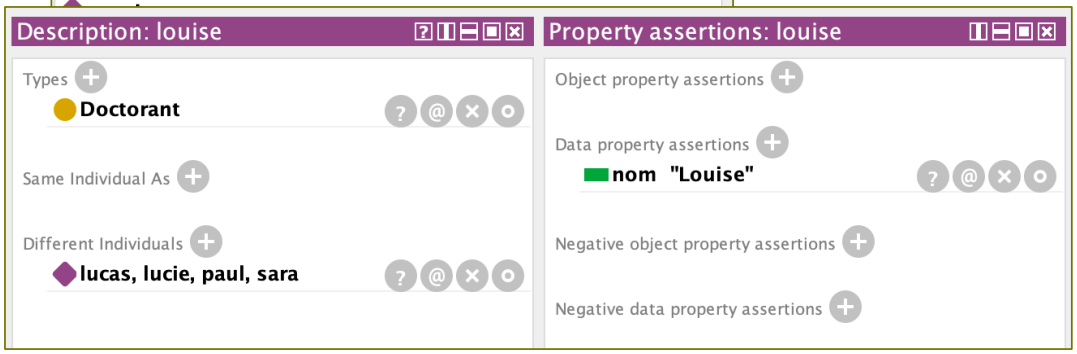
Les individus et leurs types sont déjà définis dans l'ontologie.

Explorez quelques individus :

- alice
- louise
- lucas
- projetNeuroSymbolique

Repérez leurs Types, Object property assertions et Data property assertions





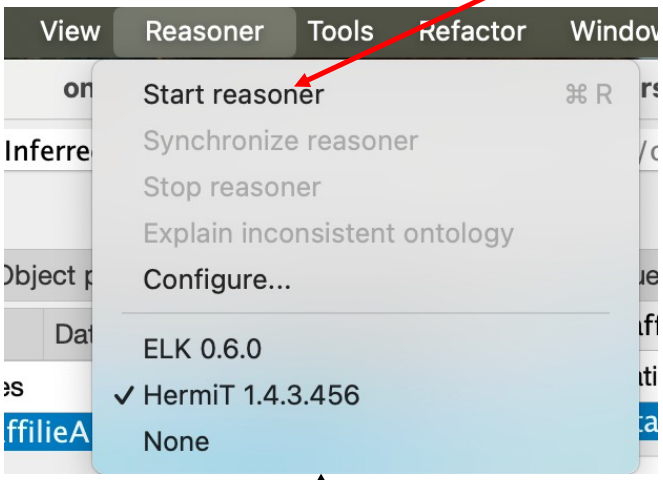
6

TP guidé ontocampus

Définition des individus

- Complétez les assertions suivantes :

Individus	Assertions à définir
alice	affilieA → univ-corse ; dirige → projetNeuroSymbolique ; estMembreDe → equipeIA
bob	affilieA → univ-corse ; dirige → projetML ; estMembreDe → equipeIA
marie	travailleSur → projetNeuroSymbolique



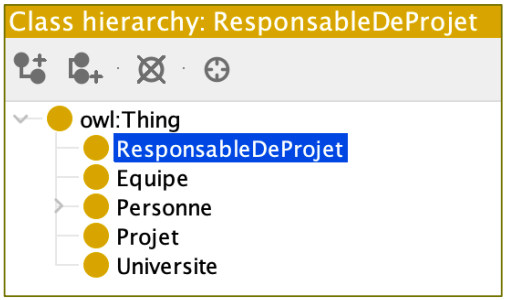
lancement du reasoner

- Lancez le reasoner HermiT

Observez maintenant l'individu louise.

Quelle nouvelle Object Property apparaît comme **inférée** ?

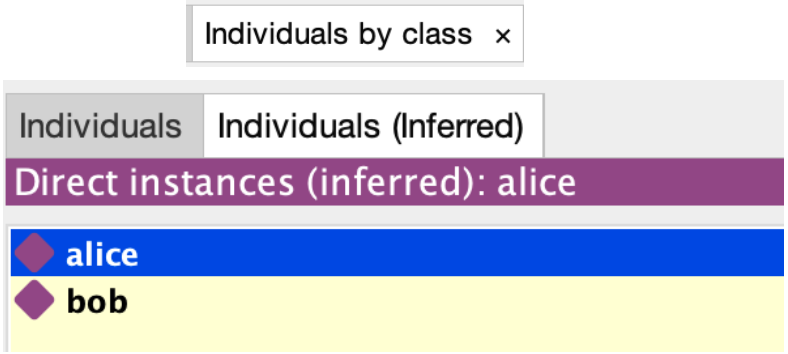
Classes définies par équivalence : classification automatique



Règle de définition de la classe

lancement du reasoner

Les instances de la classe sont inférées



Remarque : dès la définition de la classe, Protégé la positionne sous Titulaire dans la hiérarchie

Règles permettant l'inférence

- Règles OWL permettant de définir :
  - des classes définies par équivalence : *equivalent to*
    - ➡ classes dont les instances sont inférées selon une règle
  - des axiomes sur les classes: *general class axioms*
  - des contraintes sur les propriétés d'objet et de données
    - ➡ A vérifier lors de l'inférence

- Syntaxe d'écriture des règles : syntaxe lisible Manchester Syntax

lien vers la syntaxe officielle W3C : https://www w3 org/2007/OWL/draft/ED-owl2-manchester-syntax-20081128/

OWL Manchester Syntax

- Syntaxe d'écriture des règles : syntaxe lisible

OWL	Manchester	Exemple
someValuesFrom	some	enfant some Homme
allValuesFrom	only	soeur only Femme
hasValue	value	paysDOrigine value angleterre
minCardinality	min	enfant min 3
cardinality	exactly	enfant exactly 3
maxCardinality	max	enfant max 3

lien vers la syntaxe officielle W3C : https://www w3 org/2007/OWL/draft/ED-owl2-manchester-syntax-20081128/

Syntaxe OWL / DL query

Concept logique	Notation mathématique	Syntaxe Protégé (DL Query)	Exemple
Intersection	A ⊓ B	A and B	Personne **and** Enseignant
Union	A ⊔ B	A or B	Doctorant **or** Enseignant
Négation	¬A	not A	**not** Doctorant
Inclusion	A ⊑ B	— (via par "sous-classes")	Doctorant ⊑ Personne (axiome)
Équivalence	A ≡ B	— (via « Equivalent classes »)	Enseignant ≡ Personne and (encadre **some** Doctorant)
Existence	∃R C	R some C	encadre some Doctorant
Universalité	∀R C	R only C	encadre **only** Doctorant
Valeur exacte	R = a	R value a	encadre **value** paul
Cardinalité exacte	= n R C	R exactly n C	encadre **exactly** 2 Doctorant
Cardinalité minimale	≥ n R C	R min n C	encadre **min** 1 Doctorant
Cardinalité maximale	≤ n R C	R max n C	encadre **max** 3 Doctorant

TP guidé ontocampus

Classes définies par équivalence

Définissez les classes suivantes définies par équivalence :

- la classe **ResponsableDeProjet :** Titulaire qui dirige un projet
- la classe **Encadrant** : Enseignant qui encadre au moins un doctorant
- la classe **EncadrantExperimente** : Encadrant qui encadre au minimum 3 doctorants

Lancez le raisonneur et observez les instances inférées

Définition de Chain of object properties (SuperProperty of chain)

- Le symbole **o** correspond à la composition
  - similaire à l'opérateur de composition de fonctions

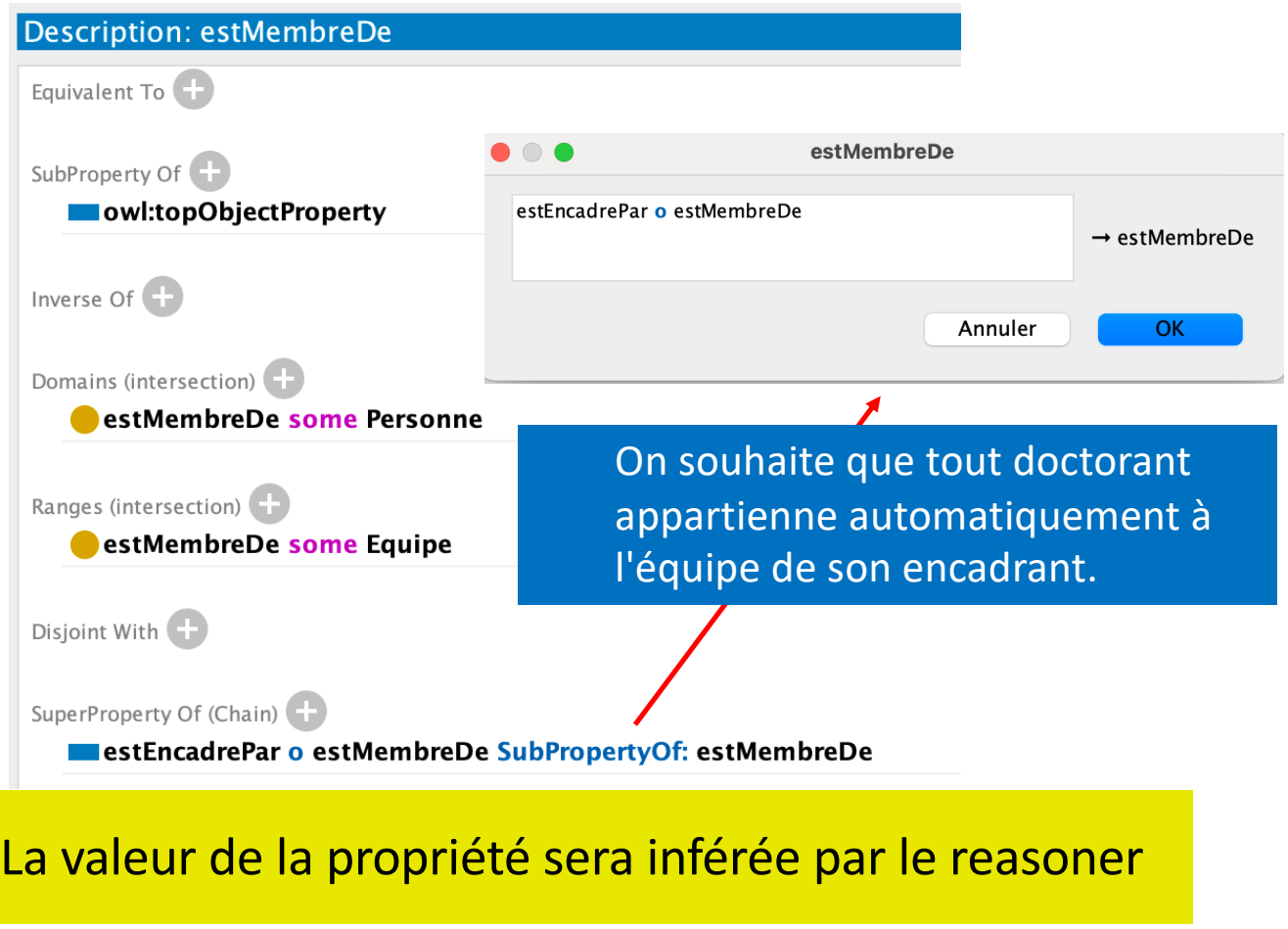
- Signifie :

Si x estEncadrePar y et

y estMembreDe z,

alors on en déduit que

x estMembreDe z



La valeur de la propriété sera inférée par le reasoner

8

TP guidé ontocampus

Définition de Chaînes de propriétés

1- Reproduisez l'exemple du cours précédent :

- Tout doctorant doit appartenir automatiquement à l'équipe de son encadrant.
- Définissez la chaîne de propriétés permettant cette inférence.

2- Définissez maintenant la chaîne permettant de répondre au besoin suivant :

- Un doctorant est affilié à l'université de son encadrant.

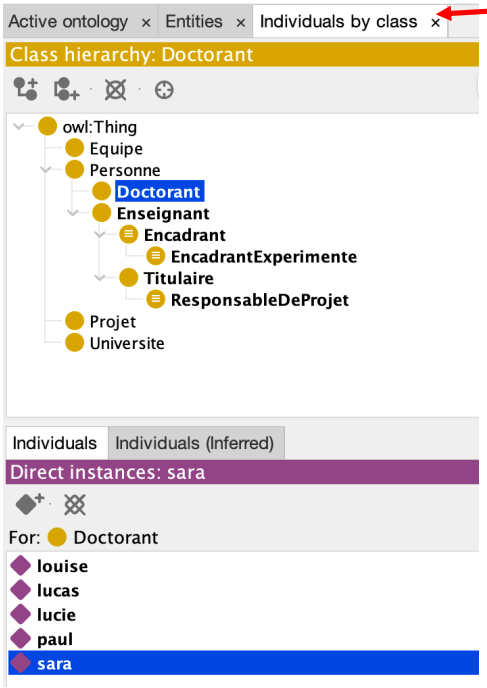
3 - Lancez le raisonneur et vérifiez le résultat.

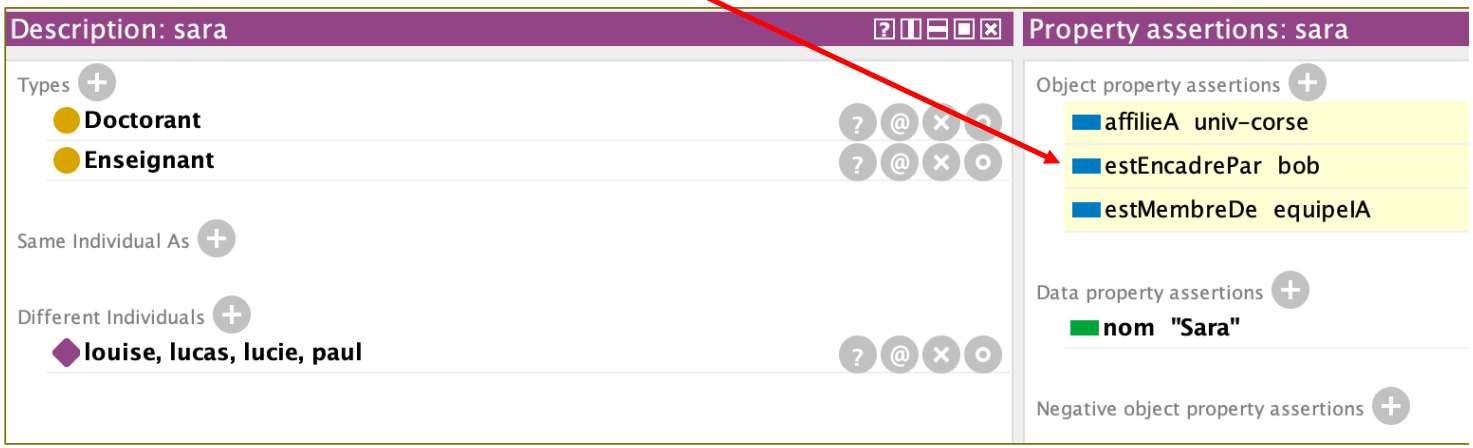
TP guidé ontocampus

Test des inférences

- Lancez le **reasoner (hermiT)**
- Visualisez les individus et les propriétés inférés

Apparaissent en jaune après lancement du reasoner





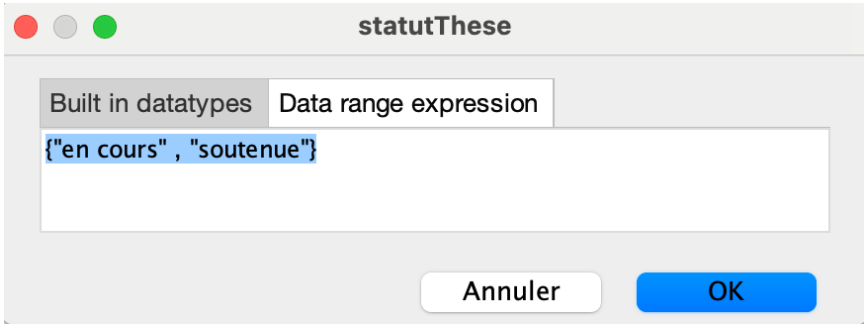
TP guidé ontocampus

Quand les Doctorants deviennent Docteurs !

- Ajoutez une data propertie "**statutThese**" de type String dans la classe Doctorant
- Définissez une expression

  range {"en cours" , "soutenue"}
- Dans la classe Doctorant,
  - Ajoutez la règle suivante dans l'onglet subclass OF

    statutThese only {"en cours" , "soutenue"}



TP guidé ontocampus

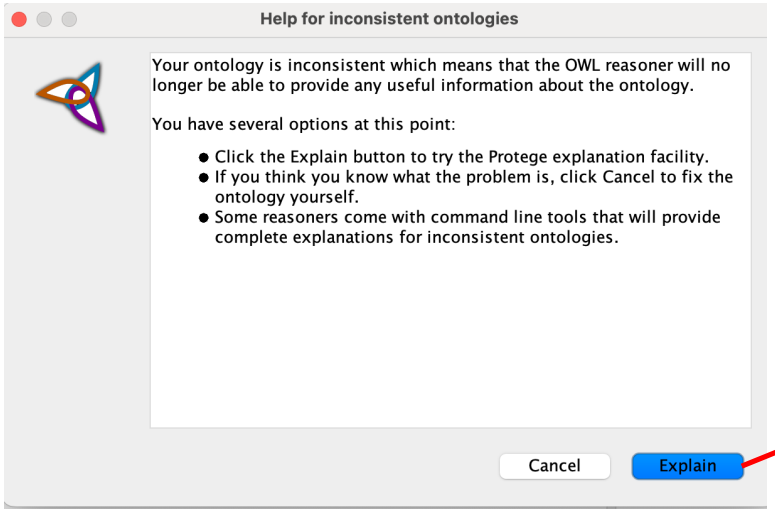
- Ajoutez une classe Docteur qui hérite de Doctorant avec une règle de définition par équivalence :
  - *Doctorant and (statutThese value "soutenue")*
- Définissez la propriété statutThese pour tous les individus doctorants à la valeur "en cours" sauf Lucas qui a déjà soutenu sa thèse: son statutThese="soutenue"
- Lancez le reasoner et vérifiez que lucas apparait bien comme une instance de Docteur

TP guidé ontocampus

- Arrêtez le reasoner
- Modifiez le statutThese de lucie en introduisant volontairement une erreur : statutThese="enCours" (et non "en cours")
- Sauvegardez
- Relancez le reasoner
  - Il devrait détecter

  le problème
- Arrêtez le reasoner et

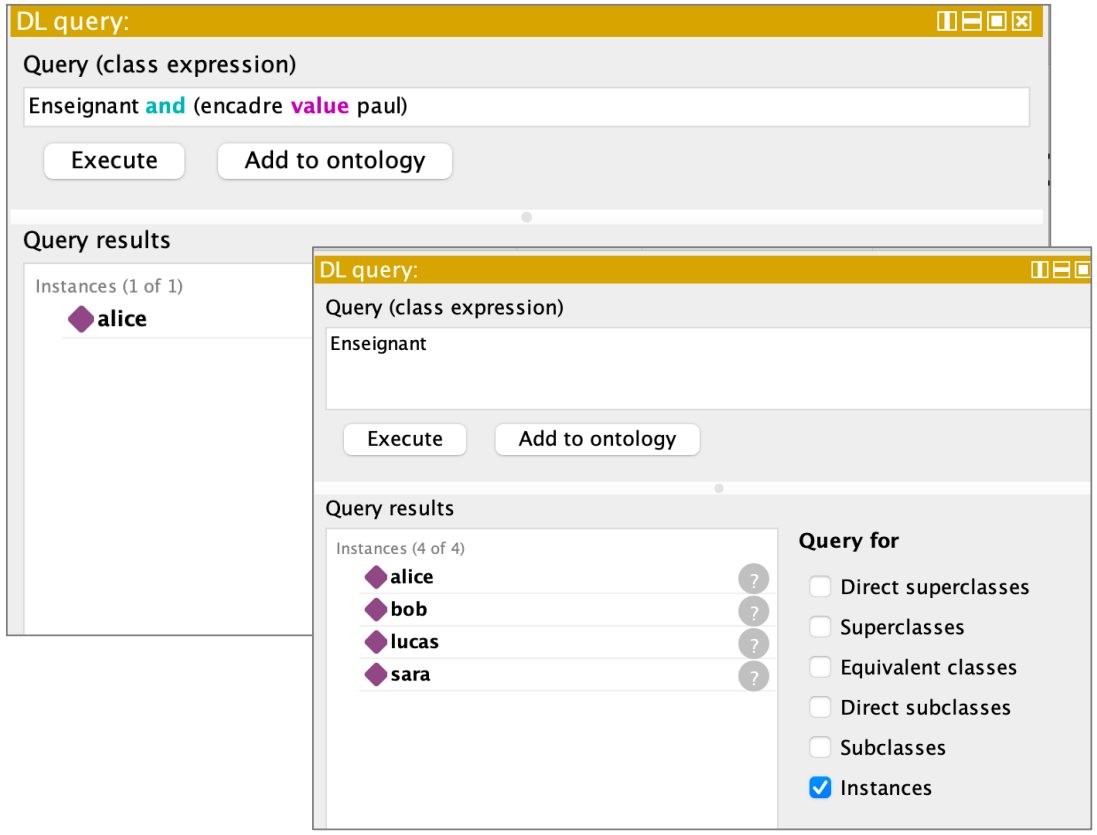
Corrigez l'erreur



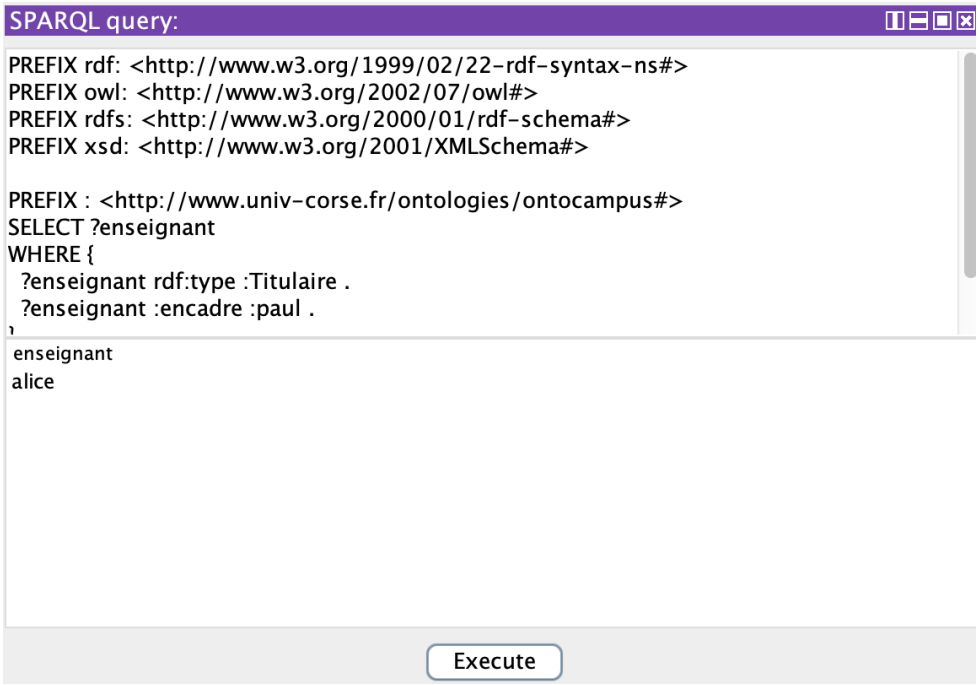
Visualisez l'explication de l'erreur

Interroger une Ontologie dans Protégé

Description Logic DL Query



SPARQL



Ajoutez les fenêtres DL Query et SPARQL Query

DL Query dans Protégé

Objectif : Interroger la base de connaissances à l'aide de la logique des descriptions

Syntaxe inspirée de la logique de description :

- Enseignant and (encadre some Doctorant)
  → Tous les enseignants qui encadrent **au moins un doctorant**
- Personne and not (encadre some Projet)
  → Personnes **n'encadrant aucun projet**
- Projet and (dirige only Professeur)
  → Projets dirigés **uniquement** par des professeurs

Attention

- Fonctionne sous l'**hypothèse du monde ouvert (OWA)**
- Utilise le **raisonneur actif** (HermiT, Pellet, etc)

13

TP guidé ontocampus

Interrogation de la base de connaissance avec DL Query

Exprimez les requêtes suivantes :

a) Lister tous les noms des doctorants
b) Lister les personnes affiliées à l'université de corse
c) Lister les doctorants qui participent à un projet
d) Lister les enseignants qui sont aussi encadr
e) Les projets dirigés par bob
f) Les doctorants encadrés par bob

Interprétation des informations manquantes

Hypothèse de Monde Ouvert

OWA

OPEN WORLD ASSUMPTION

(Description Logics / OWL, ontologies)

Fait présent dans la base	→	✓ **VRAI**
Négation démontrée (explicite)	→	✗ **FAUX**
Fait absent de la base	→	? **INCONNU** (on ne peut pas conclure)

L'absence d'information **n'implique pas la fausseté** : l'énoncé peut être vrai, faux ou simplement inconnu.

Hypothèse de Monde Clos

CWA

CLOSED WORLD ASSUMPTION

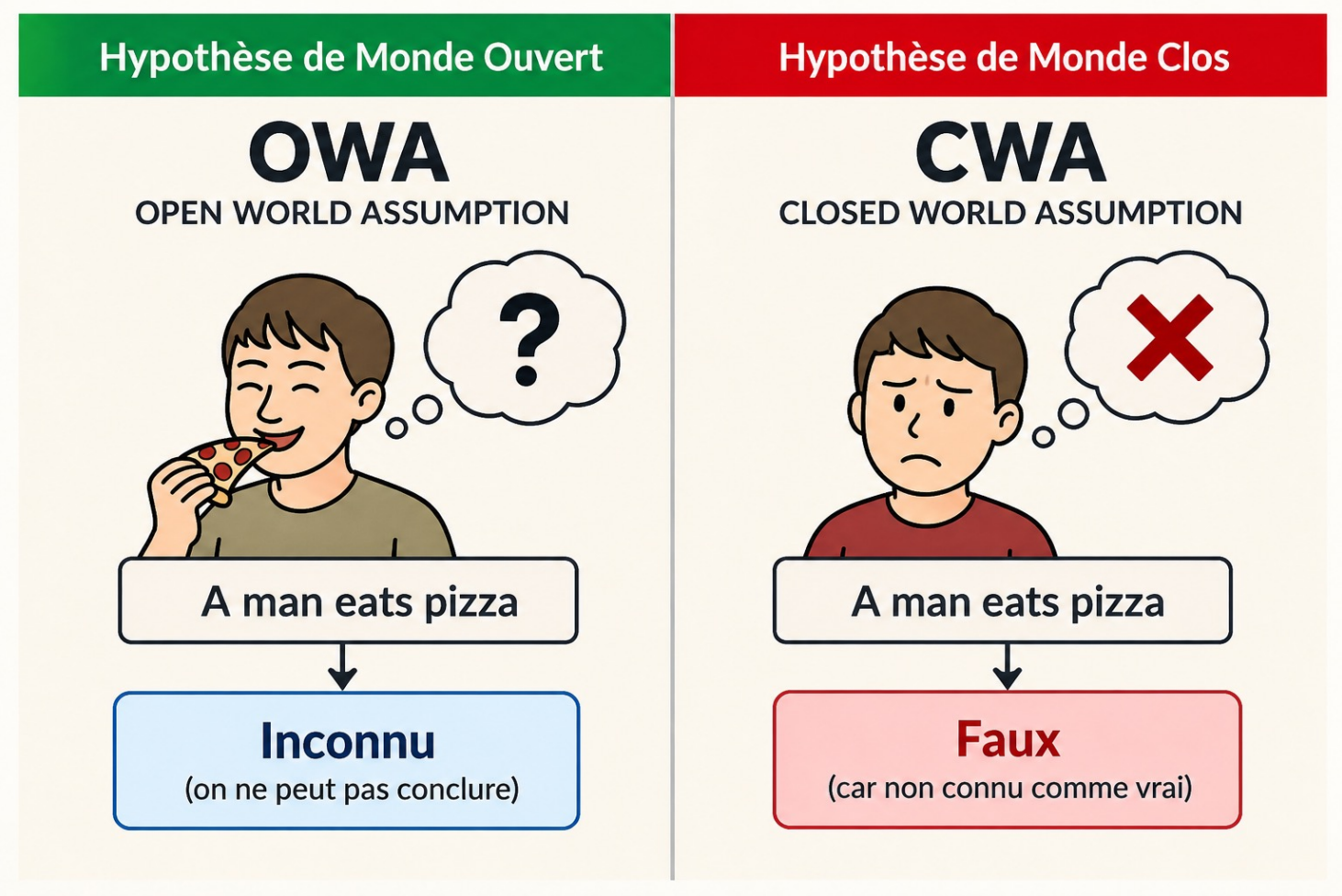
(Bases de données relationnelles / Prolog)

Fait présent dans la base	→	✓ **VRAI**
Fait absent de la base	→	✗ **FAUX** (ce qui n'est pas connu est considéré comme faux)

L'absence d'information est interprétée comme une fausseté :
si aucun fait ne le dit, on considère qu'il n'est pas vrai.

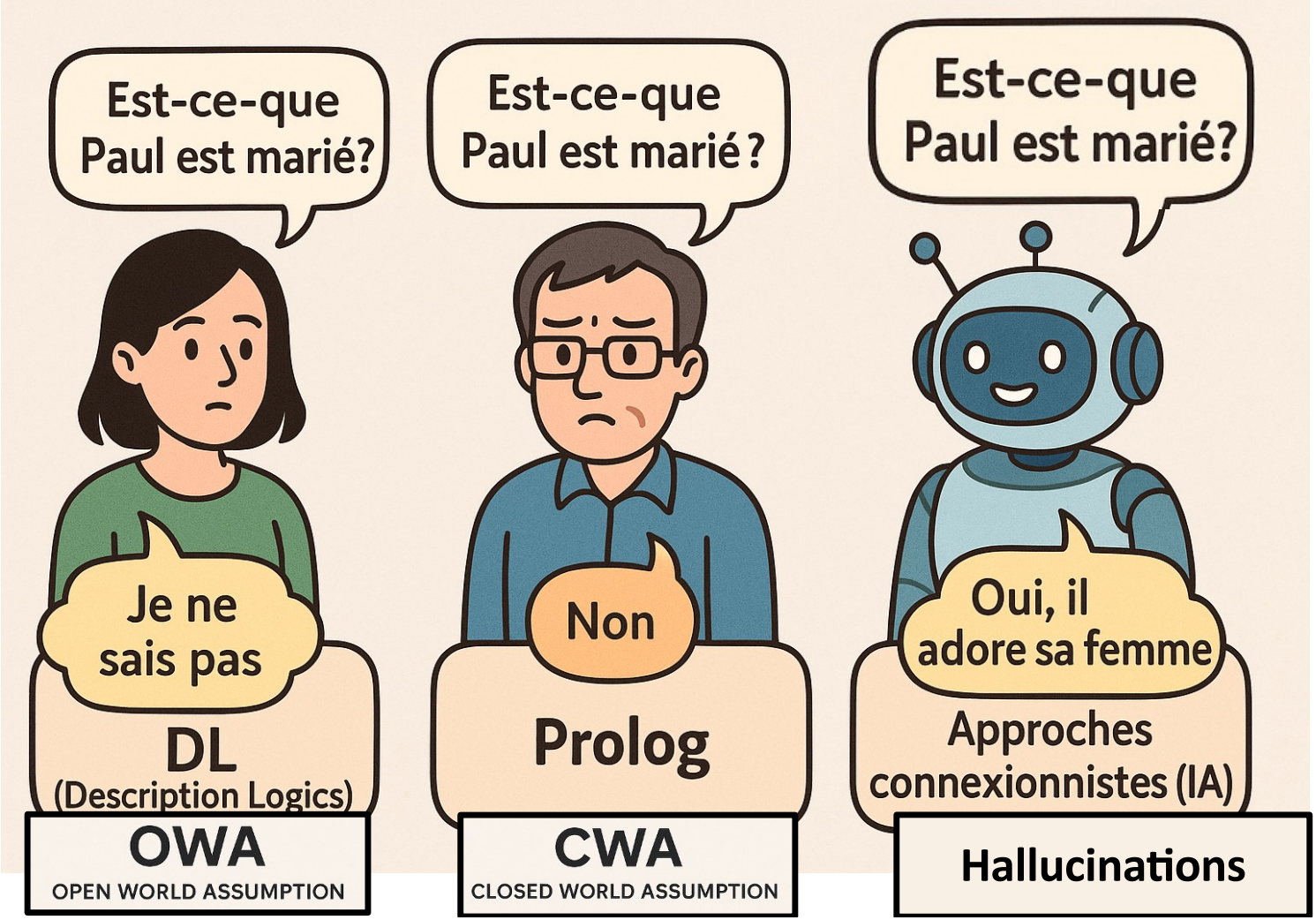
Interprétation des informations manquantes

- **Description Logics**
- Ontologies



- **SparQL**
- Systèmes Experts
- Bases de données

3 approches d'Interprétation des informations manquantes



14

TP guidé ontocampus

Interrogation de la base de connaissance avec DL Query

Exprimez la requête suivante :

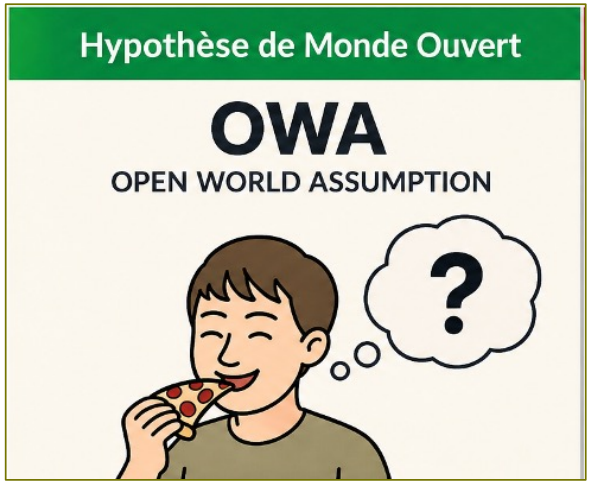
g) Quels sont les encadrants dont on sait qu'ils n'encadrent pas Lucas ?

"*Encadrant and not (encadre value lucas)*"

Que remarquez vous?

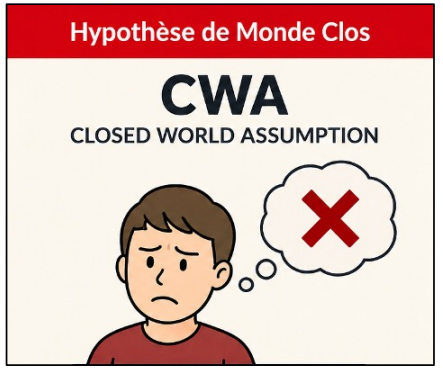
Quel est l'influence de

l'Hypothèse de Monde Ouvert (OWA)?



Langage SparQL : Principe

- Langage standard W3C pour interroger des données RDF/OWL
- Permet d'interroger des **triplets RDF** : sujet – prédicat – objet
  - phrases sujet-verbe-complément
- Chaque requête SPARQL consiste à **chercher des motifs** qui correspondent à ces triplets
- Syntaxe déclarative : SELECT, WHERE, FILTER, PREFIX
- Permet extraction, filtrage, construction et raisonnement



- Très utile pour **extraire des données**, y compris les **littéraux** (ex: noms, dates…)
- Mais SPARQL **ne raisonne pas** : il ne trouve que ce qui est **explicitement** dans le graphe RDF

Structure d'une requête SPARQL

```
PREFIX oc: <http://www.univ-corse.fr/ontologies/ontocampus#>

SELECT ?x ?y

WHERE {
   ?x   oc:aPourNom   ?y  .
}
```

1 PREFIX : donner un nom court à l'IRI

```
PREFIX oc: <http://www.univ-corse.fr/ontologies/ontocampus#>
```

On définit ici le préfixe `oc:` pour représenter l'IRI de notre ontologie.

Ainsi, `oc:aPourNom` est l'abréviation de `http://www.univ-corse.fr/ontologies/ontocampus#aPourNom`

2 SELECT : choisir ce que l'on affiche

```
SELECT ?x ?y
```

On demande d'afficher les valeurs des variables `?x` et `?y` trouvées par la requête.

3 WHERE : décrire le motif de triplets à trouver

`?x` `oc:aPourNom` `?y` .

- `?x` → **sujet** : variable à trouver
- `oc:aPourNom` → **prédicat** : propriété de l'ontologie
- `?y` → **objet** : variable à trouver

On cherche tous les triplets dont le sujet et l'objet sont des valeurs (variables) et dont le prédicat est `oc:aPourNom`.

Exemple

Triplets présents dans l'ontologie

Sujet	Prédicat	Objet
:alice	oc:aPourNom	"Alice"
:bob	oc:aPourNom	"Bob"
:marie	oc:aPourNom	"Marie"

Résultat de la requête

?x	?y
:alice	"Alice"
:bob	"Bob"
:marie	"Marie"

Lire un motif SPARQL : Sujet – Prédicat – Objet

Variables

Le symbole ? introduit une **variable** qui sera liée à un individu (une ressource) par la requête.

?pers → une variable nommée **pers**

Préfixes

Le **symbole** : suivi d'un nom abrégé désigne un IRI défini par un préfixe.

```
PREFIX : <http://www.univ-corse.fr/
 ontologies/ontocampus#>
:Titulaire ≡
<http://www.univ-corse.fr/ontologies/
ontocampus#Titulaire>
```

Un motif de triplet

Chaque ligne du WHERE décrit un triplet de la forme :

Sujet	Prédicat	Objet
?pers	rdf:type	:Titulaire

À retenir

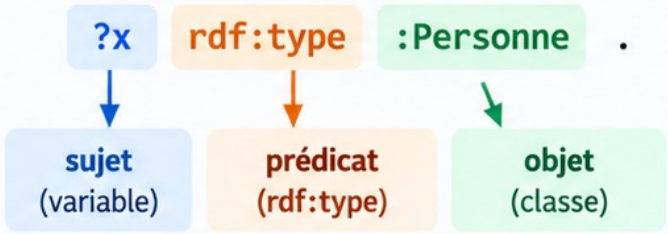
- **?variable** : à trouver dans les données.
- **:Préfixe** : écriture abrégée d'un IRI.
- Chaque ligne du WHERE est un motif de triplet.
- Plusieurs motifs partagent les mêmes variables : SPARQL réalise alors une jointure (les mêmes valeurs doivent satisfaire tous les motifs).

Type d'objet selon le prédicat dans un motif SPARQL

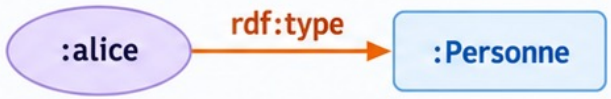
1 Prédicat `rdf:type` (ou `a`)

- Indique la **classe** (le type) d'un individu.
- L'objet est toujours **une classe** de l'ontologie.
- On peut utiliser le raccourci `a` pour `rdf:type`.

Motif SPARQL



Exemple dans l'ontologie

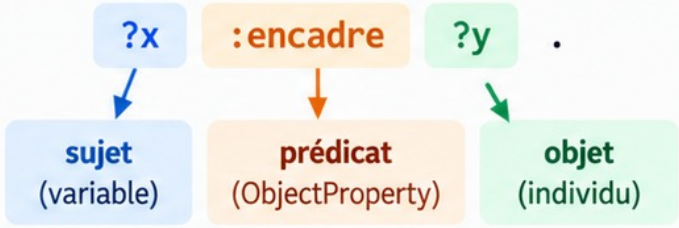


`:alice` est un individu de la classe `Personne`.

2 Prédicat : une ObjectProperty

- Décrit une **relation** entre deux **individus** (ressources).
- L'objet est un **individu** (ressource).

Motif SPARQL



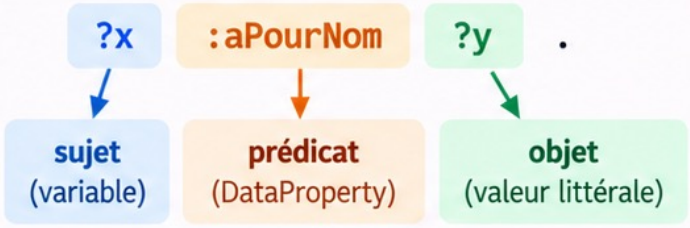
Exemple dans l'ontologie

`:alice` encadre l'individu `:bob`.

3 Prédicat : une DataProperty

- Associe un individu à une **valeur littérale** (texte, nombre, date...).
- L'objet est une **valeur littérale**.

Motif SPARQL



Exemple dans l'ontologie

`:alice` a pour nom la valeur littérale `"Alice"`.

Requête SPARQL simple (1)

Trouver tous les individus de type Personne

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs:<http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX oc:  <http://www.univ-corse.fr/ontologies/ontocampus#>

SELECT ?x
WHERE {
   ?x a oc:Personne .
}
```

Répond à la question
Quels individus possèdent **explicitement** le triplet rdf:type oc:Personne ?

Abbréviation de l'IRI complet
http://www.univ-corse.fr/ontologies/ontocampus#Personne

a est une écriture abrégée de rdf:type

Requête SPARQL simple (2)

🎓👨‍🏫 Trouver tous les titulaires qui encadrent un doctorant

```
PREFIX: <http://www.univ-corse.fr/ontologies/ontocampus#>
SELECT DISTINCT ?pers
WHERE {
    ?pers rdf:type :Titulaire .
    ?pers :encadre ?etudiant .
    ?etudiant rdf:type :Doctorant .
}
```

ET logique entre les motifs tous les motifs doivent être vrais simultanément

La requête cherche les individus ?pers pour lesquels sont **explicitement présents** les trois motifs

Langage SparQL : Opérateurs logiques

Opérateur	Signification	Exemple
OPTIONAL	Clause optionnelle	OPTIONAL { ?x :aPourAge ?age }
UNION	Disjonction (OU)	{ } UNION { }
FILTER	Filtrage (SI)	FILTER(?age > 18)
BIND	Création de variable	BIND(?age * 2 AS ?doubleAge)

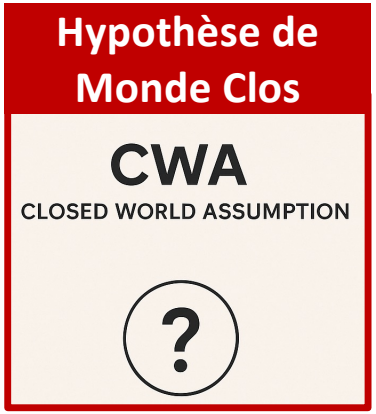
DLQuery VS Sparql : résumé

	🧪 DL Query	🔍 SPARQL
Raisonnement	✅ Oui (via reasoner)	❌ Non (par défaut)
Syntaxe	Logique déclarative (conceptuelle)	Triplets RDF + filtres SPARQL
Orientation	**Sémantique** : concepts et inférences	**Structurelle** : graphe RDF explicite
Extraction de littéraux	❌ Non	✅ Oui (FILTER, OPTIONAL, BIND…)
Expressivité logique	Limitée aux concepts OWL/DL	Très souple, mais sans logique inférentielle
Usage typique	Tester définitions, requêtes conceptuelles	Exporter, filtrer, manipuler des données

TP guidé ontocampus

Interrogation de la base de connaissance avec SPARQL

- Exprimez les requêtes précédentes en SPARQL
  a) Lister tous les noms des doctorants
  b) Lister les titulaires affiliées à l'université de corse
  c) Lister les doctorants qui participent à un projet
  d) Lister les titulaires qui sont aussi encadrants
  e) Les projets dirigés par bob
  f) Les doctorants encadrés par bob
  g) Les doctorants qui n'ont pas encore soutenu
  h) Les titulaires qui n'encadrent pas lucas



Plusieurs requêtes n'ont aucun résultat : expliquez pourquoi ?

- Que constatez-vous au niveau des données inférées ?

Pour aller plus loin dans les règles : le langage SWRL (Semantic Web Rule Language)

- Objectif SWRL :Définir des règles plus expressives
- Inférences complexes impliquant plusieurs individus ou des conditions combinées qui ne peuvent pas être exprimées en OWL seul :
  - Contraintes **temporelles** (currentDateTime, comparaisons),
  - Patrons **n-aires** (rôles, statuts, quantités attachées à une relation),
  - Heuristiques « au moins N » via variables distinctes,
  - Vérifications **éthiques/organisationnelles** (conflits, double rôle, etc)

🔄 Interopérabilité et raisonnement :

- Compatible avec **Drools**, **Jess**, ou moteurs de règles embarqués
- Export possible via l'onglet **SWRLTab** dans Protégé
  (OWL+SWRL → moteur → OWL enrichi)

Règles SWRL : Un Exemple

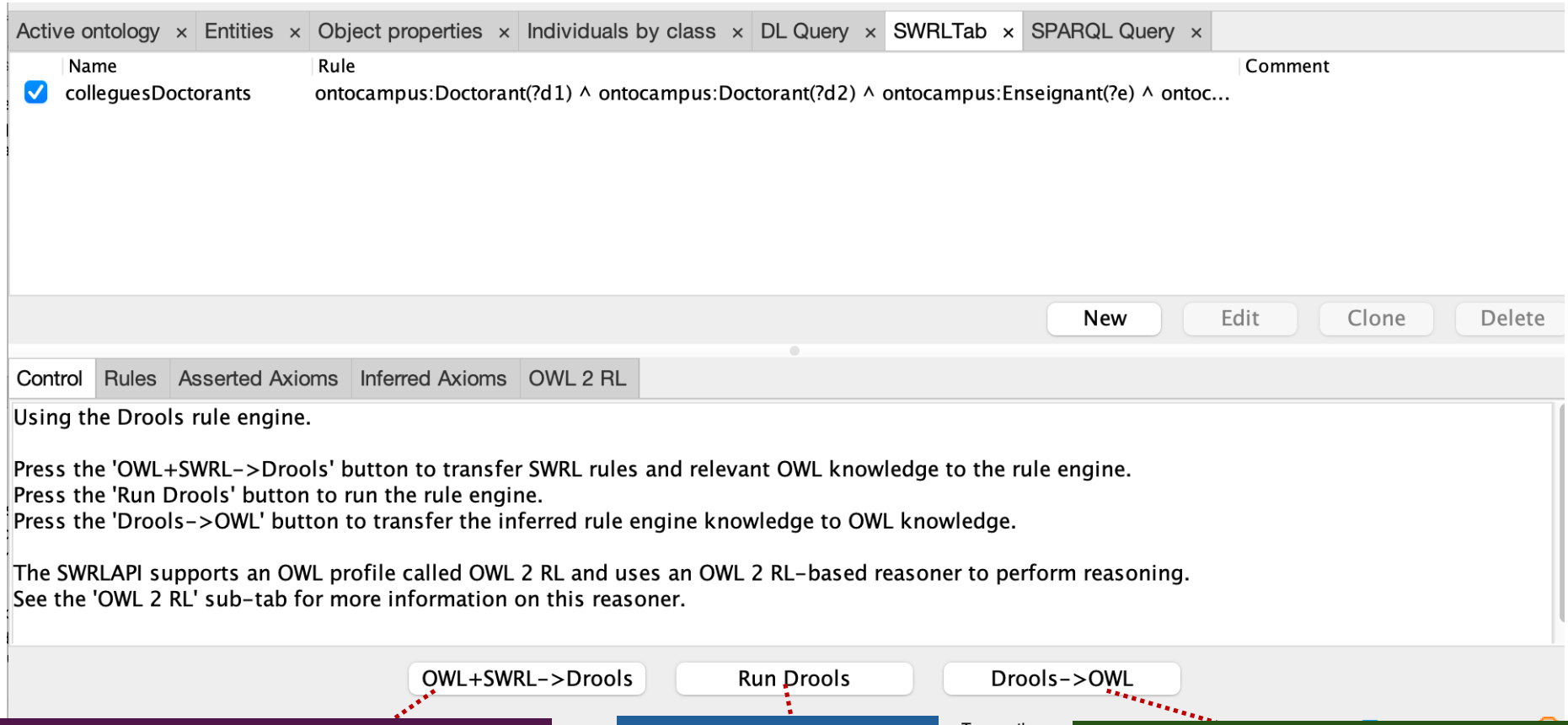
- Deux doctorants sont dits **colleguesDoctorant** s'ils sont encadrés par le même encadrant
- Impossible à exprimer en OWL :
  - relation entre deux individus en fonction d'un autre individu lié à chacun
  - logique triangulaire (si d1 et d2 partagent p)

```
Doctorant(?d1) ^
Doctorant(?d2) ^
Enseignant(?e) ^
encadre(?e, ?d1) ^
encadre(?e, ?d2) ^
differentFrom(?d1, ?d2) ->
aPourCollegueDoctorant(?d1,
?d2)
```

Exécuté par un moteur de règles (similaire au moteur d'inférence des systèmes experts)

CWA
CLOSED WORLD ASSUMPTION

Exemple dans Protégé



1 – Chargement de l'ontologie et des règles dans le moteur Drools

2 - Exécution des règles

3- Transfert des faits inférés dans l'ontologieOWL

16

TP guidé ontocampus

- Ajoutez l'object propertie "*aPourCollegueDoctorant*"
- Définissez la règle SWRL de la page 113

  Doctorant(?d1) ^ Doctorant(?d2) ^ Enseignant(?e) ^
  encadre(?e, ?d1) ^ encadre(?e, ?d2) ^ **differentFrom**(?d1, ?d2) ->
  aPourCollegueDoctorant(?d1, ?d2)

- Visualisez les individus en prenant soin d'arreter le reasoner
- Que constatez-vous?

17

TP guidé ontocampus

- Créez une classe EnseignantSansDoctorant
- Ajoutez un individu Jean dans la classe Titulaire
- Définissez la règle SWRL suivante

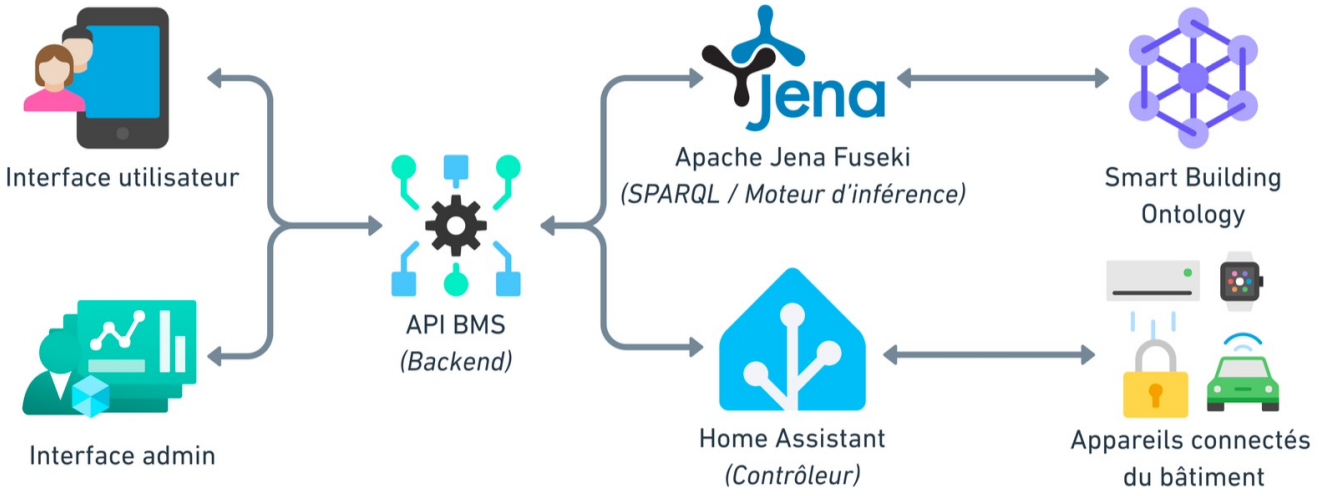
```
Enseignant(?e) ^ not(encadre(?e, ?d))
→ EnseignantSansDoctorant(?e)
```

4 - Exploiter une ontologie dans une application

Un exemple pour introduire des pistes

Building Management System
Exemple d'Exploitation opérationnelle d'une ontologie

- Exportation de l'ontologie vers un serveur Jena/Fuseki
- L'ontologie est interrogeable et modifiable dynamiquement via Jena/Fuseki en SPARQL

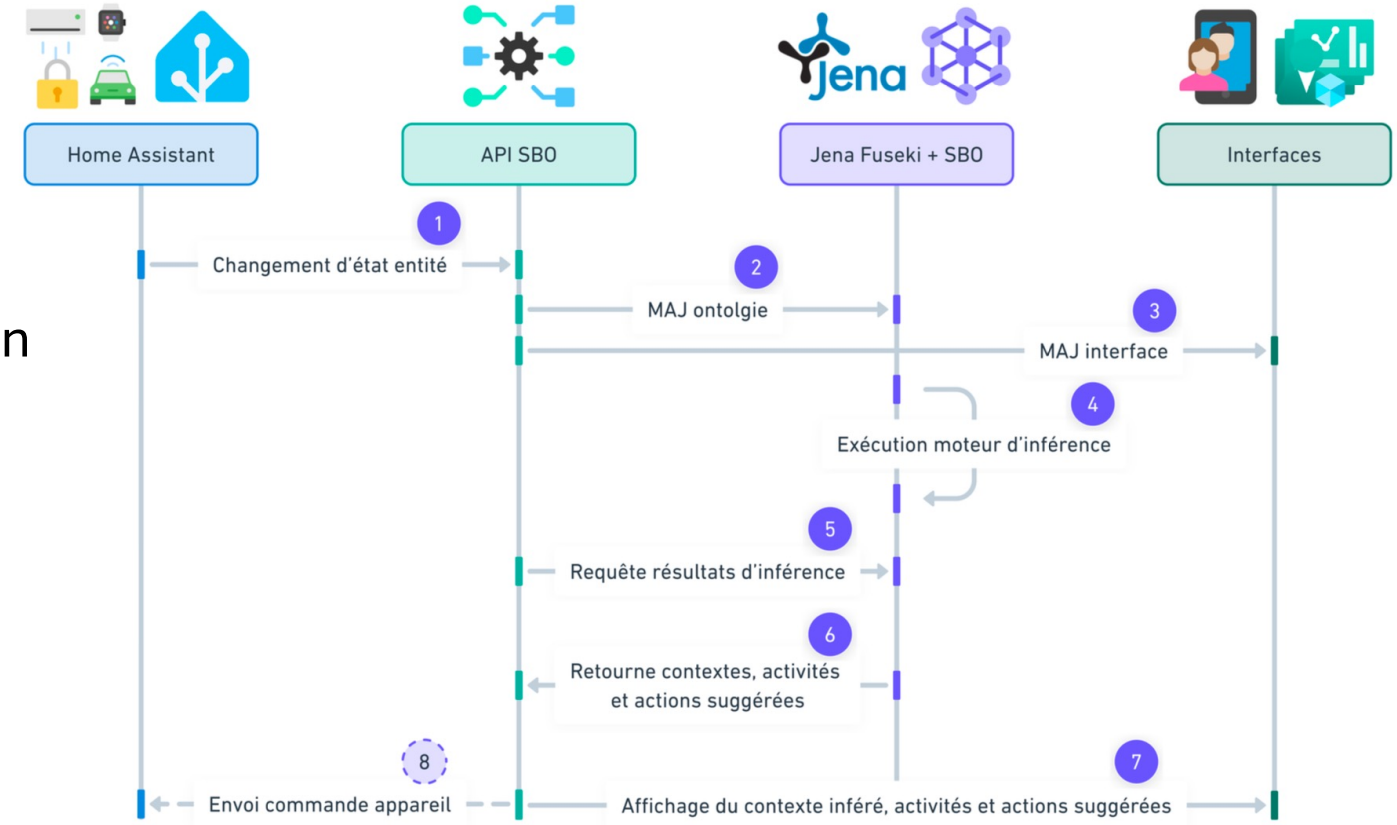


Architecture du Building Management System, avec l'API SBO
au cœur du dispositif et assurant la communication entre les différents acteurs

source : Thèse de doctorat Loris Plasson, Université de corse, Decembre 2025

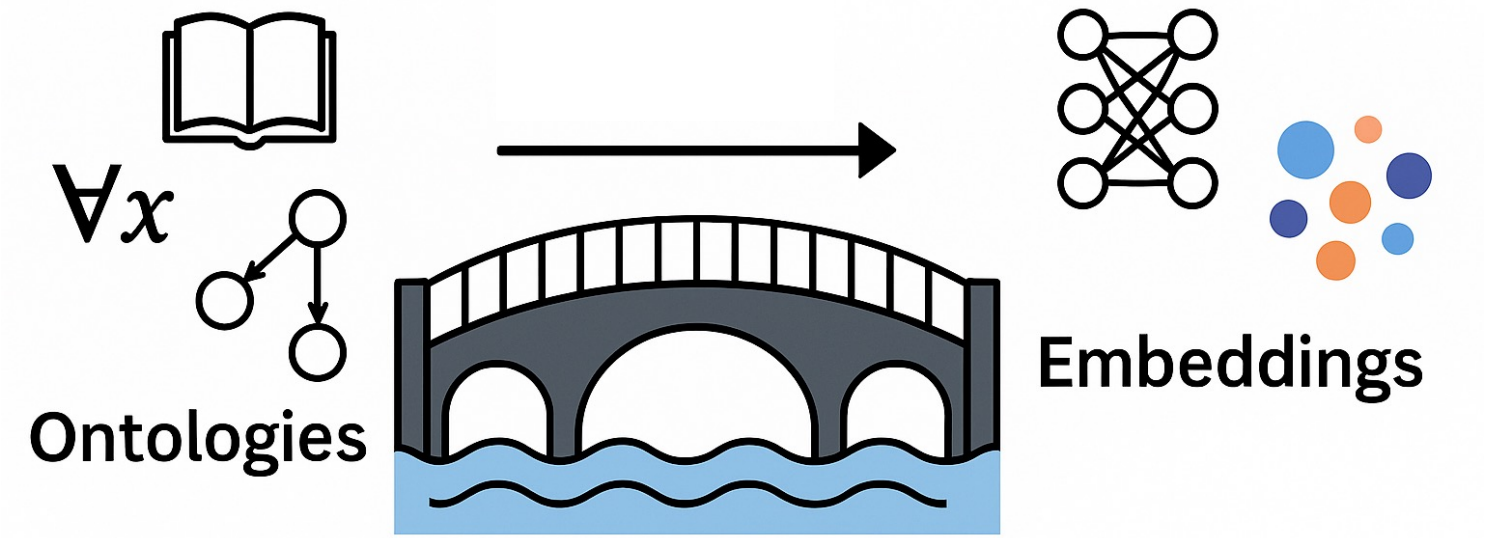
Building Management System
Exemple d'Exploitation opérationnelle d'une ontologie

Scénario d'utilisation de l'API SBO Management pour modifier l'ontologie



source : Thèse de doctorat Loris Plasson, Université de corse, Decembre 2025

Pour conclure avant le connexionisme et les embeddings ...



Perspectives : Couplage LLM - Ontologie

🧠+🧩 Dans un RAG enrichi par une ontologie

L'ontologie permet :

- la structuration explicite du savoir (types, relations, contraintes) ;
- la navigation sémantique raisonnée (via SPARQL, raisonneurs, règles SWRL, etc)

Le LLM peut être guidé ou corrigé par la structure et les inférences fournies par l'ontologie

➡️ Approche IA **neuro-symbolique**

Apports concrets des ontologies au RAG

🔗 RAG sémantique (Semantic Retrieval-Augmented Generation)

- Extraire dynamiquement des **triplés RDF**, définitions ou fragments d'ontologie
- Les injecter dans le **contexte du prompt**
- 👉 Exemple : “Voici la définition formelle d’un *Encadrant* À partir de cela, explique si Alice peut être qualifiée ainsi ”

🛠️ Génération de requêtes

- LLM → génère des requêtes SPARQL ou SWRL à partir de questions naturelles

🧮 Contrôle des hallucinations

- Vérifier les faits énoncés par le LLM via l'ontologie
- Appliquer des contraintes ou des rejets

📘 Explication interactive

Le LLM peut naviguer dans l’ontologie pour répondre à des questions d’apprentissage

Il peut s’aider des *labels*, *commentaires*, *relations hiérarchiques*, etc

🧩 Création ou extension d’ontologie

Le LLM peut proposer de nouvelles classes ou propriétés à partir d’un corpus (texte, documents), mais l’utilisateur doit valider

TP : Création autonome d'une petite ontologie

- Travail en binôme

Université de Corse - Pasquale PAOLI

Diplôme : **M2 DE et DFS**	**2026-2027**

Cours : Représenter le monde en IA

Concevoir une ontologie de projets informatiques

Enseignant : Evelyne VITTORI

Une entreprise informatique souhaite mieux décrire les compétences de ses développeurs et les technologies utilisées dans ses projets. Elle souhaite notamment pouvoir déduire automatiquement certains profils de développeurs et certaines catégories de projets, sans avoir à les déclarer explicitement.
Votre objectif est de concevoir une petite ontologie dans Protégé, de la peupler, puis d'utiliser un raisonneur pour produire ces classifications.

Récupérez le sujet sur l'ENT